FPT SOFTWARE

NVIDIA GR00T によるヒューマノイドロボット G1 の
ピック&プレース制御



プロジェクト報告書

UNITREE G1 × NVIDIA GR00T

お客様
Shimizu Corporation

期間
24/08/2026 –
06/11/2026

バージョン・発行日
v1.0 · 06/11/2026

目次

プロジェクト報告書 — Unitree G1 × NVIDIA GR00T

第 I 部 背景と基盤 5

I.01 概要と成果 5
- I.01.1 研究問い 5
- I.01.2 実験シナリオ 6
- I.01.3 研究目標 7
- I.01.4 研究範囲 7
- I.01.5 研究データ 8
- I.01.6 実施プロセス 8
- I.01.7 実施計画 9

I.02 インフラと性能 11
- I.02.1 機器 & 接続 11
- I.02.2 ソフトウェア & バージョン 12
- I.02.3 推論性能 13
- I.02.4 データ収集性能 13
- I.02.5 学習性能 14
- I.02.6 性能分析 14

第 II 部 課題の調査 16

II.01 モデルの調査 16
- II.01.1 用語集 16
- II.01.2 動作の仕組み 16
- II.01.3 入力データ 17
- II.01.4 出力データ 18
- II.01.5 モデルアーキテクチャ 19
- II.01.6 公表された結果 20
- II.01.7 バージョン比較 21
- II.01.8 実験で使用したチェックポイント 22
- II.01.9 調査の結論 22

II.02 パイプラインの調査 24
- II.02.1 全体フロー 24
- II.02.2 環境の初期化 26
- II.02.3 接続の確立 26
- II.02.4 観測の取得 26
- II.02.5 観測の送信 27
- II.02.6 action チャンクの受信 27
- II.02.7 action チャンクの処理 28
- II.02.8 ロボットの実行 28
- II.02.9 閉ループの実行 29
- II.02.10 ノートブック 12 本と報告書内の位置 29

II.03 動作の調査 31
- II.03.1 G1 の関節構造 31
- II.03.2 順運動学 32
- II.03.3 逆運動学 33
- II.03.4 動力学 33
- II.03.5 関節速度の制限 35
- II.03.6 物理ステップ：動きはどう計算されるか 35
- II.03.7 シミュレーション：物理ステップごとに進む車 37
- II.03.8 制御 tick と物理ステップ 38
- II.03.9 制御 tick の中身 39
- II.03.10 シミュレーション時間と実時間 40

II.04 データの調査 41
II.04.1 課題 1: Visual Gap 41
II.04.2 解決策 1: Domain Randomization 41
II.04.3 課題 2: Sim Data Scarcity 43
II.04.4 解決策 2: Real2sim 43
II.04.5 課題 3: Teleop Bottleneck 43
II.04.6 解決策 3: Demo Generation 44
II.04.7 組み合わせと結論 44

第 III 部 実施シナリオ 46

III.01 シナリオ Apple & Plate 46
III.01.1 Video Showcase 46
III.01.2 シナリオの状況 47
III.01.3 シナリオの目標 48
III.01.4 シナリオのデータ 48
III.01.5 直面した課題 48
III.01.6 提案手法 49
III.01.7 実施プロセス 50
III.01.8 達成した結果 51
III.01.9 結果の分析 52
III.01.10 シナリオの結論 54

III.02 シナリオ Multi Fruit 55
III.02.1 Video Showcase 55
III.02.2 シナリオ設定 56
III.02.3 シナリオの目標 57
III.02.4 シナリオのデータ 57
III.02.5 直面した課題 57
III.02.6 提案手法 58
III.02.7 実施プロセス 60
III.02.8 ファインチューニング 62
III.02.9 得られた結果 62
III.02.10 結果の分析 63
III.02.11 シナリオの結論 65

III.03 シナリオ Hammer & Bin 66
III.03.1 Video Showcase 66
III.03.2 シナリオの状況 67
III.03.3 シナリオの目標 68
III.03.4 シナリオのデータ 68
III.03.5 直面した課題 68
III.03.6 提案手法 69
III.03.7 実施プロセス 70
III.03.8 Fine-tune 71
III.03.9 達成した結果 71
III.03.10 結果の分析 73
III.03.11 シナリオの結論 75

第 IV 部 成果物と検証 76

IV.01 成果物 76
IV.01.1 成果物パッケージの構成 76
IV.01.2 役割別の想定利用者 77
IV.01.3 詳細な内容一覧 77
IV.01.4 別途提供する元データ 80
IV.01.5 ライセンスと利用条件 80

IV.02 環境構築 82
IV.02.1 接続構成 82
IV.02.2 ハードウェアとソフトウェアの要件 83

IV.02.3 ワークステーションへの成果物パッケージの配置 83
IV.02.4 ワークステーションへのクライアントのインストール 84
IV.02.5 GPU サーバーへのサーバーのインストール 86
IV.02.6 納品チェックポイントのサーバーへの配置 87
IV.02.7 接続の設定 89

IV.03 GROOT サーバーの起動 90
IV.03.1 シナリオ別のサーバー設定 90
IV.03.2 サーバーの起動 91
IV.03.3 ワークステーションからの接続確認 92

IV.04 検証の実行 94
IV.04.1 共通の手順 94
IV.04.2 サーバー不要の基本確認 95
IV.04.3 シナリオごとの検証コマンド 96
IV.04.4 結果の読み方と比較 97
IV.04.5 Jupyter での対話実行 97

IV.05 VSS による動画評価 99
IV.05.1 本プロジェクトにおける VSS の役割 99
IV.05.2 サーバーの要件 100
IV.05.3 VSS 3.2.1 のインストール 101
IV.05.4 インストール後の設定とヘルスチェック 102
IV.05.5 3 つの判定方法と 99 エピソードでの結果 103
IV.05.6 2 エピソードでの 3 つの方法の再実行（ステップ 9–11 の動画） 105
IV.05.7 Web UI 106
IV.05.8 発見事項と対応 107
IV.05.9 日常の運用 108

第 V 部 評価と付録 109

V.01 総合評価 109
V.01.1 結果のまとめ 109
V.01.2 全体的な考察 110
V.01.3 得られた教訓 111
V.01.4 今後の方向性 113

V.02 付録 115
V.02.1 用語 115
V.02.2 参考文献 116

I.01 概要と成果

NVIDIA GR00T によるヒューマノイドロボット G1 のピック&プレース制御。

2 か月余りの研究・実験（24/08 – 06/11/2026）の成果です。基盤モデル GR00T を読み解き、NVIDIA Isaac Lab Arena のパイプラインを**独自に再現**し、実機 G1 から **real2sim** データを作成しました。さらに**新しいタスク**について、テレオペによるデータ収集からファインチューニング、評価までの全サイクルを完了しました。

顧客	実施	期間
Shimizu Corporation	**FPT Software**	**24/08/2026 – 06/11/2026**

環境

Isaac Sim / Isaac Lab シミュレーション

1 研究問い

ヒューマノイドロボット **Unitree G1** は、動作を一つずつ事前にプログラムすることなく、**カメラ画像**と**言語指示**だけから自ら物体をつかみ、正しい場所に置く必要があります。本プロジェクトでは、この作業を基盤モデル **NVIDIA GR00T** に任せる方法を研究しました。モデルは画像、指示、関節状態を受け取り、上半身の動作を生成します。すべて Isaac Sim / Isaac Lab のシミュレーション上で実施しています。



3 つの実験シナリオで答えるべき**4 つの研究問い**：

1 問い

GR00T は G1 を制御して物体のピック&プレースができるか？

できる：92 % · 66 % · 73 %

3 シナリオ →

2 問い

新しいタスクを教えるのに、どれだけのデータと工数が必要か？

200 デモ、約 1 時間のファインチューニング

シナリオ 3 →

3 問い

環境がデータ記録時と異なると、結果はどう変わるか？

92 % → 56 % → 0 %

シナリオ 1 →

4 問い

実機ロボットのデータをシミュレーション経由で再び実機に戻せるか（real2sim2real）？

Real2sim：66 % · sim2real：フェーズ 2

シナリオ 2 →

2 実験シナリオ

難易度が段階的に上がる 3 つのシナリオです。NVIDIA の既存パイプラインの**再現**から、シミュレーションに取り込んだ**実機ロボットのデータ**、そして最後に本プロジェクトが独自に構築した**まったく新しいタスク**へと進みます。



シナリオ 1

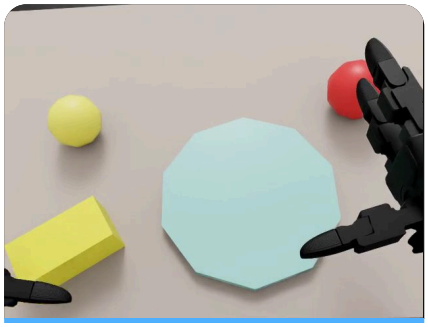
リンゴ → 皿

`"Pick up the apple from the shelf and place it onto the plate on the same shelf next to it."`

目的 NVIDIA のサンプルパイプラインを独自に再現

データ NVIDIA のデモ 208 件、既成モデル

成功条件 リンゴが皿の上にあり、中心から < 8 cm、静止



シナリオ 2

4 種の果物

`"Pick up the green grapes and place it on the plate"`

目的 妨害用の果物の中から指示どおりの果物を選ぶ

データ 実機 G1 データから作成した real2sim 790 エピソード

成功条件 果物が持ち上げられ、皿の上にあり、他の果物を動かしていない



シナリオ 3

ハンマー → 箱

`"put the hammer in the bin"`

目的 まったく新しいタスクを最初から最後まで教える

データ Meta Quest テレオペで自前収集：200–400 エピソード

成功条件 ハンマーが箱の中で静止し、手が離れている

3 研究目標

全体目標

GR00T でロボット G1 のピック&プレースを制御できるかを評価し（問い 1）、モデルを新しいタスクに適用するのに必要なデータ量と工数を明らかにします。全体目標は以下の 6 つの具体的目標に分かれます。

1 基盤

GR00T と制御パイプラインの習得

GR00T N1 から N1.7 までのアーキテクチャとファインチューニング手順を理解し、NVIDIA の既成フレームワークに依存せず GR00T ↔ G1 の処理フロー（クライアント、安全レイヤー、AGILE によるバランス保持、Isaac Lab バックエンド）を自ら再構築します。

II.01–II.03

2 シナリオ 1

NVIDIA の結果の再現

自作パイプライン上でリンゴ → 皿のチェックポイントを実行し、元の環境と比較して、新しいタスクに使う前にパイプラインを検証します。

III.01

3 シナリオ 2

実機ロボットデータの活用

実機 G1 のテレオペデータ（関節角度と動画のみ）を real2sim でシミュレーション用データセットに変換し、指示どおりの果物を選ぶようモデルをファインチューニングします。

III.02　問い 4

4 シナリオ 3

まったく新しいタスクを教える

「ハンマー → 箱」の全サイクル：環境構築、VR ヘッドセットによるテレオペ、データの追加生成、ファインチューニング、評価を行い、必要なデモ数と時間を記録します。

III.03　問い 2

5 VISUAL GAP

画像の差異の実験と分析

モデルが受け取る画像がデータ記録時と異なる場合（シミュレーションのシーン間、実画像とレンダリング画像の間）に成功率がどう変わるかを測定し、低下の要因を特定して、domain randomization などの差を縮める方法を提案します。

シナリオ 1、2　問い 3、4

6 横断

定量的で再現可能な評価

明確な成功条件による閉ループ評価、構成ごとに ≥ 50 エピソード、Wilson 95 % 信頼区間、フェーズ別の失敗分析を行います。各実行で変更する要因は 1 つだけとします。

III.01–III.03

4 研究範囲

すべての実験はシミュレーション内で行います。パイプラインは、後のフェーズで実機ロボットのバックエンドに差し替えられるよう設計しています。

対象範囲

- ✓ Isaac Sim 6.0 / Isaac Lab 3.0 シミュレーション、Isaac Lab Arena 0.2.1

対象外 · 後のフェーズ

- – 実機 G1 ロボットへの展開

- ✓ その場で立つ G1、両腕 + Dex3 ハンド
- ✓ GR00T N1.7-3B：NVIDIA のチェックポイントと自前ファインチューニング
- ✓ データ：NVIDIA のデモ、公開されている実機 G1 データセット（real2sim）、シミュレーション内で自前収集したテレオペ
- ✓ H100 GPU 1 枚でのファインチューニング

- – 移動と操作の組み合わせ
- – モデルのゼロからの学習
- – 実機ロボットでのデータ収集
- – 大規模マルチ GPU 学習

5 研究データ

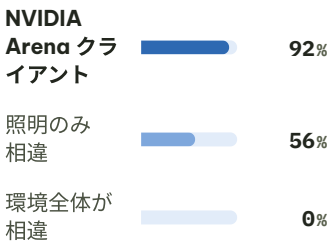
各シナリオの最良結果と、その数値に至るまでの実行結果です。各シナリオをクリックすると詳細を表示します。

シナリオ 1

リンゴ → 皿

92%

NVIDIA の元の環境



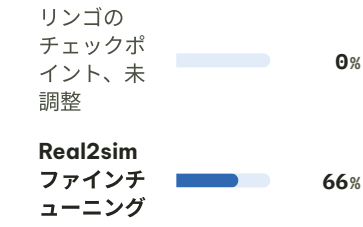
モデルは画像に非常に敏感です。同じチェックポイントでも、環境を変えるだけで結果は 92 % から 0 % まで変化します。

シナリオ 2

4 種の果物

66%

33/50 · 95 %: 52–78 %



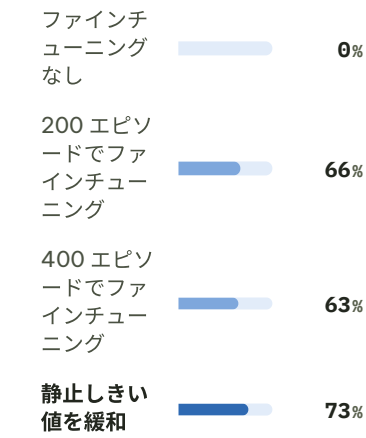
real2sim データでのファインチューニングは必須です。モデルはすべての試行で指示どおりの果物を選びました。

シナリオ 3

ハンマー → 箱

73%

73/100 · 95 %: 64–81 %



自前収集データと 1 回あたり 1–2 時間の学習（mix200 で 61 分、mix400 で 2 時間 15 分）だけで、新しいタスクが 0 % から 73 % に向上しました。

6 実施プロセス

プロジェクトは 3 つのステップで進めます。**共通基盤**を構築し、その上で**3 つのシナリオ**を実行し、評価結果を使って**改善**します。各シナリオは同じ 4 段階（データ源 → 中心作業 → モデル → 閉ループ評価）を経ますが、中心作業が異なります。シナリオ 1 は NVIDIA のデータとモデルを再利用し、GR00T ↔ G1 の**処理フローの研究と書き直し**が中

心です。シナリオ 2 は実機ロボットのデータからシミュレーションデータを自作し、**real2sim** が中心です。シナリオ 3 は新しいタスクをゼロから作り、テレオペデモからの**データ拡張**が中心です。図中の濃い背景のセルがその中心作業を示します。

共通基盤 GR00T N1 → N1.7 の研究 · Isaac Lab Arena · 独立した GR00T ↔ G1 パイプライン（クライアント、安全レイヤー、AGILE、Isaac Lab バックエンド）

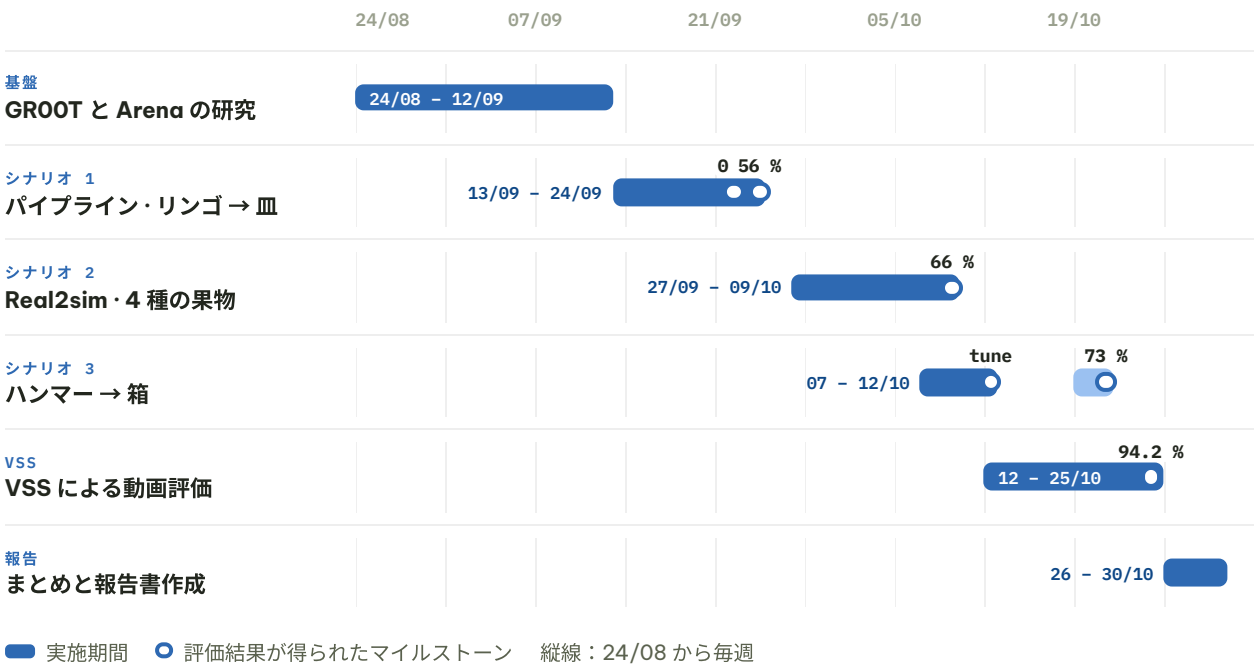
	データ源	中心作業	GR00T N1.7 モデル	閉ループ結果
シナリオ 1 **処理フローの研究と書き直し · リンゴ → 皿**	NVIDIA **Static apple データセット** シミュレーション内の XR テレオペ 208 エピソード	処理フローの書き直し **12 ノートブック → 独立パイプライン** オープンループ検証、デモ再生	既成 **NVIDIA のチェックポイント** リンゴ → 皿向けにファインチューニング済み	ARENA の元の環境 **92 %** 再構築した Galileo 56 % · 自作シーン 0 %
シナリオ 2 **Real2sim · 4 種の果物**	実機 G1 ロボット **テレオペ軌跡 1,095 本** 関節角度 + 動画のみ	REAL2SIM **シミュレーション 790 エピソード** FK → シーン、妨害物体、再生、フィルタリング	FINE-TUNE · H100 **4 種の果物のマルチタスクモデル** 20,000 ステップ · 2 時間 34 分	50 エピソード **66 %** リンゴのチェックポイント 0 % · 取り違え 0 回
シナリオ 3 **データ拡張 · ハンマー → 箱**	META QUEST テレオペ **テレオペデモ 100 件** Isaac Lab 内、自作環境で収集	データ拡張 **mix200 · mix400** テレオペ 100 + 生成 100。mix400 はテレオペ 200 を追加	FINE-TUNE · H100 **ハンマー → 箱モデル** mix200 約 1 時間 · mix400 約 2 時間 15 分	100 エピソード **73 %** 未調整 0 % → 66 % → 73 %

↺ **改善ループ** フェーズ別の失敗分析 → 環境または評価方法の修正、データの追加 → 再学習と再評価。例：Galileo シーンの再構築（シナリオ 1）、カメラが毎ステップレンダリングされない不具合の修正（シナリオ 2）、mix200 → mix400 および静止しきい値 0.01 → 0.02（シナリオ 3）。

7 実施計画

プロジェクトは同じ基盤の上で**3 つのシナリオを順に**実施し、各シナリオは約 2 週間です。終盤の 2 週間（12–25/10）は VSS によるエピソード動画の評価（IV.05 節）を、シナリオ 3 の終盤と並行して行い、最後の 1 週間（26–30/10）でまとめと報告書の作成を行います。下図は実施期間と結果が得られたマイルストーンを示します。各

シナリオの詳細は III 章の各節にあります。



I.02 インフラと性能

2 台のマシン、1 台の VR ヘッドセット – そしてシステムを制約する数値。

シミュレーション、テレオペ、評価は**1 台のワークステーション**で実行し、学習とモデル実行は遠隔の**H100 GPU サーバー 1 台**で行います。本節では、使用した構成、各マシンに必要なソフトウェア、および計測した性能指標を示します。特に**モデルの遅延**は、実機ロボットへ移行する際の最も重要な制約です。

ワークステーション
RTX 5090 · Ryzen 7 9800X3D · 64 GB

サーバー
1× H100 80 GB · 188 GB

シミュレーション
Isaac Sim 6.0.1 · Isaac Lab 3.0-beta2

1 機器 & 接続

ワークステーションが中心となり、VR ヘッドセットからの操作を受け取り、ロボットのシミュレーションを実行し、観測を GPU サーバーへ送って動作シーケンスを受け取ります。

テレオペ
Meta Quest ヘッドセット

オペレーターがシミュレーション内でロボットを操作し、デモを作成します。

機器	ヘッドセット + コントローラー 2 本
接続	USB 3 ケーブル
プロトコル	OpenXR · CloudXR

USB 3 CloudXR

ワークステーション · 中心
シミュレーション & 制御

Isaac Sim、Arena、制御クライアント、バランス制御器、評価、データ記録を実行します。

GPU	RTX 5090 · 32 GB
CPU	Ryzen 7 9800X3D · 8 コア
RAM / SSD	64 GB · NVMe 1 TB
OS	Ubuntu 24.04 LTS

SSH tunnel ZeroMQ

GPU サーバー · 遠隔
学習 & モデル実行

GR00T をファインチューニングし、ワークステーションへ動作シーケンスを返す policy server を実行します。

GPU	1× H100 · 80 GB
RAM	188 GB
ドライバー	575.57
利用形態	複数プロジェクトで共用

次フェーズ

実機 Unitree G1 ロボット

Unitree SDK2 経由でクライアントから関節指令を受け取ります。現時点ではロボットはシミュレーション内でのみ動作しています。

関節	43 · Dex3 ハンド
カメラ	ロボット頭部、640×480

2 ソフトウェア & バージョン

各列は 1 台のマシンのソフトウェア層で、基盤（下）からプロジェクトのアプリケーション（上、強調表示）の順に並べています。

ワークステーション

シミュレーション

g1gr00t クライアント + AGILE
numpy, pyzmq, msgpack, opencv, onnxruntime — `Python 3.10+`

Isaac Lab Arena
Docker 29 で実行、ハンマータスク用の専用ブランチ — `0.2.1`

Isaac Lab
Isaac Sim 6.0 の公式な組み合わせ — `3.0.0-beta2`

Isaac Sim
Python 3.12、ローカルにインストール — `6.0.1`

NVIDIA ドライバー + CUDA
Isaac Sim 6.0 にはバージョン 580 で十分、Isaac Sim 6.1 では ≥ 595 が必要になる予定 — `580 · 12.6`

Ubuntu 24.04 LTS
オペレーティングシステム — `LTS`

GPU サーバー

モデル

GR00T N1.7-3B + checkpoint
ZeroMQ policy server、checkpoint は 1 つあたり 12.6 GB — `3B`

Cosmos-Reason2-2B
視覚–言語バックボーン、Hugging Face から取得 — `2B`

Isaac-GR00T
uv による Python 環境、Arena で検証済みの commit に固定 — `4b1dca9`

Linux + NVIDIA ドライバー — `575.57`

実機ロボット

次フェーズ

g1gr00t クライアント
そのまま再利用し、実機用バックエンドのみ追加 — `Python 3.10+`

Unitree SDK2
unitree_sdk2py パッケージ、設定に記載済み — `–`

ロボット搭載コンピューター
またはロボットの近くに置くコンピューター

その他：notebook とテスト用に Jupyter、pytest、pandas、	Hugging Face 上で NVIDIA のモデルライセンスへの同意が必要	本フェーズでは未実装です。

3 推論性能

ロボットが 1 秒間に何回指令を受け取るか、モデルを 1 回呼ぶと何ステップ分の動作が得られるか、モデルの応答にどれだけ時間がかかるかを示します。

制御周波数
50 Hz
物理 200 Hz、1 tick あたり 4 ステップ · real2sim 20 Hz

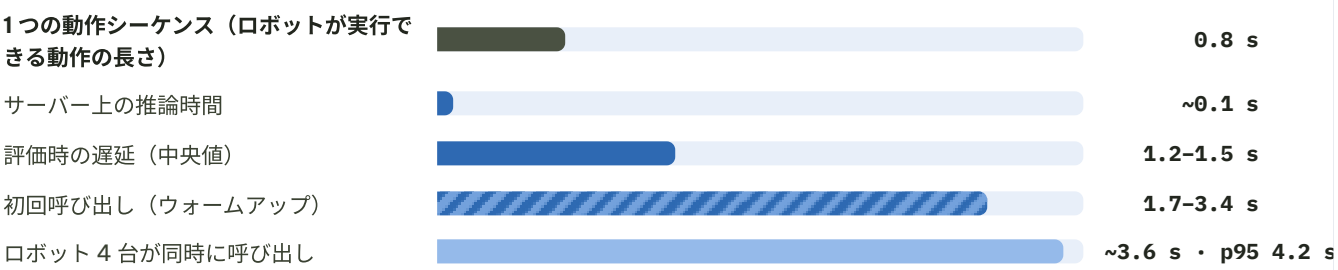
モデル呼び出し 1 回あたり
40 ステップ
= 0.8 s の動作 · real2sim は 16 ステップ、うち 8 ステップを実行

サーバー上の推論
~100 ms
H100 上でモデルが 40 ステップのシーケンスを計算

モデルの遅延
~1.2 s
評価時の 1 回あたりの中央値、SSH tunnel 経由で H100 へ

時間予算：動作時間とモデル遅延の比較

秒 · 目盛 0–4 s



モデルの応答は、ロボットが 1 つのシーケンスを実行し終える時間より**遅い**状態です。シミュレーションでは応答を待つ間シミュレーションが停止するため問題ありませんが、実機では次の呼び出しを先行して行い、GPU をロボットの近くに置かない限り、シーケンスの合間でロボットが止まってしまいます。モデル自体の計算は ~100 ms のみで、残りの遅延の大部分は SSH tunnel 経由の通信とデータのシリアライズにあります。

4 データ収集性能

Meta Quest ヘッドセットと CloudXR でハンマータスクのデモを記録した際に計測しました。

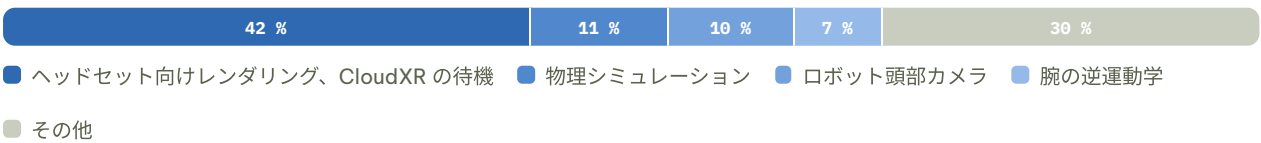
ヘッドセット向けレンダリング
42 %
1 ループの中で最大の割合、CloudXR の待機を含む

物理シミュレーション
11 %
ロボット頭部カメラ 10 %、逆運動学 7 %

RAM
~12 GB
記録中は ~15 MB/s で増加、デモ保存時に解放

テレオペ 1 ループの時間内訳

ハンマータスクのデモ記録時に計測



ヘッドセット向けレンダリング、CloudXR の待機　物理シミュレーション　ロボット頭部カメラ　腕の逆運動学　その他

テレオペ時のボトルネックはシミュレーションではなく、ヘッドセットへの映像伝送です。プロセスは ~12 GB の RAM を使用し、記録中は ~15 MB/s で増加します（カメラ画像のバッファで、デモ保存時に解放）。

5 学習性能

H100 GPU 1 枚での GR00T N1.7 のファインチューニングと、学習データの準備です。

ファインチューニング時間
61 分
10,000 ステップ · 0.36 s/ステップ · batch 32

GPU メモリ
≈ 53 GB
LLM は凍結、vision・projector・diffusion を調整

チェックポイント
12.6 GB
重みのみ、optimizer を含めると ≈ 27 GB

指標	値	備考
学習速度	0.36 s / ステップ	Batch 32、1× H100（GPU が他のジョブに占有されている場合は 2.9 s/ステップ）
学習時の GPU メモリ	≈ 53 GB	LLM は凍結、vision・projector・diffusion を調整
学習 10,000 ステップ	61 分	Loss 1.27 → 0.020
Checkpoint サイズ	12.6 GB	重みのみ、optimizer の状態を含めると ≈ 27 GB
拡張データの生成	100 デモ / 17 分	133 回実行し、成功した 100 回を保持
データ容量	131 MB / 13 GB	ハンマー 200 エピソード、LeRobot 形式 / 画像付きの元の HDF5 形式
自動テスト	84 unit test	シミュレーション内テスト：~3.5 分

6 性能分析

上の 3 つのカードを合わせて読み、各工程で時間とリソースが実際にどこで消費されているか、それが次フェーズにとって何を意味するかを整理します。

推論
ボトルネックはモデルではなく通信
モデルの計算は ~100 ms のみですが、1 回の呼び出しには ~1.05–1.3 s かかり、1 つの動作シーケンス（0.8 s）より長くなります。差の大部分は SSH tunnel とデータのシリアライズにあります。ロボット 4 台が同時に呼び出すと、サーバーが順番に処理するため遅延は ~3.6 s（p95 4.2 s）に増えます。

データ収集
ボトルネックはヘッドセットへの映像
レンダリングと CloudXR の待機が各ループの 42 % を占め、物理シミュレーション（11 %）の約 4 倍です。記録中は RAM が ~15 MB/s で増えるため、長いセッションを続けて記録するのではなく、デモごとに定期的に保存する必要があります。

学習
共用でなければ H100 1 枚で十分
ファインチューニング 10,000 ステップは 61 分で、GPU メモリは ≈ 53 GB と H100 1 枚に収まります。GPU が他のジョブに占有されると、速度は約 8 倍遅くなります（2.9 s/ステップ 対 0.36 s/ステップ）。

改善の方向性：GPU サーバーをロボットの近く、または同じローカルネットワーク内に置き、次のシーケンスを先行して呼び出してロボットが止まらないようにする。データ収集時はヘッドセットに有線接続を使う。ファインチューニングには専用の GPU を確保する。

II.01 モデルの調査

GR00T N1 と本プロジェクトで使用する N1.7。

GR00T N1 はヒューマノイドロボット向けのオープンな VLA モデルで、**理解**を担う視覚–言語モデルと、**行動**を担う Diffusion Transformer で構成されます。本プロジェクトでは後継版の **GR00T N1.7-3B** を使用します。本節では、モデルが何をするか、何で構成されるか、効果はあるか、プロジェクトでどう適用したかを順に説明します。

論文	使用モデル	キーワード
arXiv:2503.14734 (03/2025)	GR00T N1.7-3B	VLA · flow matching · data pyramid

1 用語集

Token
画像や文章を符号化した後の小さな断片です。モデルは元の画像や文字ではなく、トークン列を読み込みます。

VLA (Vision-Language-Action)
画像を見て指示文を読み、ロボットの行動を出力するモデルです。

System 2 / System 1
GR00T の理解部（低速）と行動部（高速）です。

Action / アクション列
1 つのアクションは 1 ステップ分の目標関節角の組で、アクション列はこれからの動作区間です。

Flow matching
アクション列の生成方法です。ランダムノイズから始め、少しずつ修正して動作にします。

Embodiment
ロボットの機体の種類（G1、GR-1…）です。種類ごとに専用のエンコーダを持ちます。

Pre-train / fine-tune
大規模な汎用データで事前学習すること / 特定のロボット・タスク向けに追加で調整することです。

Checkpoint
学習済みモデルの重みで、再利用のためにファイルとして保存したものです。

Teleop
人がロボットを遠隔操作（ここでは VR ヘッドセットを使用）してデモを記録することです。

2 動作の仕組み

以下の動画は、GR00T のような **Vision-Language-Action** モデルがカメラ画像と指示文をどのように動作に変換するかを示します。右側の各ステップをクリックすると、動画の該当箇所に移動します。各ステップには本プロジェ

クトでの適用方法を記載しています。



7 ステップ · クリックで動画を移動

画像と指示文から動作まで

0:00 IMAGE TOKENS

カメラ画像を画像トークンに変換

画像を小さなパッチに分割し、視覚エンコーダ（画像–キャプションのペアで学習）に通した後、言語モデルの空間に射影します。

プロジェクト：G1 頭部カメラの画像 640×480。GR00T N1.7 は SigLIP-2 エンコーダを使用します。

0:12 TEXT TOKENS

指示文をテキストトークンに変換

"pick up the dessert item" のような指示文を単語の断片に分割します。

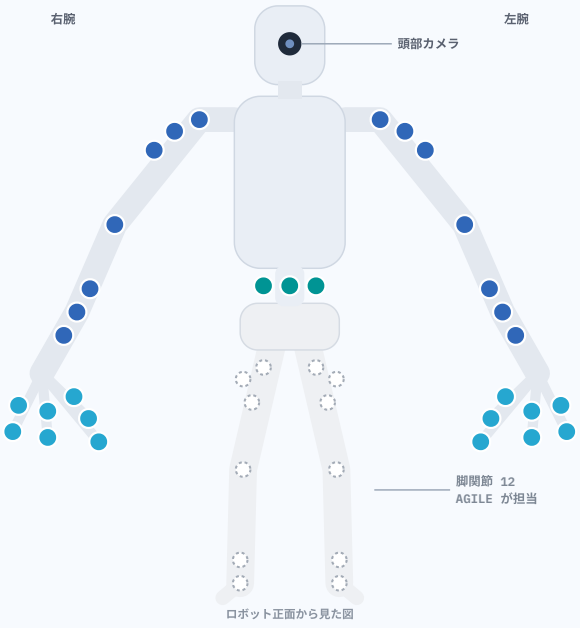
プロジェクト：例："put the hammer in the bin"。指示文はデータ記録時と一字一句同じである必要があります。

VISION-LANGUAGE MODEL

3 入力データ

呼び出しごとに、モデルは**現在の時点のみ**の 3 種類のデータを受け取ります：カメラ画像 1 枚、上半身の関節角、指示文です。過去のフレームの履歴は受け取らず、12 の脚関節も見ません（脚は AGILE が担当）。

1 頭部カメラ画像 640×480、現在のフレーム	**31** 上半身の関節角 腕、指、腰	**1** 指示文 エピソード中は不変



状態 · G1 ロボット

青い点：モデルに送る 31 関節 · 白い点：AGILE が制御する 12 の脚関節



ロボットの視点 · 頭部カメラ

シナリオ 1 の最初のフレーム

送信する指示文

```
"Pick up the apple from the shelf and place it onto the plate on the same shelf next to it."
```

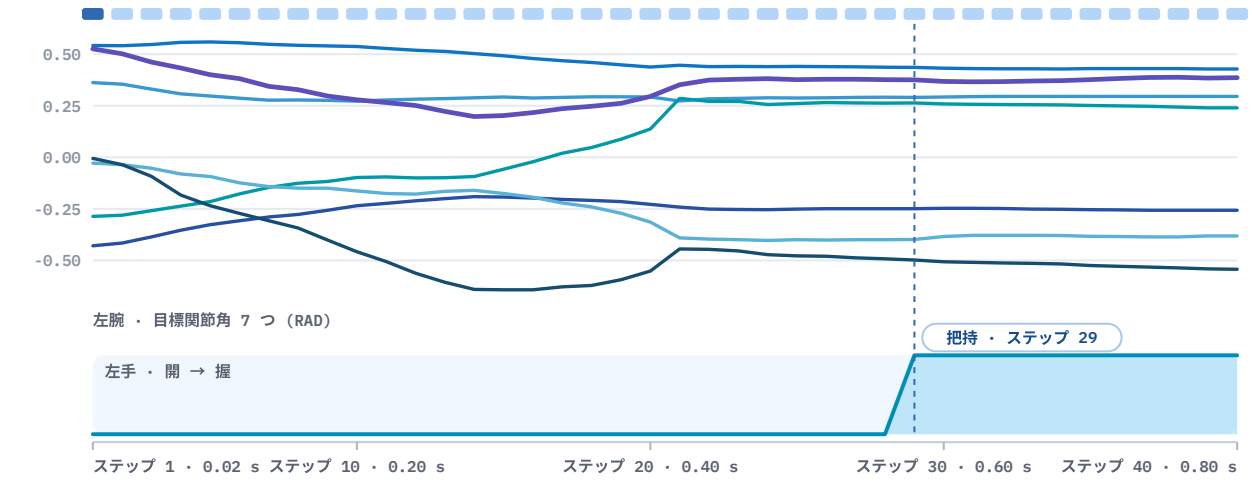
> **本プロジェクトでは：**シナリオ 1 と 3 は上記の `ego_view` カメラを、シナリオ 2（real2sim）は同じ 31 関節で `rs_view` カメラを使用します。

4 出力データ

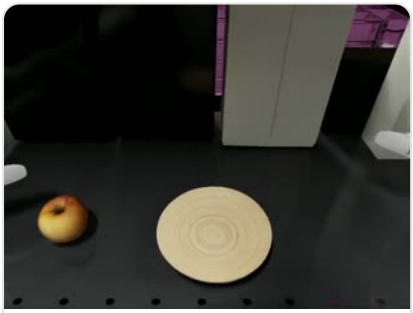
呼び出しごとに、モデルは近い将来に向けた **40 ステップの列**を返し、各ステップは 35 個の値を持ちます。ロボットは 20 ms のティックごとに 1 ステップを実行するため、1 つの列で次の列が必要になるまでの 0.8 s 分の動作をまかなえます。

40	**35**	**1**
列あたりのステップ数 1 ステップ 20 ms = 0.8 s の動作	**ステップあたりの値** 関節角 31 + 脚への指令 4	**呼び出しあたりの列数** 列が尽きる前に再度呼び出し

例 · リンゴ → 皿デモの 40 ステップ列 1 つ（EPISODE 0、0.4 秒から）



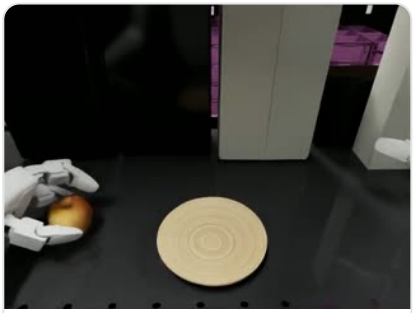
肩 pitch　肩 roll　肩 yaw　肘　手首 roll　手首 pitch　手首 yaw　左手の握り



ステップ 1 · 手がリンゴに向かう



ステップ 21 · 手がリンゴのすぐそばに到達



ステップ 40 · リンゴを把持

これはデモデータに記録されたアクション列で、モデルが返す列と同じ形式です。オープンループ検証では、同じ時点に対してモデルが予測した列とこの列のずれは、動作の振幅より 10–70 倍小さくなりました。

1 ステップ · 35 個の値

左腕	右腕	左手	右手	腰	移動	高さ
7	7	7	7	3	3	1

目標関節角 31 → そのまま関節へ　　指令 4 → AGILE へ渡す

絶対角

31 個の関節値は目標角であり、前ステップからの変化量ではありません。使用前に、クライアントは関節の可動範囲とティックごとの速度上限でクランプします（II.02、カード 7）。

脚への指令

モデルは 12 の脚関節を直接制御しません。移動指令と腰の高さ（指令値 0.72 m）のみを出力し、AGILE がそれを基にバランスを保ちます。本プロジェクトの 3 シナリオはいずれもその場で静止したままです。

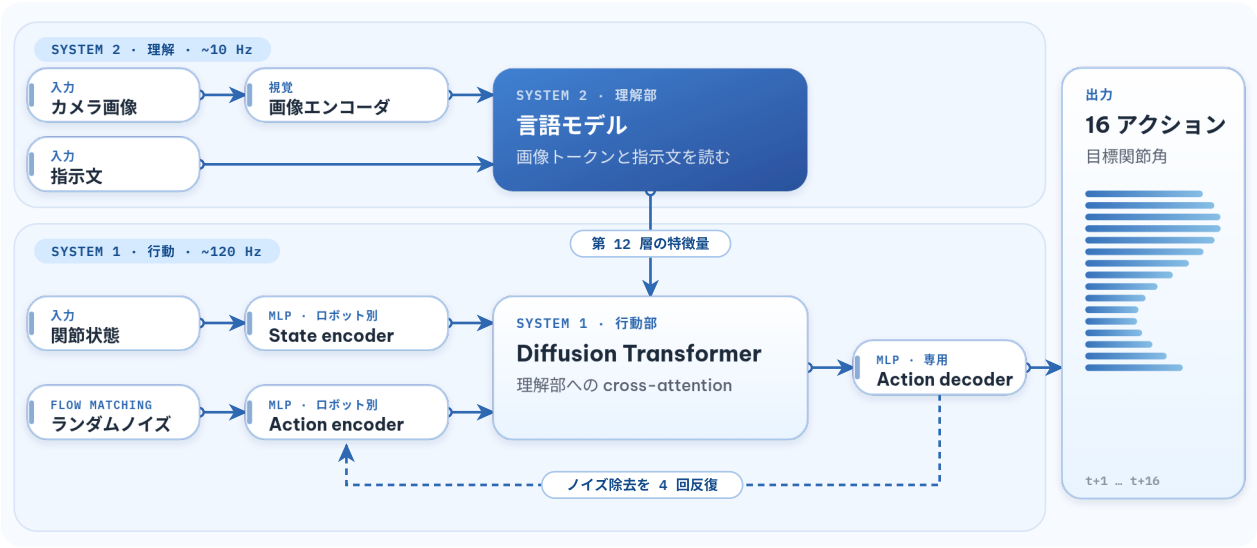
本プロジェクトでは：シナリオ 1 と 3 は上記のとおり 50 Hz の 40 ステップ列を使用します。シナリオ 2 は腰を固定するため脚への指令がなく、20 Hz の 16 ステップ列（同じく 0.8 s）、各ステップ 31 個の値で、呼び出しごとに 8 ステップを実行します。

5 モデルアーキテクチャ

理解部（論文では **System 2**）は視覚–言語モデルで、毎秒約 10 回と低速で動作します。行動部（**System 1**、動画では「action expert」）は理解部の出力とロボットの現在の関節角を受け取り、はるかに高速に動作します。図の各ブロックをクリックすると、その役割が表示されます。

GR00T N1 のデータフロー

図 · 数値は論文より



SYSTEM 2 · 理解部

言語モデル

画像トークンと指示文を同時に読み込み、シーンと要求を理解します。第 12 層の特徴量を行動部に送ります。毎秒約 10 回動作します。

本プロジェクトでは：N1.7 は Cosmos-Reason2-2B を使用します。この部分はファインチューニング時に凍結されます。

本プロジェクトでは：GR00T N1.7 はこの二部構成を維持し、理解部をより新しいモデルに置き換えています（カード 7）。入力は G1 頭部カメラの画像と上半身 31 関節の角度です。3 つの MLP ブロック（state encoder、action encoder、action decoder）はロボットの種類ごとに専用で、G1 をモデルに導入する際に学習が必要な部分です。

総パラメータ数	理解部	速度
22 億	13.4 億	10 · 120 Hz

GR00T N1、論文の版

パラメータの大部分は視覚–言語モデルにある

理解部 ~10 Hz、制御ループ ~120 Hz

技術的詳細 · 論文 –

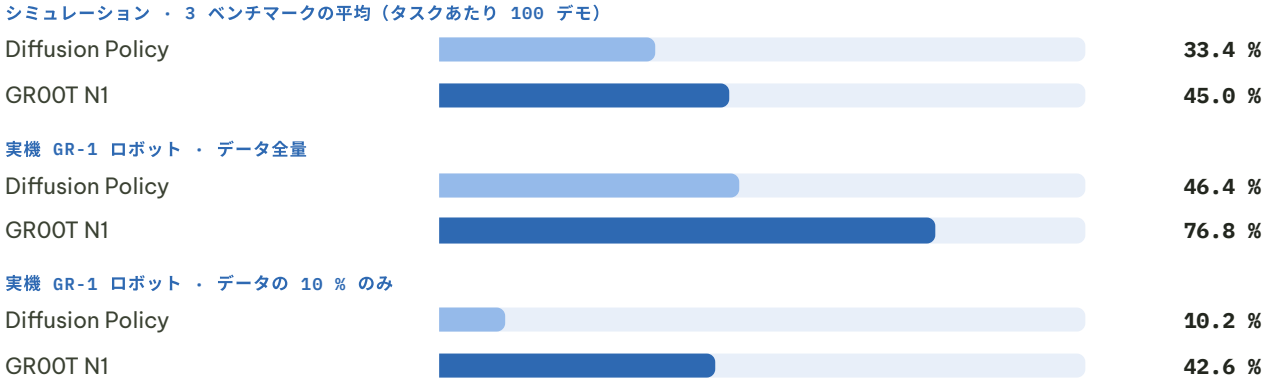
- 理解部：Eagle-2。画像エンコーダ SigLIP-2 と言語モデル SmolLM2 で構成されます。224×224 の画像を 64 トークンに圧縮します（pixel shuffle）。
- 特徴量は言語モデルの最終層ではなく第 12 層から取得します。論文では、この方が高速かつ成功率も高いと報告されています。
- 行動部：Diffusion Transformer (DiT)。cross-attention（理解部のトークンを参照）と self-attention（状態トークンとアクショントークンの間）を交互に配置します。
- 状態とアクションは、ロボットの種類（embodiment）ごとに専用の MLP エンコーダ/デコーダを通ります。

6 公表された結果

論文では、GR00T N1 を、事前学習なしでそのタスク自身のデータのみから学習するモデル Diffusion Policy と比較しています。GR00T N1 はすべての構成で上回り、データが少ないときに差が最も大きくなります。

成功率：Diffusion Policy と GR00T N1 の比較

論文の数値



本プロジェクトでは：これが、数百件のデモで G1 の新しいタスクをファインチューニングでき、real2sim データや生成データで十分に足りる理由です。

シミュレーションベンチマーク別の詳細表 · 論文の表 4

ベンチマーク	DIFFUSION POLICY 30 / 100 / 300 デモ	GR00T-N1-2B 30 / 100 / 300 デモ
RoboCasa（24 タスク）	14.7 / 25.6 / 43.2	**17.4 / 32.1 / 49.6**
DexMimicGen（9 タスク）	23.7 / 46.9 / 68.4	**29.6 / 58.5 / 74.2**
GR-1 Tabletop（24 タスク）	21.3 / 32.7 / 40.4	**43.2 / 50.0 / 49.3**

平均成功率（%）。GR00T N1 はすべてのセルで上回っています。GR-1 Tabletop のみ、100 から 300 デモに増やしてもほとんど向上しません。

7 バージョン比較

カード 6 の数値は、Fourier GR-1 ロボット上の N1 のものです。本プロジェクトの実験はすべて Unitree G1 上の N1.7 を使用しています。

項目	GR00T N1（論文）	GR00T N1.7（本プロジェクト）
規模	パラメータ約 22 億	パラメータ約 30 億
理解部（System 2）	NVIDIA Eagle-2	Cosmos-Reason2-2B（Qwen3-VL アーキテクチャ）、画像は元のアスペクト比のまま
行動部（System 1）	DiT + flow matching	変更なし
複数ロボットへの対応	ロボットごとに専用の MLP	変更なし。状態は固定長までパディング

8 実験で使用したチェックポイント

3 シナリオで使用した 3 つのチェックポイントは、いずれも GR00T N1.7-3B を起点としています。違いは、誰がどのデータでファインチューニングしたかです。

CHECKPOINT	タスク	提供元	ファインチューニングデータ	成功率
シナリオ 1 `GN1x-Tuned-Arena-G1-Static-PickNPlace`	リンゴ → 皿	NVIDIA、そのまま再利用	シミュレーション内のテレオペデモ 208 件	92 % 元の Arena 56 % 再構築した Galileo 0 % 自作シーン
シナリオ 2 `g1_fruits_n17`	4 種類の果物から正しい 1 つを選ぶ	本プロジェクトでファインチューニング	G1 実機データからの real2sim エピソード 790 件	66 % (33/50)
シナリオ 3 `hammer_mix200_n17`	ハンマー → 箱	本プロジェクトでファインチューニング	200 エピソード：テレオペ + 生成	66 % (66/100)
シナリオ 3 `hammer_mix400_n17`	ハンマー → 箱	本プロジェクトでファインチューニング	400 エピソード：mix200 + 新規テレオペデモ 200 件	63 % (63/100)、静止しきい値を緩和すると 73 %

> **ライセンス：**N1.7 は NVIDIA Open Model Agreement に準拠し、Arena チェックポイントには "non-commercial use" と記載されています。

9 調査の結論

3 シナリオすべてで GR00T N1.7 を使用した結果、本プロジェクトが得た知見です。

長所

少ないデータで新しいタスクを素早く学習

- 数百件のデモで十分です：ハンマータスクは 200–400 エピソードで 0 % から 66–73 % に向上しました（III.03 節）。
- ファインチューニングは H100 1 枚で約 1 時間、GPU 1 枚で実行できます。
- 自然な指示文を理解します：4 種類の果物モデルは指示どおりの果物を選び、誤った果物をつかんだことは一度もありません（III.02 節）。
- 1 つのモデルで腕、手、腰を同時に制御します。

制約

画像に敏感で、タスクごとの調整が必要

- シーンに依存します：リンゴタスクは照明が異なると 92 % から 56 % に、シーン全体が異なると 0 % に低下しました（III.01 節）。
- 他のタスクに再利用できません：リンゴのチェックポイントは 4 種類の果物タスクで 0 % でした。
- 1 回の呼び出しはネットワーク経由で約 1.05–1.3 s（モデルの計算は約 100 ms）かかり、0.8 s の動作列より長くなります（I.02 節）。
- NVIDIA の Arena チェックポイントには "non-commercial use" と記載されています。

結論：GR00T は G1 のピック＆プレースタスクの基盤として適しています。結果を左右するのはモデルではなくデータです：実際の作業環境で収集するか、画像の多様性を十分に確保する必要があります。

II.02 パイプラインの調査

調査用ノートブックで GR00T ↔ G1 フローの各ステップを再現。

NVIDIA の既存フレームワークに依存しないよう、本プロジェクトではカメラ画像から関節指令までの**フロー全体を独自に再構築**しました。各ステップは個別の**ノートブック**で調査し（手作業で実装、データを観察、リファレンスと照合）、その後テスト付きのソースコードとしてパッケージ化しています。各カードの「処理」欄にある `nbXX` は、そのステップを実装・検証したノートブック（`codes/standalone_g1_gr00t/notebooks/`）を示します。成果物には 12 本のノートブックがあり、パイプラインを構築する 7 本を本節で、そのパイプラインを使ってシナリオを検証・評価する残り 5 本を第 III 部で扱います。12 本すべての一覧と報告書内の位置はカード 10 にまとめています。

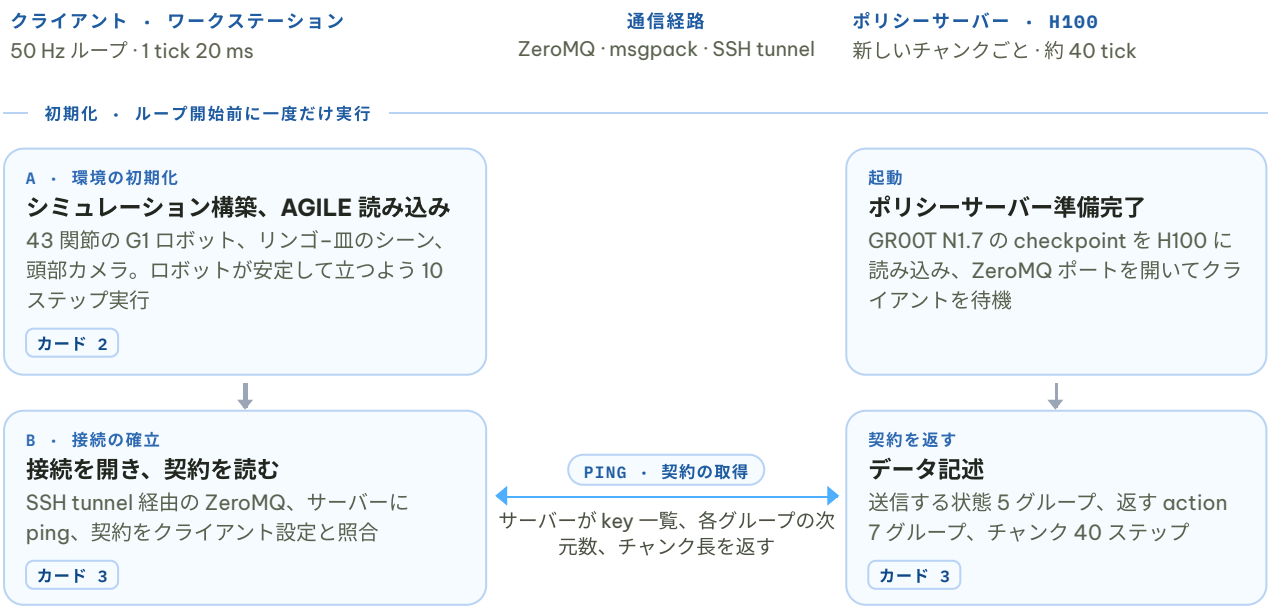
ノートブック
ノートブック 12 本 · 本節 7 本、第 III 部 5 本

ソースコード
Package g1gr00t · unit test 84 件

キーワード
notebook → package · 階層ごとの検証

1 全体フロー

実行前に、クライアントはシミュレーション環境を構築し、サーバーに一度だけ接続します。その後、各制御ループは 2 台のマシンを経由します。ワークステーション上の**クライアント**はシミュレーションとすべてのデータ処理ステップ（センサー読み取り、パッケージ化、デコード、安全チェック、関節への指令送出）を担います。H100 GPU 上の**ポリシーサーバー**はモデルの実行のみを担います。クライアントは 20 ms ごとに動作するが、サーバーを呼び出すのは新しいチャンクが必要なときのみ、つまり約 40 tick に 1 回です。カードラベルは、そのステップを説明しているカードを示します。



制御ループ ・ エピソード終了まで繰り返し

1 ・ センサー読み取り
シミュレーション内の G1 ロボット
頭部カメラ画像 640×480、関節角 43、胴体 IMU
カード 4

2 ・ パッケージ化
観測の作成
関節名に基づき 43 → 上半身 31 関節、指示文を追加、次元数をチェック
カード 5

送信 ・ get_action
画像 0.92 MB + 関節角 31 + 指示文

受信
契約チェック
各グループの名前と次元数を checkpoint の設定と照合
カード 3

3 ・ 待機
新しいチャンクを待つ
シミュレーションは停止して待機。実機では停滞しないよう事前に呼び出す必要がある
カード 7

推論
GR00T N1.7
理解部が画像 + 指示文を読み取り、DiT が 4 ステップでノイズ除去して動作チャンクを生成
~100 ms

4 ・ デコード & 安全
チャンクのチェック
関節グループを分割、NaN/Inf を遮断、関節・移動・高さの制限内にクランプ
カード 6 カード 7

受信 ・ action チャンク
40 ステップ × 35 値、6.8 KB。呼び出し 1 回全体で約 1.05–1.3 s

返却
40 ステップのチャンク
各ステップに上半身 31 関節 + 移動 3 + 高さ 1

5 ・ スケジューリング
tick ごとに 1 ステップを取り出す
40 ステップ = 0.8 s の動作。残りが少なくなったら次のチャンクを事前に呼び出す
カード 7

6 ・ 速度制限
指令の平滑化
各 tick で腕の変化は最大 0.2 rad、特に 2 つのチャンクの境界で有効
カード 7

7 ・ 実行
関節へ指令を送出
上半身 31 関節は GR00T、脚 12 関節は AGILE に従い、その後物理ステップを 4 回
カード 8

20 ms の tick ごとに繰り返し

ロボットが 40 ステップを実行している間

サーバーは空き状態です。クライアントはチャンクの残りが少なくなったときのみ再度呼び出します。事前呼び出し（カード 7）では観測を早めに送り、新しいチャンクが間に合うため、ロボットは停止せずに済みます。

各呼び出しでは時間の大部分が通信経路で費やされます。モデルの計算は約 100 ms ですが、呼び出し 1 回全体では約 1.05–1.3 s かかります。

事前 ・ オフライン
デモ → LeRobot dataset → H100 でファインチューニング → checkpoint をポリシーサーバーに読み込み。

各エピソードの後
リセットして次のエピソードを実行します。N エピソードの結果を集計して成功率とします（カード 9）。

2 環境の初期化

入力
- G1 ロボット（USD、43 関節）
- シーン、リンゴ、皿
- AGILE ポリシー（ONNX）

処理 nb05 nb04
- シミュレーション構築：地面、照明 → ロボット → 頭部カメラ → 物体 → 指の摩擦
- 初期状態を記録し、安定させるため 10 ステップ実行
- AGILE を読み込み、LSTM 状態をリセット

出力
- デフォルト姿勢で立つロボット
- 物理ステップ 5 ms、制御 tick 20 ms

達成した結果

ロボットの安定
0.786 m
10 ステップ後の腰の高さ、その後静止を維持

関節の順序
43 / 43
シミュレーションの関節が作成済みの順序表と一致

シーンの構築
~4 s
1.3 GB の asset を事前ダウンロードした後（初回は 15 分超）

3 接続の確立

入力
- ポリシーサーバーのアドレス（SSH tunnel 経由）
- H100 に読み込み済みの checkpoint

処理 nb01
- ZeroMQ 接続を開き、サーバーに ping
- サーバーからデータ契約を読み取る
- 実行前にクライアント設定と照合

出力
- 接続準備完了
- key 一覧、各グループの次元数、チャンク長

達成した結果

サーバーから読んだ契約
5 → 7 グループ
送信する状態 5 グループ、返す action 7 グループ

チャンク長
40 ステップ
= 50 Hz で 0.8 s の動作

応答しないポート
0.5 s
正常にエラーを返し、接続を再作成

4 観測の取得

入力
- 直前の tick 実行後のシミュレーション状態

処理 nb05 nb02
- 頭部カメラをレンダリング
- 関節角と関節速度 43、胴体 IMU を読み取る
- 4 つの順序体系の間で関節名に基づき並べ替え

出力
- 画像 640×480
- 関節角 43 + IMU（AGILE 用）
- 上半身の関節角 31（GR00T 用）

達成した結果

関節の順序体系
4 体系
シミュレーション、WBC、モデル、dataset

相互変換
0 誤差
シミュレーション → WBC → シミュレーションで値の欠落なし

新しい画像
1 tick ごと
最後の物理ステップでレンダリング。このステップがないとモデルには静止画像しか見えない

5 観測の送信

○ 入力
- 画像 640×480
- 上半身の関節角 31
- 指示文

● 処理 nb02 nb01
- 契約に従ってパッケージ化し、次元数をチェック
- ZeroMQ（msgpack）で get_action を送信
- サーバーが GR00T N1.7 を実行

● 出力
- サーバーへの 0.92 MB のリクエスト
- サーバーが 40 ステップのチャンクを生成

達成した結果

呼び出し 1 回の遅延
~1.05 s
ロボット 1 台 · p95 1.39 s

モデルの計算
~100 ms
サーバー上の計算時間。残りの大部分は通信経路

ロボット 4 台同時
~3.6 s
1 つのバッチでまとめて送信 · p95 4.2 s

送信する指示文が返却チャンクに影響する

左腕の誤差、rad



Arena のデフォルト指示文は無意味な指示文よりも誤差が大きいため、クライアントは dataset の指示文をそのまま送信します。

6 action チャンクの受信

○ 入力
- サーバーの応答：7 key、6.8 KB

● 処理 nb01 nb02
- msgpack をデコード
- 関節角 31（40×31）、移動（40×3）、高さ（40×1）に分割
- 関節名に基づきクライアントの関節順序に並べ替え

● 出力
- クライアント内の 40 ステップ × 35 値のチャンク

達成した結果

1 回の受信

サイズ

返却グループ

40 × 35 値
40 ステップ、各ステップ 35 値

6.8 KB
送信リクエストの約 1/135

7 key
両腕、両手、腰、移動、高さ

7 action チャンクの処理

○ 入力
- 受信したばかりの 40 × 35 のチャンク

● 処理 nb08 nb02
- NaN/Inf を遮断、関節・移動・高さの制限内にクランプ
- スケジューリング：tick ごとに 1 ステップを取り出し、残りが少なくなる前に事前呼び出し
- tick ごとの変化量を制限

● 出力
- 現在の tick 用の安全な指令：関節角 31 + 脚への指令 4

達成した結果

制限外
0 / 1,240
テスト用チャンク 1 つで、モデルの返した値が関節制限を超えた数

意図的なテスト
81 値をクランプ
関節角 1、移動指令 40、高さ指令 40

待ち時間
0.28 s
事前呼び出し時。逐次呼び出しでは 0.84 s

8 ロボットの実行

○ 入力
- 目標関節角 31
- 移動 + 高さの指令
- 現在の観測（AGILE 用）

● 処理 nb04 nb05
- AGILE が 80 値のベクトルから脚の関節角 12 を計算
- 目標角 43 をシミュレーションに書き込む
- 5 ms の物理ステップを 4 回実行

● 出力
- ロボットが 1 tick（20 ms）分動作
- 次の tick 用の新しい状態

達成した結果

AGILE とリファレンスの比較
50 tick
ビット単位で一致、LSTM 状態も同一

安定した直立
10 s
500 tick、最大傾斜 9.8°

デモの再生
121 tick
モデル不使用：成功、リンゴと皿の中心の距離 2.9 cm

9 閉ループの実行

入力

- checkpoint、シーン、指示文
- カード 4–8 の各ステップ

処理 nb06 nb07 nb08

- 各 tick：観測の取得 → 処理 → ロボットの実行
- 約 40 tick に 1 回：観測を送信し、新しいチャンクを受信
- 成功、転倒、タイムアウトで終了。リセットして次のエピソードを実行

出力

- エピソードごとの結果
- ログ、動画、統計

達成した結果

成功率：同じモデルでシーンのみ変更 50 エピソード

独自構築のシーン	0 / 50
Galileo シーン（データ収録時と同じ）	28 / 50

成功したエピソードは約 120 tick で、サーバー呼び出しちょうど 3 回に相当します。独自構築のシーンでもロボットは 50 回中 11 回リンゴを持ち上げましたが、皿の上には置けませんでした。Galileo シーンを再構築すると、リンゴと皿の最小距離（中央値）は 20.6 cm から 3.8 cm に縮小しました。

10 ノートブック 12 本と報告書内の位置

nb01、nb02、nb04–nb08 はカード 2–9 でパイプラインの各ブロックを構築・検証します。nb03 はシミュレーションを使わずに実データでモデルを確認します。nb09–nb12 はパイプラインをシナリオ 2・3 に適用します。各シナリオには、シーンを構築して 1 エピソードを実行するノートブックと、N エピソードを実行して成功率を算出するノートブックが 1 本ずつあります。

ノートブック	内容	掲載箇所	実行環境
パイプラインの構成要素 · II.02 節			
`nb01_server_wire`	ZeroMQ + msgpack 接続、データ契約の読み取り、遅延測定、接続断の処理	カード 3、5、6	`.venv` · サーバー必要
`nb02_joints_obs_action`	4 種類の関節順序、観測のパッキング、action チャンクのデコード、チャンクのスケジューリング	カード 4–7	`.venv` · サーバー必要
`nb04_agile_onnx`	AGILE 脚ポリシー：80 次元観測ベクトル、リファレンスとのビット単位の一致確認	カード 2、8	`.venv`
`nb05_backend_anatomy`	Isaac Sim バックエンドを手作業で構築：ロボット、頭部カメラ、リンゴ/皿、摩擦、関節の読み書き	カード 2、4、8	Isaac Sim
`nb08_runner_policy_safety`	安全レイヤー（NaN 遮断、制限内へのクランプ、変化量制限）、runner、ログ記録	カード 7、9	Isaac Sim

ノートブック	内容	掲載箇所	実行環境
閉ループ · II.02 節カード 9 とシナリオ 1			
`nb06_full_pipeline`	全ブロックを自作シーンの閉ループに統合：直立、デモ再生、モデル実行	カード 9 · III.01	Isaac Sim · サーバー必要
`nb07_full_pipeline_loop`	Galileo シーンを再構築し 50 エピソード実行：0/50 → 28/50	カード 9 · III.01 · IV.04	Isaac Sim · サーバー必要
オープンループ検証 · シナリオ 1			
`nb03_open_loop_dataset`	データセットの実画像と状態をサーバーに送り、返されたチャンクを記録済み action と比較	III.01 ステップ 3	`.venv` · サーバー必要
シナリオ 2・3 への適用 · 第 III 部			
`nb09_multi_object_task_pipeline`	果物 4 種の real2sim シーン、指示文の選択、1 エピソード実行	III.02	Isaac Sim · サーバー必要
`nb10_eval_multi_object`	nb09 のシーンで N エピソード実行：66 %	III.02 · IV.04	Isaac Sim · サーバー必要
`nb11_hammer_to_bin`	Arena のハンマー → 箱シーンを Isaac Sim で再構築、1 エピソード実行	III.03	Isaac Sim · サーバー必要
`nb12_eval_hammer_to_bin`	nb11 のシーンで N エピソード実行、失敗を分類：66 %（mix200）、73 %（mix400）	III.03 · IV.04	Isaac Sim · サーバー必要

> ノートブックのコードの大部分は package `g1gr00t` にまとめてあります。構成要素のノートブック（nb01、nb04、nb05、nb08）には、手書き実装と package を照合するセルがあります。ノートブックの再実行方法：IV.04 節。

関節角度から動作と力へ。

GR00T が出力するのは**目標関節角度**だけです。その数値を実際の動作に変えるには、**運動学**（関節角度 ↔ 手・カメラの位置）と**動力学**（トルク、慣性、接触、バランス）の理解が必要です。本節ではこれらの知識と、プロジェクト内で現れる場面をまとめます。各概念には試せる図解を用意しています。

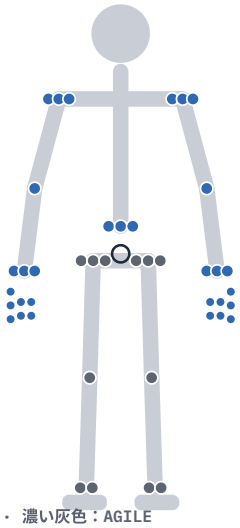
ロボット	適用先	キーワード
Unitree G1 · 43 関節	**Real2sim · テレオペ · 関節制御 · バランス**	**FK · IK · PD · 接触**

1 G1 の関節構造

G1 はリンクを回転関節で結んだツリー構造で、根元は骨盤（pelvis）です。関節グループごとに異なるコンポーネントが制御します。各グループをクリックして確認できます。

G1 の 43 関節 · 正面図

模式図



青：GR00T · 濃い灰色：AGILE
円：根元（骨盤）

12	**脚** 片側あたり 股関節 (3)、膝、足首 (2)	AGILE
3	**腰** ヨー、ロール、ピッチ	GR00T
14	**腕** 片側あたり 肩 (3)、肘、手首 (3)	GR00T
14	**Dex3 ハンド** 片側あたり 人差し指 (2)、中指 (2)、親指 (3)	GR00T

GR00T は上半身の 31 関節を制御します（タスク中、腰はほぼ固定）。AGILE は脚の 12 関節を担当します。

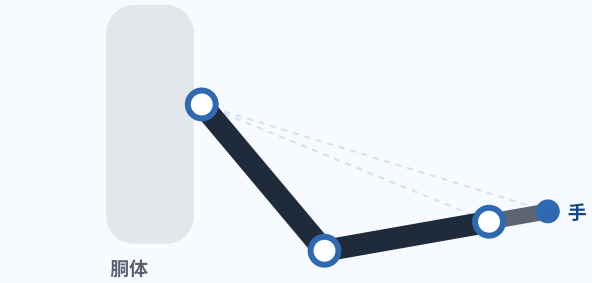
4 種類の関節順序が共存しています：シミュレーション内の順序（USD ツリーの走査順）、全身コントローラの順序、GR00T のグループ順序、AGILE の入力順序。1 列ずれるだけでロボットが転倒したりモデルが「盲目」になったりするため、プロジェクトではすべてを**関節名**で対応付け、自動テストで確認しています。

2 順運動学

順運動学（FK）は、URDF ファイルに記述された関節ツリーに沿って変換を順に掛け合わせることで、関節角度から各リンク（手、指先、カメラ）の位置を計算します。スライダーを動かすと分かるとおり、関節角度が分かれば手の位置が正確に分かります。

腕の側面図 ・ 3 関節

図解 ・ リンク長は概略値



テーブル面

FK の結果

$T_{手} = T_{肩}(0.70\ \mathrm{rad}) \cdot T_{肘}(1.05\ \mathrm{rad}) \cdot T_{手首}(0.00\ \mathrm{rad})$

手は肩より前方	52.2 cm
手は肩より下	16.1 cm

各関節が回転 1 つと並進 1 つを加え、肩から外側へ順に掛け合わせると手の位置が得られます。実機ロボットはこれを 3D で、すべてのリンクについて行います。

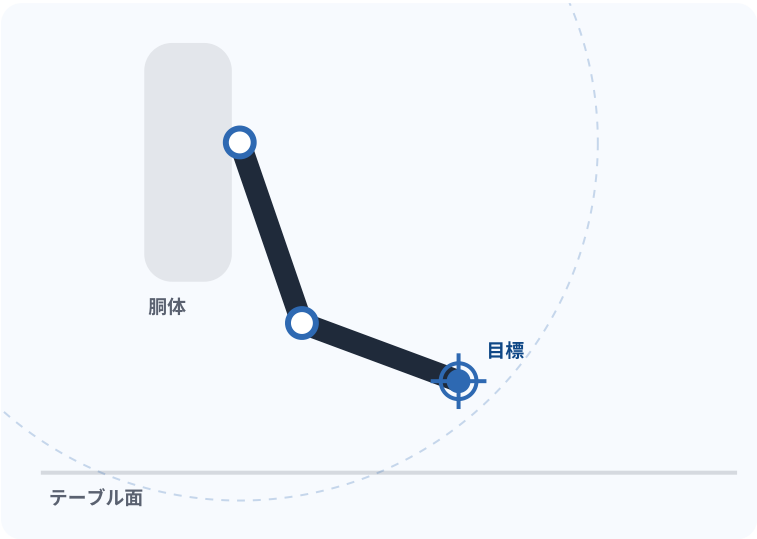
real2sim での活用：実機 G1 のデータセットには物体の位置がありません。自作の FK（numpy のみ使用）で関節角度から手のひらの軌跡を計算します。手を閉じた場所に果物があり、手を開いた場所に皿があります。胴体の姿勢は両足が床に平らに接しているという仮定から推定します。詳しい図解は II.04 節を参照してください。

3 逆運動学

逆運動学（IK）はその逆で、手首の目標位置を与えて、それを実現する関節角度を求めます。Meta Quest ヘッドセットでテレオペする際、コントローラの姿勢がロボットの手首目標となり、IK ソルバー **Pink** が毎フレーム関節角度を計算します。枠内をクリックまたはドラッグして目標を設定できます。到達範囲外にも置いてみてください。

クリック / ドラッグで手首目標を設定

図解 · 2 関節 IK



IK の解

肩の角度	0.33 rad
肘の角度	0.88 rad
目標までの距離	48.8 cm

✓ 到達：手首が目標点と一致

プロジェクトでの用途：IK はハンマーのタスクの**データ作成**にだけ使い、モデルの実行中には使いません。

- **Meta Quest ヘッドセットによるテレオペ**（II.04 節、III.03 節）：コントローラーの姿勢を骨盤座標系での手首目標に変換し、Pink が制御ステップごとに腕の関節角を解きます（embodiment `g1_wbc_agile_pink`）。記録中、IK は 1 ループの約 7 % を占めます（I.02 節）。目標が届かない位置にあると、IK は関節の限界で止まり、腕はエラーを出さずに静止します。上の図と同じ現象です。
- **デモの追加生成**（II.04 節）：スクリプトがテレオペのデモから取った手首軌道の開始・把持・リリース点をずらし、ヘッドセットと同じ種類の指令として送ります。Pink がそれを関節角に変換します。mix200 の追加生成デモ 100 件はすべてこの工程を通っています。
- **評価時**は GR00T が関節角を直接出力するため（embodiment `g1_wbc_agile_joint`）、IK は不要です。データセットには記録時に IK が計算した関節角が保存され、モデルはその角度を直接学習します。real2sim のシナリオ（III.02 節）も、関節角が実機データに含まれているため IK を使わず、代わりに FK を使います（カード 2）。

4 動力学

運動学が教えてくれるのは、関節角度が分かったときに**手がどこにあるか**だけです。**動力学**は次の問いに答えます：関節を**実際にその角度まで回転させる**にはどうするか。GR00T が送るのは**目標角度**だけで、力は送りません。各関節のモーターは、リンク（質量と慣性を持つ）を振動させずに目標まで動かすのに十分なトルクを自ら生み出す必要が

あります。

1 · 目標
到達すべき関節角度
GR00T から 20 ms ごと

▶

2 · PD 制御器
トルクを計算
角度誤差と速度に基づく

▶

3 · 物理
関節が回転
トルク ÷ 慣性 = 角加速度

▶

4 · 再計測
新しい角度
5 ms の物理ステップごとに繰り返し

P · 剛性（K_p）

ばねのように

目標から離れるほど、引き戻す力が強くなります。ばねが硬いほど関節は速く目標に追従しますが、行き過ぎて振動しやすくなります。

D · 減衰（K_d）

ショックアブソーバーのように

関節が速く回るほど、制動力が大きくなります。減衰が十分なら関節は行き過ぎずに滑らかに目標へ到達し、多すぎると到達が遅くなります。

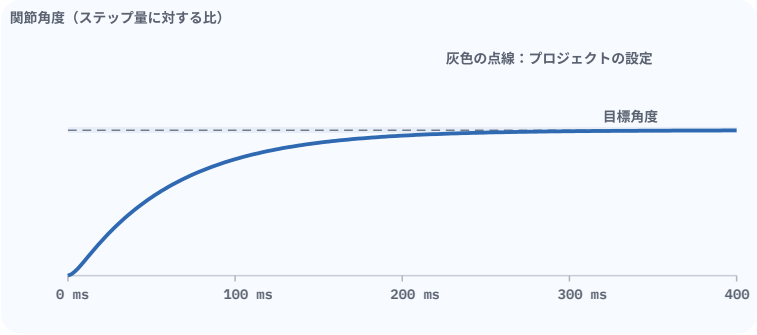
$$\text{トルク} = K_p \times (\text{目標角度} - \text{現在角度}) - K_d \times \text{関節速度}$$

（ばねが目標へ引き戻す）　（ショックアブソーバーが制動）

プロジェクトでは：K_p、K_d は手で選ばず、分かりやすい 2 つのパラメータからモーター慣性に基づいて導出します：**固有振動数 10 Hz**（速さ）と**減衰比 ζ = 2**（滑らかさ）。結果：関節は約 140 ms で目標の 90 % に達し、オーバーシュートはありません。下のスライダーで比較できます。

関節にステップ指令 · 実際の角度の時間変化

図解 · 理想的な 2 次系



応答

目標の 90 % に到達	141 ms
オーバーシュート	0 %
±2 % 以内に整定	237 ms

ζ ≥ 1 · 過減衰：滑らかに目標へ到達、オーバーシュートなし。 固有振動数が高いほど追従は速くなりますが、モーターはより大きなトルクを出す必要があります。プロジェクトでは腕が振動しないよう ζ = 2 を選択しています。

シミュレーションで PD トルクを計算するのは誰か？

方式 1 · 暗黙的 PD（IMPLICIT）

物理ソルバーが自ら計算

PhysX がソルバーのステップ内で PD 力を計算します。関節が硬くても非常に安定しており、Isaac Lab のデフォルトです。

方式 2 · 明示的 PD（IDEAL PD）

コードで計算してトルクを与える

各ステップで上の式をそのまま適用するため、実機ロボットのドライバーにより近い方式です。Arena は G1 にこの方式を使用しています。

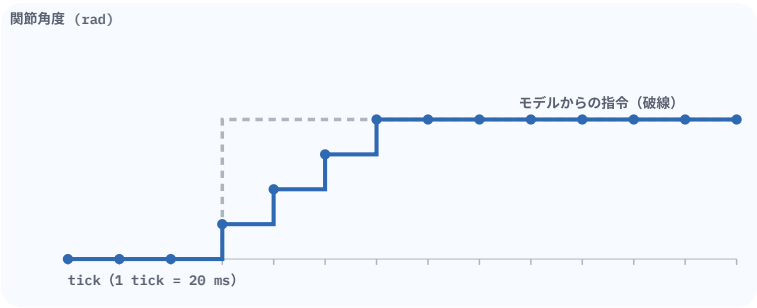
注意すべき理由：同じ目標角度の系列でも、2 つの方式では腕の動きがわずかに異なります。モデルがどちらの方式で記録したデモから学習したかに合わせて、評価でも同じ方式を使うべきです。

5 関節速度の制限

安全層が関節角度の急な跳びを抑えます：1 tick（20 ms）あたりの角度変化は、腕で最大 0.2 rad、腰で 0.1 rad、手で 1.5 rad です。しきい値は実データから決定しています（腕：99.9 パーセンタイルで 0.15 rad/tick）。大きなステップ指令を試してみてください：

ステップ指令と制限後の角度 · 腕の関節

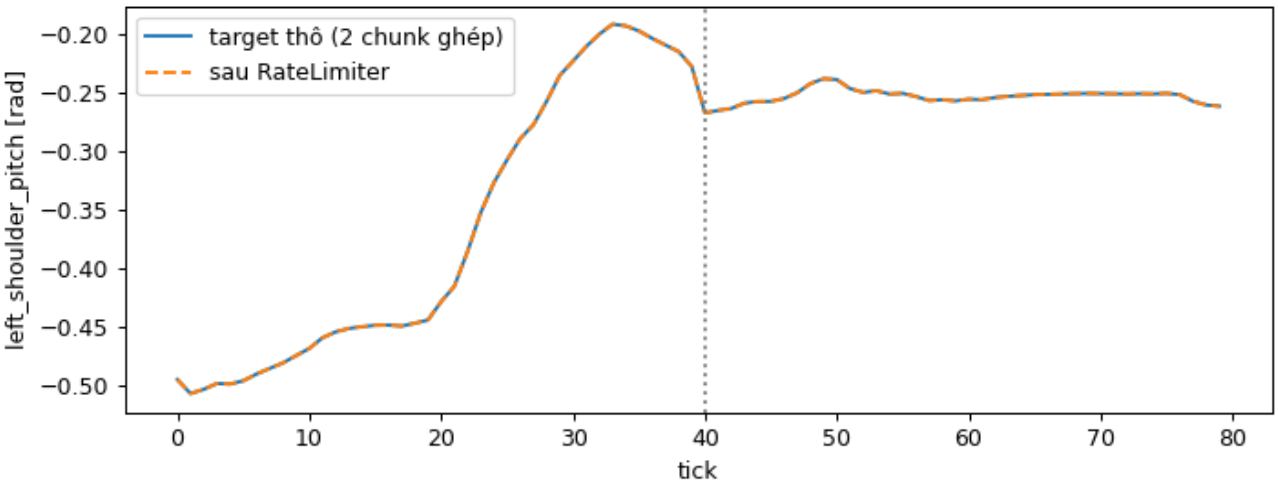
図解



結果

指令のステップ量	0.80 rad
1 tick あたりの最大値	0.2 rad
減速された tick 数	3

指令は 4 個の小ステップに分割され、20 ms ではなく 80 ms かかります



実データ：左肩関節の指令、80 tick 分（40 ステップの系列 2 本を連結）。通常の動作では、リミッター後の線（破線）は元の指令と一致します。安全層が介入するのは異常な跳びがあるときだけです。

6 物理ステップ：動きはどう計算されるか

コンピューターは連続した動きをそのまま計算できません。時間をごく短い**物理ステップ**（5 ms）に分け、各ステップで力 → 加速度 → 速度 → 位置を計算し直します。最も分かりやすい例は道路を走る車で、ロボットの腕の関節もまったく同じ方法で計算されます。

例 · 車

5 ms ごとに計算し直す

各ステップで、今働いている力（エンジン、摩擦）を見る → 加速度 a = F / m → 新しい速度 v + a·Δt → 新しい位置 x + v·Δt。運転者は 1 秒に 200 回ペダルを踏むわけではなく、20 ms ごとに判断してその 20 ms はペダルを保ちます。つまり**制御 1 tick = 物理 4 ステップ**です。

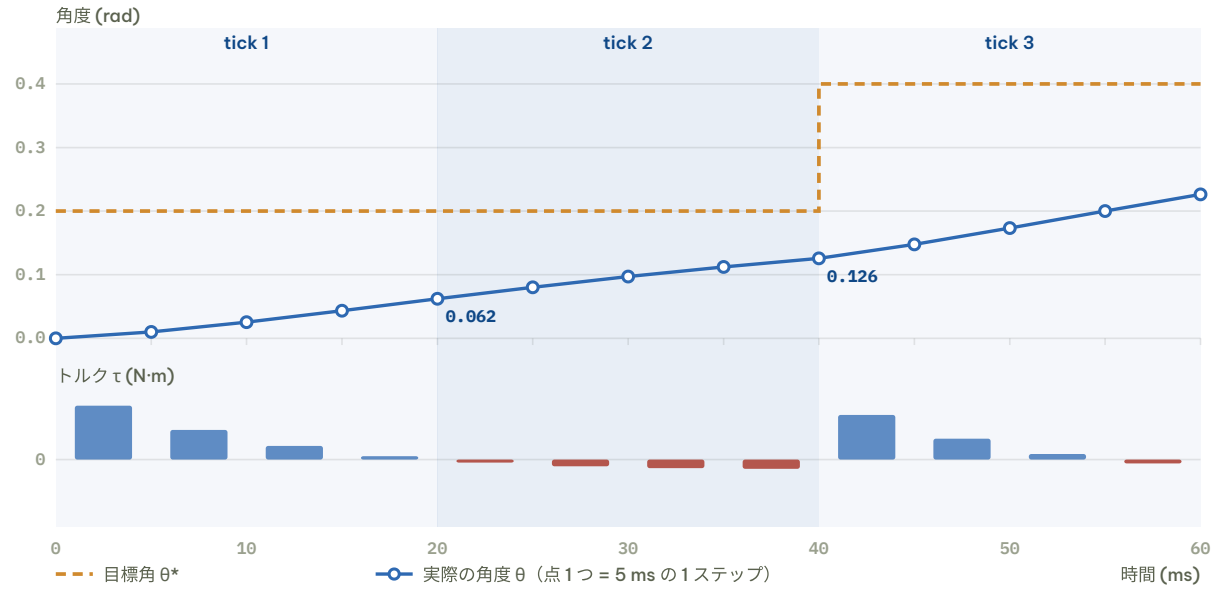
適用 · 腕の関節

ばねとダンパーが関節を目標角へ引く

モデルは関節角を直接設定せず、**目標角** θ* だけを与えます。PD 制御器（カード 4）は、関節を θ* へ引くばねと、速く回るとブレーキをかけるダンパーのようにトルクを生みます。物理ステップごとに、そのときの角度と角速度からトルクを計算し直します。

車	腕の関節	意味
位置 x (m)	関節角 θ (rad)	計算される状態
速度 v (m/s)	角速度 ω (rad/s)	計算される状態
力 F (N)	トルク τ (N·m)	物理ステップごとに再計算
質量 m	慣性 I	大きいほど加速しにくい
運転者がペダルを決める	パイプラインが目標角 θ* を決める	制御 tick（20 ms）ごとにだけ変わる

```
物理ステップごと（Δt = 5 ms）：
  τ = Kp·(θ* − θ) − Kd·ω      # ばねが θ* へ引き、ダンパーが回転を抑える
  α = τ / I                   # 角加速度
  ω = ω + α·Δt                # 新しい角速度
  θ = θ + ω·Δt                # 新しい角度
```



説明用の例（I = 0.05、K_p = 100、K_d = 4。G1 の実際のパラメーターではありません）。tick 1–2 は目標角 0.2 rad、tick 3 は 0.4 rad。目標角は tick の始めに階段状に変わり、実際の角度は 5 ms ステップごとに徐々に上がります。トルクは目標から遠いうちは大きく、速く回り出すと符号が変わってブレーキになります。

TICK	ステップ	時刻	Θ*	前の Θ	T	後の Ω	後の Θ
1	1	5 ms	0.2	0.000	20.0	2.00	0.010
	2	10 ms	0.2	0.010	11.0	3.10	0.026
	3	15 ms	0.2	0.026	5.1	3.61	0.044

TICK	ステップ	時刻	Θ*	前の Θ	T	後の Ω	後の Θ
	4	20 ms	0.2	0.044	1.2	3.73	0.062
2	5	25 ms	0.2	0.062	-1.1	3.62	0.080
	6	30 ms	0.2	0.080	-2.5	3.37	0.097
	7	35 ms	0.2	0.097	-3.2	3.05	0.112
	8	40 ms	0.2	0.112	-3.4	2.71	0.126
3	**9**	**45 ms**	**0.4**	**0.126**	**16.6**	**4.37**	**0.148**

- **目標角は制御 tick ごとにしか変わらない**：最初の 8 ステップは 0.2 rad、ステップ 9（tick 3）から 0.4 rad。
- **トルクは物理ステップごとに再計算される**：目標から遠いステップ 1 では 20 N·m、その後小さくなり、ステップ 5 からは負になって行き過ぎを防ぐ。
- **角度は跳ばずに徐々に上がる**：指令は 0.2 rad でも、20 ms 後に 0.062 rad、40 ms 後に 0.126 rad。ペダルを踏んだ瞬間に速度が出ない車と同じ。
- **ステップを短くする理由**：20 ms ごとにしか計算しないと、最初の 20 N·m が 20 ms の間ブレーキなしで働き、関節は行き過ぎて振動し、指と物体の接触も崩れやすくなる。

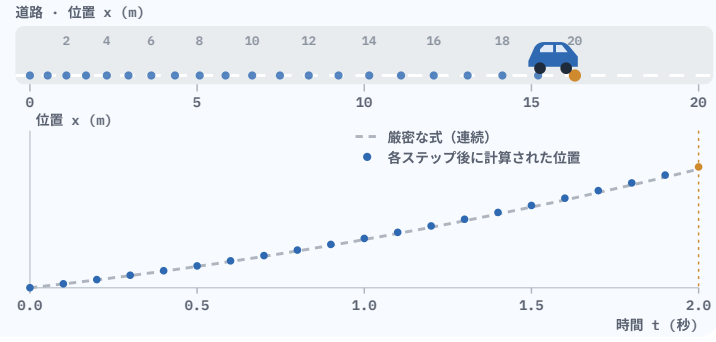
> **Isaac Sim では：**PD 制御器は物理ソルバーの中で解かれる（*implicit*、カード 4）ため、上の 1 行ずつの計算より安定していますが、考え方は同じです。目標角は tick の間固定、トルクと角度は 5 ms ステップごとに、43 関節すべてについて、ハンマー・ビン・指の接触や摩擦と一緒に更新されます。

7 シミュレーション：物理ステップごとに進む車

車は速度 v_0 から出発し、加速度 a で 2 秒間等加速します。**実行**を押すと、車は滑らかに動くのではなく、**物理ステップが終わるたびに新しい位置へ跳びます**。点はコンピューターが計算した位置で、2 つの点の間で車がどこにいるかはコンピューターには分かりません。ステップの長さを変えると、長いステップでは点がまばらになり、計算位置が厳密な式からずれることが分かります。

道路を走る車 ・ 物理ステップごとの位置

説明用 ・ 等加速度



現在のステップ

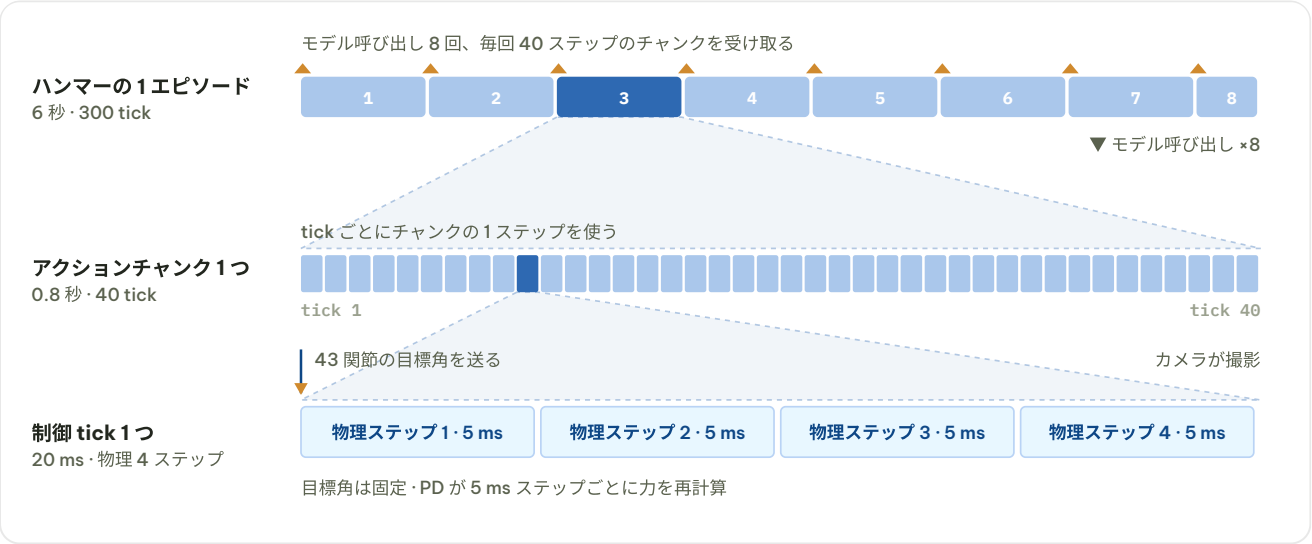
ステップ	20 / 20
時刻 t	2000 ms
速度 v	11.00 m/s
位置 x	16.300 m

2 秒後（20 ステップ）：計算値 16.30 m、厳密値 16.00 m、誤差 0.300 m（1.9 %）

各ステップはカード 6 と同じ 3 行です：`a`（ここでは一定）→ `v = v + a·Δt` → `x = x + v·Δt`。初期値のままプロジェクトと同じ Δt = 5 ms なら 2 秒後の誤差は約 1.5 cm ですが、Δt = 200 ms では 50 cm を超えます。ロボットの腕の関節もまったく同じ方法で計算され、位置 x が角度 θ に、加速度 a が角加速度 α = τ / I に置き換わり、トルク τ はステップごとに再計算されます。

8 制御 tick と物理ステップ

シミュレーションは**入れ子になった 3 つのクロック**で動きます。最も速いのは Isaac Sim の物理ステップで、物理 4 ステップでパイプラインの制御 1 tick、40 tick ごとに GR00T モデルを 1 回呼んで新しいアクションチャンクを得ます。



入れ子の 3 つのクロック。 モデル呼び出し 1 回で 0.8 秒のチャンク、チャンクは制御 40 tick で実行、1 tick は 5 ms の物理 4 ステップ。

物理ステップ
5 ms
200 Hz · Isaac Sim の PhysX

制御 tick
20 ms
50 Hz · = 物理 4 ステップ

アクションチャンク 1 つ
0.8 秒
40 tick · モデル呼び出し 1 回

ハンマーの 1 エピソード
6 秒
300 tick · 物理 1,200 ステップ · モデル呼び出し 8 回

クロック	周期	実行主体	内容
物理ステップ	5 ms · 200 Hz	PhysX（Isaac Sim）	目標角から PD で各関節の力を計算し（カード 4）、指・物体・机の接触と摩擦を解き、すべての物体の位置を更新
制御 tick	20 ms · 50 Hz	本プロジェクトのパイプライン	観測を読み、アクションチャンクから 1 ステップを取り、関節速度を制限し、AGILE に脚 12 関節を計算させ、43 関節の目標角を送る。カメラは 1 枚撮影
アクションチャンク	0.8 秒 · 40 tick	GR00T サーバー	画像、指示文、関節状態を見て、腕・手・腰の目標角を 40 ステップ分返す

コード上の制約：物理ステップ × tick あたりのステップ数は 1 tick とちょうど等しくなければなりません（0.005 秒 × 4 = 0.02 秒）。食い違う設定は読み込み時にエラーになります（`g1gr00t/config.py` の `ControlCfg`）。シナリオ 2（III.02 節）は 20 Hz で記録された実機 G1 のデータに合わせて別の周期を使います：物理は 200 Hz のまま tick あたり 10 ステップ、つまり制御 20 Hz で、16 ステップのチャンクもやはり 0.8 秒です。

9 制御 tick の中身

パイプラインは tick ごとにちょうど 1 回のループを実行します：状態を読み、43 関節すべての目標角を決め、その目標角で物理を 4 ステップ進めます。

01 · 観測
画像と状態
頭部カメラ、43 関節角、速度、IMU

→

02 · チャンク
次のステップを取る
40 ステップ終われば GR00T サーバーを呼ぶ

→

03 · 安全
速度制限
腕 0.2 rad/tick（カード 5）

→

04 · 脚
AGILE がバランスを保つ
観測 80 値 → 脚 12 関節

→

05 · 物理
4 ステップ × 5 ms
目標角は固定

```
制御 tick ごと（20 ms）:
  obs   = backend.observe()                      # 画像、43 関節角、速度、IMU
  step  = scheduler.step(policy.fetch, obs)      # 40 ステップ中 1 つ。尽きたら呼ぶ
  upper = rate_limiter.apply(step["upper_q"])    # 腕 ≤ 0.2 rad/tick（カード 5）
  lower = agile.step(q, dq, imu)                 # 脚 12 関節、これも 50 Hz
  robot.set_joint_position_target(upper + lower)
  for _ in range(4):                             # 4 ステップ × 5 ms、目標角は固定
      sim.step()                                 # PD で力を計算。接触・摩擦
  camera.update()                                # 次の tick 用の画像
```

指令の保持

目標角は物理 4 ステップの間固定

送る指令は力ではなく目標角です。1 tick の 20 ms の間に PD 制御器が変化する実際の角度から力を 4 回計算し直すため、関節は跳ばずに目標へ近づきます。実機も同じで、モータードライバーが位置指令を受け取り、はるかに速い独自の制御ループを回します。

2 つの周期

物理が制御より速く動く必要がある理由

衝突や指と物体の接触は、時間ステップが小さくないと安定しません。ステップが長いと物体がめり込んだり跳ねたりしやすくなります。一方、制御 tick はちょうど 20 ms でなければなりません。AGILE もデモデータも 50 Hz なので、tick が違うとアクションチャンクの 1 ステップが学習時の 20 ms と対応しなくなります。

10 シミュレーション時間と実時間

モデルの 1 回の呼び出しには約 1.2–1.5 秒かかり（評価時の中央値、I.02 節）、アクションチャンク 1 つの 0.8 秒より長くなります。シミュレーションと実機ではこの差の扱いが大きく異なります。

シミュレーション

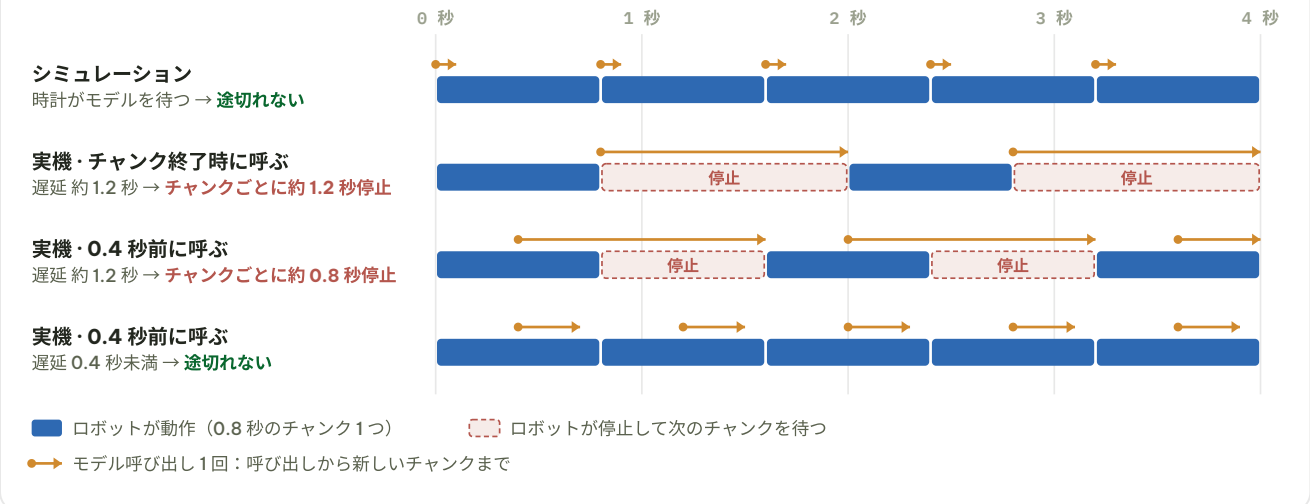
シミュレーションの時計はモデルを待つ

シミュレーション時間は `sim.step()` を呼んだときだけ進みます。サーバーを待つ間は物理ステップが走らないため、遅延は結果に影響しません。その代わり実時間より遅く、ハンマーの 6 秒のエピソードは平均で実時間約 36 秒かかります（mix400 の実行、1 エピソードあたりモデル呼び出し 8 回）。

実機

時間は止まらない

チャンクが尽きてから呼ぶと、待つ間ロボットは止まってしまいます。コードには `PrefetchingChunkScheduler` があり、チャンクの残りが 20 tick（0.4 秒）になった時点でサーバーを呼び、古いチャンクが終わるころに新しいチャンクが届くようにします。代償として、新しいチャンクは 0.4 秒早く撮った画像から計算されます。



最初の 4 秒の時間軸。シミュレーションの時計はモデルを待つため、止まる必要がありません。実機では 0.4 秒前に呼びかつ遅延が 0.4 秒未満のときだけ動作が途切れません。モデル自体は約 0.1 秒で、残りは通信経路です（I.02 節）。

II.04 データの調査

少ない実デモから、多くの学習データを。

GR00T のファインチューニングにはタスクごとに数百件のデモが必要ですが、テレオペデモはコストが高く、モデルは画像の変化に非常に敏感です。本節では、テレオペを増やさずにデータを増やすための 3 つの手法、**domain randomization**、**real2sim**、**デモ追加生成**を説明します（本プロジェクトでは後の 2 つを適用済みで、最初の手法はまだ提案段階です）。各手法には動作を確認できるインタラクティブな図解があります。

手法
Domain randomization · real2sim · デモ追加生成

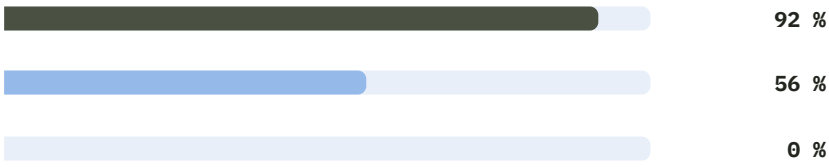
適用先
Apple & Plate · Multi Fruit · Hammer & Bin

キーワード
Visual gap · digital twin · MimicGen

1 課題 1: Visual Gap

すべてのデモを同じシーンで記録すると、モデルはその画像を丸暗記してしまいます。リンゴのタスクがこれをはっきり示しています。

同じリンゴ → 皿のチェックポイントで、シーンだけを変更　　成功率 · 実測値、III.01 節



チェックポイントと指示文は同じです。92 % は NVIDIA クライアントで、残りの 2 つは本プロジェクトのパイプラインで測定しました。モデルは学習時に 1 つのシーンしか見ていないため、リンゴではなくその画像に結びついてしまいます。対処法は domain randomization（カード 2）です。

2 解決策 1: Domain Randomization

一言でいうと：タスクに**関係のない**もの（照明、色、背景、カメラ角度）をすべてランダムに変え、モデルが頼れるのを変わらないもの、つまり**物体と手**だけにします。新しいシーンに出会っても、実画像であっても、モデルはそれを変化のひとつとして扱います（Tobin ら、2017）。

試してみる：画像を多様化する 2 つの方法

方法 1 · 2D 画像拡張

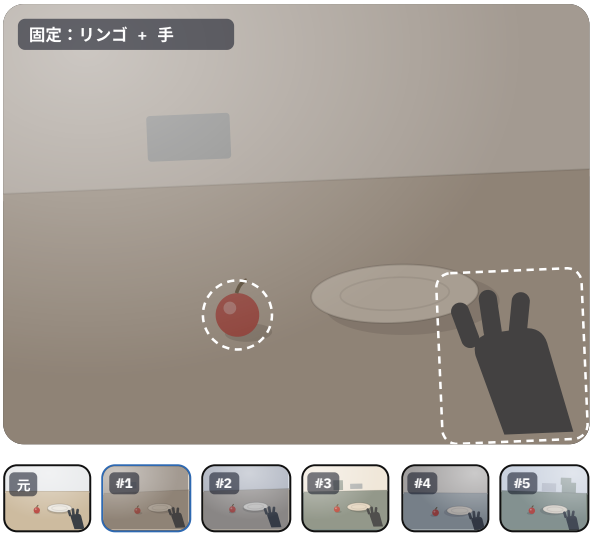
実画像 · 頭部カメラ



Color jitter：低コストで学習時に画像へ適用できますが、**影、反射、新しい視点は作れません**。本プロジェクトでは Multi Fruit のファインチューニング時にこの方法を使用しました。

方法 2 · DOMAIN RANDOMIZATION

図解 · 実測データではない



シーンそのものを変えて**再レンダリング**します。リンゴと手（破線の枠）は変わりません。これがモデルが学ぶべきものです。

ランダム化できる要素

易しいものから難しいものまで、3 つの要素グループ：

外観

モデルが見るもの

照明、材質、テクスチャ、背景、操作領域外の装飾物。

幾何

配置と視点

物体の位置・向き、カメラの位置・俯角・焦点距離。

動力学

物理

質量、摩擦、関節剛性、遅延（Peng ら、2018）。実ロボットに移行する際に必要です。

トレードオフ：ランダム化の範囲が広いほどモデルは頑健になりますが、必要なデータと学習ステップも増えます。まず実際に性能低下を引き起こした要素（ここでは照明とレンダリング）を優先し、その後範囲を広げるべきです。

本プロジェクトでの適用

適用先	実施済み	未実施
Apple & Plate	Visual gap を測定 92 → 56 → 0 %	シーンをランダム化して 208 件のデモを再生（III.01 節で提案）
Multi Fruit	エピソードごとに果物、位置、向き、皿、妨害用の果物を変更。ファインチューニング時に color jitter	照明と材質は未変更

3 課題 2: Sim Data Scarcity

Multi Fruit シナリオには G1 + Dex3 向けの適切なシミュレーションデータセットがありません。最も近いのは **PhysicalAI-Robotics-GR00T-Teleop-G1**（テレオペ軌道 1,095 件）ですが、実ロボットで記録されたものであり、シミュレーションでそのまま使うことはできません。

不足 1

物体の位置がない

データには関節角度と動画しかなく、シーンを構築するための果物と皿の位置がわかりません。

不足 2

実画像 ≠ レンダリング画像

実際の RealSense 動画は、実行時にモデルが見るシミュレーション画像と一致しません。

不足 3

Action の意味が異なる

実ロボットとシミュレーションの action は名前が同じでも意味が異なります。

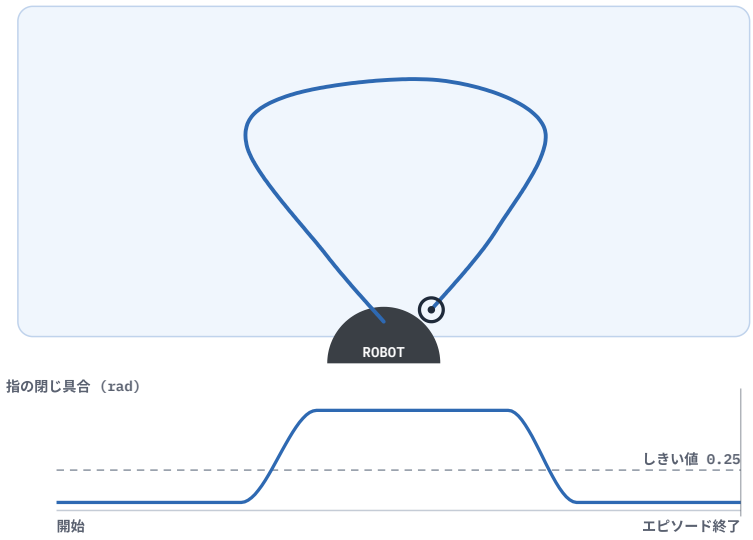
4 解決策 2: Real2sim

> **一言でいうと：**実際のシーンの**デジタルコピー**（digital twin）をシミュレーション内に再構築し、その中で実際の動きを再生してシミュレーションデータを得ます。データセットには物体の位置がないため、本プロジェクトでは**手の動きからシーンを推定**します。手が閉じた場所に果物があり、手が開いた場所に皿があります。

試してみる：1 本の軌道からシーンを再構築

テーブルを真上から見た図

図解 ・ 実測データではない



ステップ 1 / 5

順運動学（FK）

各フレームの関節角度から、腕に沿った変換（URDF ファイル）を順に掛け合わせて手のひらの位置を求めます。これをつなげるとテーブル上の手の軌道になります。

本プロジェクトでは： numpy のみで独自実装した FK。胴体の姿勢は両足が床に平らに接しているという仮定から推定（II.03 節）。

5 課題 3: Teleop Bottleneck

Hammer & Bin シナリオはまったく新しいもので、既存のデータセットやチェックポイントはありません。すべての

デモを Isaac Lab 上で Meta Quest ヘッドセットによるテレオペで自ら収集する必要があります。

遅い

1 セッションあたり ~10 件

最初の実デモ 100 件に 10 回のテレオペセッションを要しました。

不安定

やり直しが多い

コントローラーの信号が途切れるとロボットの手が停止し、2 回目以降はカクつきが発生します。

品質のばらつき

失敗デモがデータに混入

ハンマーが箱の縁に引っかかったまま、または手が離れていないのに成功と判定されたデモがありました。

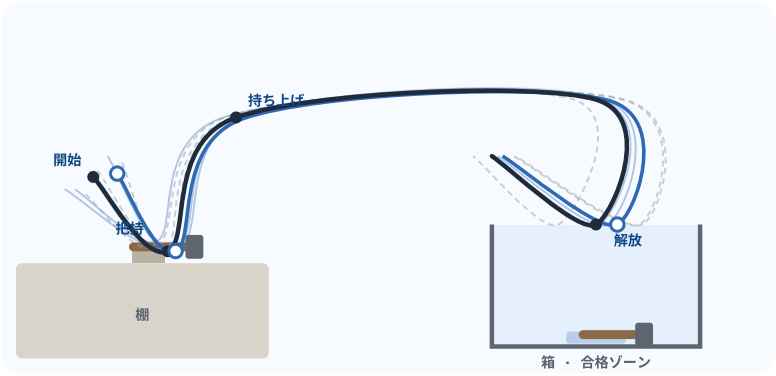
6 解決策 3: Demo Generation

一言でいうと：テレオペデモを 1 つ**テンプレート**とし、重要な点（開始、把持、解放）と速度を少しずらして新しいデモを作り、**シミュレーションで再実行**して、成功したデモだけを残します。これは MimicGen（Mandlekar ら、2023）の手法で、NVIDIA は大規模に用いて GR00T N1 向けに 780,000 本の軌道を生成しました。

試してみる：ハンマー → 箱のデモを生成

側面図 · 棚 → 箱

図解 · 見やすさのためずれを誇張



5	3
採用	除外

直近の生成デモ · #8

時間	×0.96
開始点のずれ	1.2 cm
把持点のずれ	4 mm
解放点のずれ	+1.0 cm

✓ シミュレーションで再実行：ハンマーが箱の中央に入る → 採用

本プロジェクトでの実績：**100 / 133** 回を採用、テンプレートデモ 60 件、デモ 100 件に ~17 分。

テンプレートデモ（テレオペ）　生成デモ、合格　生成デモ、除外（ハンマーが箱の中央に入らない）

7 組み合わせと結論

3 つの手法は互いを置き換えるものではなく、1 つのパイプラインとしてつながります。モデルを実ロボットへ移行す

る段階での組み合わせ方針：

01
実機テレオペ
作業環境で少数のデモ

→

02
Real2sim
シーンを構築し、シミュレーションで再生

→

03
デモ追加生成
件数を増やし、シミュレーションでフィルタ

→

04
Domain randomization
外観と物理を多様化

→

05
Fine-tune & sim2real
シミュレーションで評価し、その後実ロボットで評価

結論

本プロジェクトは 3 つの手法のうち 2 つを検証済みです。**real2sim** は実ロボットのデータを 790 件のシミュレーションエピソードに変換し（66 %）、**デモ追加生成**は ~17 分でテレオペデータを倍増させました（mix200 で 66 %）。**Domain randomization** は visual gap の測定と手法の提案にとどまっており、実ロボットへ移行する前に取り組むべき次のステップです。

III.01 シナリオ Apple & Plate

G1 が棚の上のリンゴをつかみ、皿に置く。

NVIDIA Isaac Lab Arena のサンプルシナリオを、**プロジェクトが独自に構築した制御パイプライン**で再実行しました。パイプラインを層ごとに検証（データセットとの open-loop 比較、デモ再生）したうえで、NVIDIA のチェックポイントを 3 つのシーンで評価しました。結果：元の環境で **92 %**、照明のみ異なる場合 **56 %**、シーン全体が異なる場合 **0 %** – モデルは画像に非常に敏感であり、パイプラインは正しく動作しています。

DATASET

208 エピソード

データ源	NVIDIA · XR テレオペ
フレーム	35,066
周波数	50 Hz

FINETUNE

なし 再学習なし

使用	NVIDIA チェックポイント
調整対象	リンゴ → 皿
指示文	データセット準拠

MODEL

GR00T N1.7 3B

チェックポイント	GN1x-Tuned-Arena
Embodiment	G1 + Dex3
アクション列	40 ステップ · 50 Hz

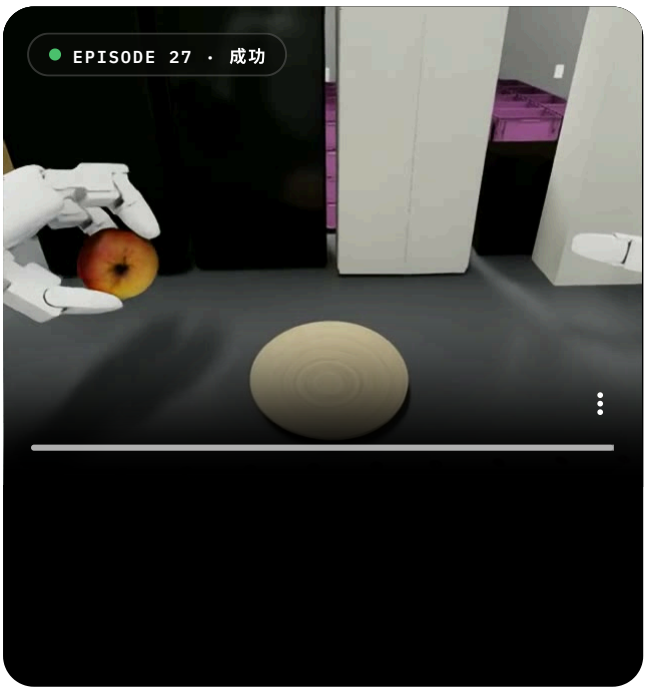
RESULT

92 % 成功

92 % 元の環境	56 % 照明	0 % シーン

同じモデル、実行環境のみ変更

1 Video Showcase



ロボットがリンゴを拾って皿に置く様子（頭部カメラ視点）

モデルへの指示文

> "Pick up the apple from the shelf and place it onto the plate on the same shelf next to it."

2.9 s
成功までの時間

1.0 cm
リンゴ中心と皿中心の距離

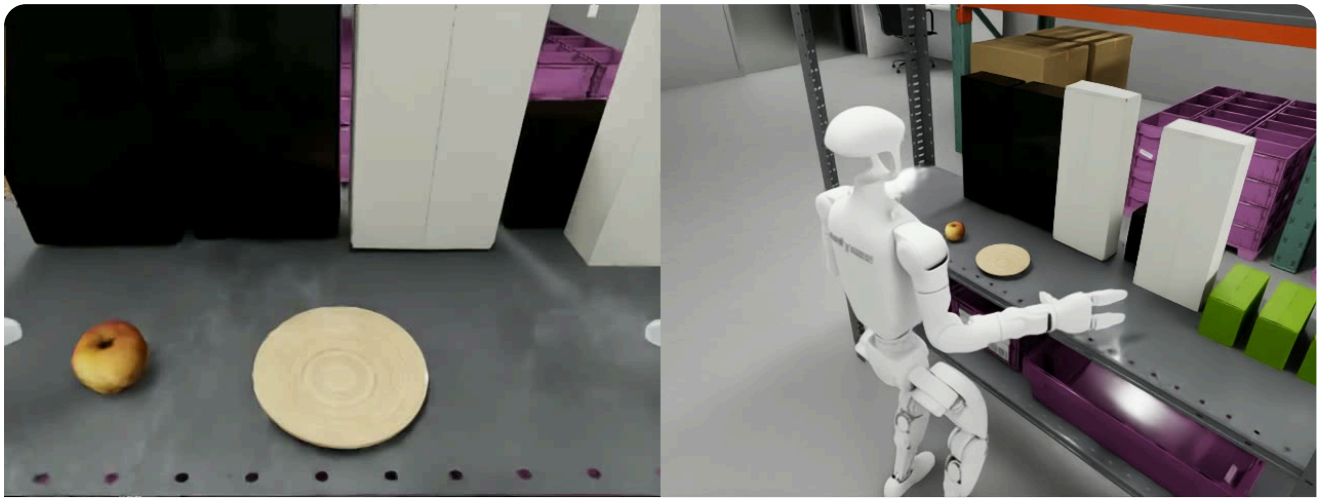
4
モデル呼び出し回数（各 40 ステップ）

1.1 s
呼び出し 1 回あたりの平均遅延

i 再構築した Galileo シーンでの 50 エピソード評価（28/50 成功）、NVIDIA チェックポイントから抜粋。

2 シナリオの状況

G1 ロボットは Arena の Galileo ラボの棚の前に立っています。リンゴと皿は棚の面に並べて置かれています。下の画像は、Video Showcase のエピソードの**最初のフレーム**（ステップ 0 でモデルが受け取る画像）と、斜めからの全景を組み合わせたものです。



フレーム 0 · 左：ロボット頭部カメラ（モデルへの入力画像） · 右：外部カメラからの斜め視点（Isaac Sim で再構築した Galileo シーン、Hammer & Bin シナリオと同じカメラ角度）、説明用のみ。

シーンと物体	詳細
背景	Galileo ラボ（`galileo_locomanip.usd`）。手の届く範囲をふさぐ機器の箱 3 個を非表示（jetson_orin_06、jetson_orin_03、hesai_box_06）
棚の面	非表示の支持板 0.8 × 1.5 × 0.04 m、上面の高さ 0.765 m
リンゴ	位置 (0.58, 0.27) m
皿	位置 (0.58, 0.06) m — リンゴからロボットの右側へ約 21 cm
摩擦	リンゴをしっかり保持するため Dex3 の指の摩擦を増加

ロボット・カメラ・制御	詳細
ロボット	Unitree G1 + Dex3 ハンド、位置 (0.25, 0.08) m、+x 軸方向を向く。脚は AGILE コントローラーでバランスを保持、腰の高さ約 0.73–0.75 m（指令値 0.72 m）
カメラ	`head_link` に取り付け、640×480、焦点距離 15 mm、下向き約 35°
周波数	物理 200 Hz、制御 50 Hz（1 指令あたり物理 4 ステップ）
エピソード	最大 300 ステップ = 6 s。モデル呼び出し 1 回につき 40 ステップのアクション列を返す
指示文	"Pick up the apple from the shelf and place it onto the plate on the same shelf next to it."
成功条件	リンゴ中心が皿中心から 8 cm 未満、皿より 0–4 cm 高く、速度 < 0.1 m/s

3 シナリオの目標

NVIDIA のサンプルシナリオを**比較基準**として使用します。モデルとデータは既に揃っているため、差異はすべてプロジェクトのパイプラインまたは環境に起因します。

目標 1

データ経路の理解

カメラと関節角からモデルへ、モデルから G1 のモーターへの流れを完全に把握します。

目標 2

パイプラインの再実装

実行時に Arena に依存しない独立パッケージを構築します。Arena はシミュレーション内でしか動作しないため、実機へ移行する前に必須の工程です。

目標 3

検証と測定

パイプラインの正しさを層ごとに証明し、NVIDIA チェックポイントの成功率を測定して、失敗の原因を特定します。

4 シナリオのデータ

データセット ARENA-G1-STATIC-PICKNPLACE	値
データ源	Isaac Lab 上の XR ヘッドセットによるテレオペ、50 Hz — NVIDIA チェックポイントのファインチューニングに使われたデータ
エピソード数	208 ファイル（カードの記載は 200、メタデータのインデックスは 251 まで）
総フレーム数	35,066（11.7 分）。各エピソード 2.3–5.3 s
指示文	"Pick up the apple from the shelf and place it onto the plate on the same shelf next to it."
観察	リンゴをつかむのは左手のみ。腰と右手はほぼ静止。ロボットは立ったまま、胴体の高さ約 0.72 m

5 直面した課題

Arena のパイプラインを独自パッケージで再現したところ、制御の各工程はすべて正しく動作しましたが、成功率は

モデルが受け取る**カメラ画像**に強く依存していました。

> **Visual gap：**同じチェックポイント、同じ指示文で、シミュレーションのシーンを変えただけで成功率が **92 %**（NVIDIA クライアント、元の Arena）から **56 %**、さらに **0 %**（本プロジェクトのパイプライン）へ低下しました。

課題 1

データセットと異なるシーンではモデルがタスクを実行できない

独自に構築したシーン（箱型テーブル、グリッド床、dome ライト）では成功 **0/50**、リンゴを持ち上げられたのは 11/50 のみでした。モデルはデモと同様に手を伸ばすものの、把持の指令を一度も出しません。

課題 2

シーンを正確に再構築しても不十分

再構築した Galileo シーンは配置とカメラ角度がデータセットと一致していますが、元の Arena の 92 % に対し **56 %** にとどまります。残る差は主に照明とレンダリング方式です。

課題 3

モデルは 1 つのシーンからしか学習していない

チェックポイントはただ 1 つのシーン、1 つの照明条件で記録したデータでファインチューニングされているため、操作対象の物体ではなくその画像に強く結びついています。

課題 4

実環境への移行が難しい

実機はシミュレーションとまったく異なる照明・背景・材質に直面します。現在の画像への敏感さでは、チェックポイントをそのまま使うことはできません。

6 提案手法

domain randomization の採用を提案します。学習データを生成する際、タスクに関係のない視覚的要素をランダムに変化させ、モデルが背景や照明ではなく物体と手の姿勢に頼らざるを得ないようにします。

domain randomization は **sim2real** でも一般的な手法です。ここではまずシミュレーションのシーン間の差に適用し、実機へ移行する前の前段階とします。

> **状況：**未実施。本カードでは手法と既存パイプラインへの適用方法のみを示します。

ランダム化する要素

外観 – モデルには見えるが、タスクの実行方法には影響しないもの：

照明

強度・色・方向

dome/rect ライトの強度と色温度、照射方向を変化させ、ベイク済みライトのオン／オフを切り替えます。92 % を 56 % に下げたのはまさにこの要因です。

材質

色とテクスチャ

棚／テーブルの面、皿、部屋の背景の色・光沢・テクスチャを変更します。物体の形状は変えません。

シーン

背景と周囲の物体

背景を変更し、操作領域外の箱や装飾物を追加／削除します。

幾何 – モデルが固定の配置を暗記しないよう、視点と物体の位置を変える：

カメラ

姿勢とパラメーター

頭部カメラの位置・俯角・焦点距離をわずかに変動させ、学習時にノイズ、露出、color jitter を加えます。

物体

位置と向き

手の届く範囲内でリンゴと皿の位置・向きをシャッフルします。

適用方法

ステップ 1
ランダムなシーン
Isaac Lab / Replicator のリセットイベントで上記の要素をエピソードごとに変更

→

ステップ 2
デモ再生
208 件のデモのアクションを各シーンで再生し、新しい画像を記録

→

ステップ 3
フィルタリング
成功条件を満たし続けるエピソードを残す

→

ステップ 4
再ファインチューニング
多様化したデータで NVIDIA チェックポイントから

→

ステップ 5
交差評価
Galileo、独自構築シーン、未見のシーン

期待される効果：シーン間の 92 % → 56 % → 0 % の差を縮めることです。指標は独自構築シーン（現在 0 %）と再構築した Galileo（現在 56 %）での成功率で、各構成 ≥ 50 エピソードとします。

7 実施プロセス

Isaac Lab Arena の実行フローを独自パッケージ `standalone_g1_gr00t` で再現しました。Arena の各工程を切り出

し、書き直して notebook で個別に検証し、最後に Arena に依存しない閉ループとして統合しました。

ステップ 1 · ARENA
元の Arena を実行
元の環境での NVIDIA チェックポイント – 比較基準 92 %

→

ステップ 2 · NB01-02
モデルとの通信
ZMQ サーバー、観測とアクションの変換

→

ステップ 3 · NB03-04
ブロックごとの検証
データセットとの open-loop 比較、AGILE 脚制御

→

ステップ 4 · NB05-06
Isaac Sim バックエンド
ロボット、カメラ、リンゴ／皿、デモ再生

→

ステップ 5 · NB07-08
閉ループとパッケージ化
Galileo シーン、runner、安全層

ステップ	作業内容	NOTEBOOK / ツール	結果
1	NVIDIA のクライアントとチェックポイントで Arena パイプラインを実行し、比較基準を得る	Isaac Lab Arena	元の環境で 92 %
2	ZMQ + msgpack プロトコル、modality 契約。関節順序 4 種の対応表、`build_observation`、`decode_action`、`ChunkScheduler`	nb01_server_wire, nb02_joints_obs_action	40 ステップのアクション列を正しく送受信
3	データセットの実画像と状態 → サーバー → 記録済みアクションと比較。ONNX 脚ポリシーの 80 次元観測ベクトルを再構成	nb03_open_loop_dataset, nb04_agile_onnx	誤差は振幅の 1/10–1/70。AGILE はビット単位で一致
4	バックエンドを手作業で構築：SimulationContext → ロボット → 頭部カメラ → リンゴ／皿 → 指の摩擦 → observe/apply。データ経路全体を接続	nb05_backend_anatomy, nb06_full_pipeline	10 s 安定して直立。デモ再生に成功
5	Arena の Galileo シーンを再構築し、GR00T で 50 エピソード実行。runner、policy、`sanitize_chunk`、`RateLimiter` をパッケージ化	nb07_full_pipeline_loop, nb08_runner_policy_safety	56 %（照明の差）、0 %（独自構築シーン）

8 達成した結果

ブロックごとの検証から閉ループ実行まで、プロセスの各ステップで測定した数値です。すべての実行で同じ NVIDIA チェックポイントと同じ指示文を使用しています。

元の Arena
92 %
NVIDIA クライアント · 元の環境

再構築 Galileo
56 %
28/50 · Wilson 42–69 %

独自構築シーン
0 %
0/50 · 持ち上げ 22 %

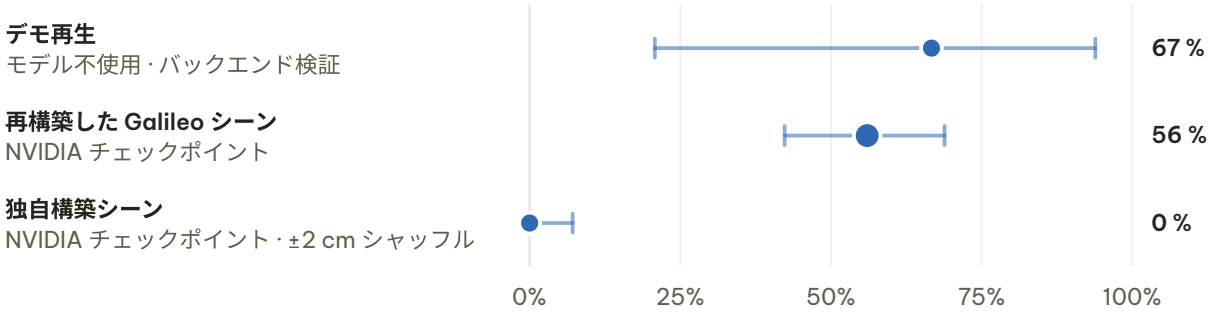
Open-loop 誤差

≤ 0.032 rad

MAE 0.008–0.032 · 振幅の 1/10–1/70

成功率と Wilson 95 % 信頼区間

同じチェックポイントで、シーンのみ変更。デモ再生はモデルを使わず、バックエンドの検証のみ。



ステップ	測定値
モデルとの通信	呼び出しごとに 40 ステップのアクション列。遅延 約 1.2 s/回（50 エピソードの中央値。シミュレーションはモデルを待つため結果に影響しない）
データセットとの open-loop 比較	MAE 左腕 0.032 · 右腕 0.019 · 左手 0.008 rad（3 エピソード、11 chunk）
指示文	データセットの文 0.025 rad、Arena 既定の文 0.047 rad、無意味な文 0.039 rad
AGILE 脚制御	参照実装とビット単位で一致。500 tick 安定して直立、胴体 0.73–0.75 m、最大傾き約 10°
デモ再生	3 回中 2 回成功。リンゴと皿中心の距離 3 cm。関節追従誤差 0.04 rad
閉ループ · Galileo	**28/50 成功。持ち上げ 72 %。リンゴ–皿距離の中央値 3.8 cm。成功までの時間の中央値 2.7 s**
閉ループ · 独自構築	0/50 成功。持ち上げ 22 %。リンゴ–皿距離の中央値 20.6 cm

9 結果の分析

タスクの段階ごとに各実行を比較し、失敗の原因がどこにあるかを特定します。

段階（50 エピソード）	独自構築	GALILEO
リンゴを持ち上げた	11 (22 %)	36 (72 %)
皿に置けた / 持ち上げた数	0 / 11	28 / 36 (78 %)
成功	**0 %**	**56 %**
転倒	3	2

所見 1

視覚が結果を決める

チェックポイントと指示文は同じで、シーンを変えただけで 92 %（NVIDIA クライアント、元の Arena）→ 56 % → 0 %（本プロジェクトのパイプライン）となりました。2 つの信頼区間 0–7 % と 42–69 % は重なりません。

所見 2

失敗は認識と接近の段階にある

独自構築シーンではモデルはほとんどリンゴを持ち上げられません（22 %）。Galileo では、持ち上げた後は 78 % が皿に置けています — 操作スキルは保たれています。

所見 3

独自パイプラインは正しく動作

最終距離 1–6 cm での 28 回の成功と、open-loop および再生による検証により、データ変換の誤りは否定されます。

所見 4

展開への示唆

1 つのシーンでファインチューニングしたモデルは汎化しません。作業環境でデータを収集するか、学習時に画像の多様性を高める必要があります。

失敗 22 回の詳細（Galileo シーン）

12 **リンゴを持ち上げられない** –

9 回はリンゴがほぼ動かず、2 回はリンゴが押されて 4–9 cm 滑り、1 回はリンゴが棚から弾き落とされました。

考えられる原因：把持の滑り、または接近の誤り。初期位置は同一 ⇒ 差はモデルのサンプリングのランダム性に起因します。

8 **持ち上げたが置く位置がずれた** –

6 回は皿から 9–20 cm の位置で離しました（最も近いもので 9.1 cm、8 cm のしきい値をわずかに超過）。2 回はリンゴを棚から落としました。

2 **エピソード開始直後に転倒** –

リンゴに触れる前の tick 7 と tick 30 で転倒しました。最初の腕の chunk が、リセット後に AGILE がまだ安定化している期間と重なっています。モデルに制御を渡す前に、ロボットを数十 tick「ウォームアップ」させることを提案します。

除外した要因

遅延

シミュレーションはサーバーを待ちます。成功エピソードと失敗エピソードの遅延は同等です。

安全層

chunk の境界で腕の関節をクリップするのみで、手や腰をクリップしたことは一度もありません。

パイプラインの不具合

最終距離 1–6 cm での 28 回の成功 ⇒ データの連鎖は正しく動作しています。

10 シナリオの結論

結論

1 つのシーンでファインチューニングしたモデルは**画像に非常に敏感**です：データ記録時の元の環境では 92 %、照明だけが異なると 56 %、まったく異なるシーンでは 0 % です。実運用では、作業環境で直接データを収集するか、学習時に画像の多様性を高める必要があります。

- パイプラインを**習得し、独自に再現**しました。ソースコードは実機バックエンドへの差し替えに対応済みです。
- データ記録時と同じ指示文を使う必要があります。結論を出すには各構成で ≥ 50 エピソードが必要です。
- 未対応：92 % の基準と直接比較するため、元の Docker Arena 内で本プロジェクトのパイプラインを実行すること、±2 cm シャッフルを加えた Galileo シーン、リセット後のウォームアップ段階。

III.02 シナリオ Multi Fruit

G1 が指示文どおりの果物を選び、皿に置く。

テーブルには 4 種類の果物（リンゴ、洋ナシ、スターフルーツ、ブドウ）があり、指示文によってつかむ果物が決まります。実機 G1 の 1,095 件のテレオペ軌道から、本プロジェクトは**関節データのみからシミュレーションシーンを再構築**し（real2sim）、790 件の学習エピソードを生成して GR00T をファインチューニングしました。結果：流用したリンゴ用チェックポイントは **0 %**、ファインチューニング後のモデルは妨害果物を含むランダムシーンで **66 %**、**誤った果物をつかんだ回数はゼロ**。

DATASET

790 エピソード

データ源	Real2sim · 実機 G1
フレーム	86,822
テストセット	89 エピソード · 20 Hz

FINETUNE

20,000 ステップ

学習時間	2 時間 34 分
Batch · GPU	32 · 1× H100
Loss	1.50 → 0.019

MODEL

GR00T N1.7 3B

チェックポイント	`g1_fruits_n17`
指示文	果物 4 種類
アクション列	16 ステップ · 20 Hz

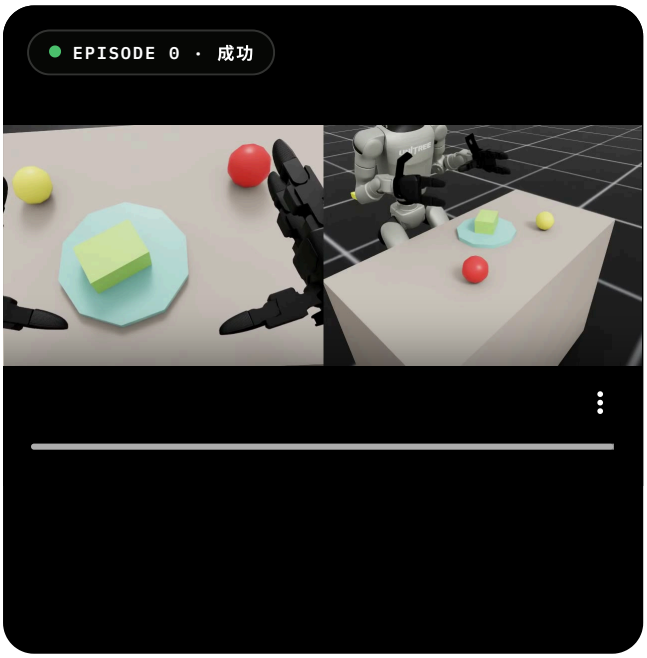
RESULT

66 % 成功

0 %	66 %
未調整	FINE-TUNE

誤った果物をつかんだ回数はゼロ

1 Video Showcase



ロボットが他の果物の中からブドウの房を正しく選び、皿に置く

> モデルへの指示文
> `"Pick up the green grapes and place it on the plate"`

1.4 cm
果物中心と皿中心の距離

16.5 cm
最大持ち上げ高さ

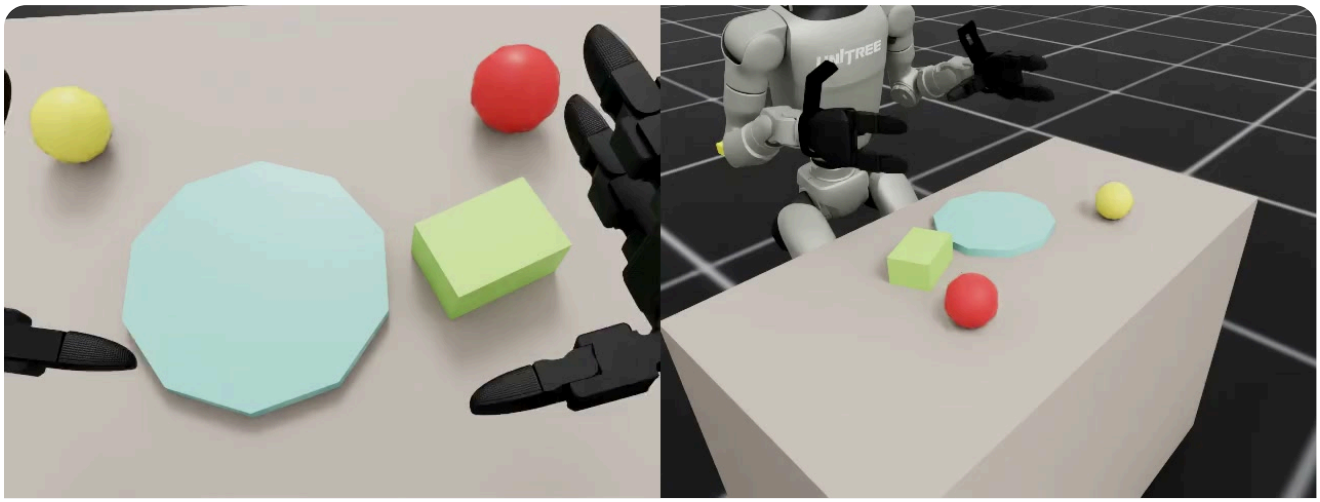
2
妨害果物、ずれなし

10 s
エピソード長（200 ステップ × 50 ms）

ランダム real2sim 50 エピソードの評価（33/50 成功）から抜粋。ブドウは緑のブロック。

2 シナリオ設定

G1 ロボットはテーブルの前に固定して立っています。テーブル上には皿 1 枚と果物 3 個があり、1 個が目標、残り 2 個が妨害物です。下の画像は Video Showcase のエピソード（エピソード 0：緑のブドウ）の**最初のフレーム**です。



フレーム 0 · 左：ロボット頭部カメラ、モデルが受け取る画像 · 右：外部カメラ、動画撮影専用。目標はブドウ（緑のブロック）、妨害物はリンゴと洋ナシ。

シーン & 物体	詳細
テーブル	グリッド床上のシンプルなテーブル。照明は dome 700 + sun 600
皿	kinematic の皿、半径 0.115 m。エピソード 0 では (0.40; 0.08) m に配置
果物	シンプルなブロック形状の 4 種類（リンゴ、洋ナシ、スターフルーツ、ブドウ = 緑のブロック）。摩擦 1.5
目標	エピソード 0：ブドウ、(0.41; -0.12) m
妨害物	手の掃引範囲外に置いた果物 2 個（エピソード 0：リンゴ、洋ナシ）
ランダム化	エピソードごとに目標果物、果物の位置と向き、皿の位置、妨害果物を変更。seed は個別

ロボット、カメラ & 制御	詳細
ロボット	Unitree G1 29 自由度 + Dex3 ハンド、腰を (0; 0; -0.74) m に固定
カメラ	頭部搭載 RealSense D435、640×480（fx 606）+ 撮影用外部カメラ 1 台
周波数	物理 200 Hz、10 サブステップ → 制御 20 Hz
エピソード	最大 200 ステップ = 10 s。16 ステップのアクション列を受け取り、8 ステップ実行して再度問い合わせ
指示文	目標果物に応じて決定。例："Pick up the green grapes and place it on the plate"
成功条件	果物を 3 cm 超持ち上げ、皿の上にある（半径 + 1 cm 以内、最下点がテーブル面より上）、妨害果物の移動 ≤ 1 cm

3 シナリオの目標

実機 G1 のテレオペデータをシミュレーションに移し（**real2sim**）、指示文どおりの果物を選ぶモデルを学習・評価します。

目標 1

閉ループ評価

実機データしかなかったタスクにシミュレーション環境を用意。

目標 2

データの増量

追加のテレオペなしで妨害物、位置、照明を変更。

目標 3

再現可能な測定

チェックポイントと学習構成を同一の尺度で比較。

4 シナリオのデータ

データセット **PhysicalAI-Robotics-GR00T-Teleop-G1**（NVIDIA GEAR、CC-BY-4.0）：実機 G1 のテレオペ、RealSense カメラ 640×480、20 Hz。ロボットが指示文どおりの果物を選び、皿に置きます。データは**関節角と動画のみ**で、物体の位置は含まれません。

果物	エピソード	フレーム
リンゴ	311	29,864
ブドウ	233	26,970
洋ナシ	264	32,319
スターフルーツ	287	34,640
合計（データセットカードの記載は 1,000）	**1,095**	**123,793**

5 直面した課題

目標は指示文どおりの果物をつかむモデルです。独自にデータを作る前に、既存のデータセットでのファインチューニングを試しました。

> **適切なシミュレーションデータセットがない。** G1 ロボットかつピック&プレースという条件に合う唯一のデータセットは**実機**で記録されたもので、Isaac Sim でそのまま使用できません。

課題 1

既存チェックポイントは流用不可

シナリオ 1 のリンゴ用チェックポイントに果物 4 種類の指示文を与えると **0 %** でした：モデルは 1 つのシーンで 1 つのタスクしか学習していません。

課題 2

シミュレーションデータセットの不足

試験的なファインチューニング用に公開データセットを調査しましたが、G1 + Dex3 で指示文に応じて複数種類の果物をピック&プレースするシミュレーションデータは存在しません。

課題 3

唯一適合するのは実機データ

PhysicalAI-Robotics-GR00T-Teleop-G1：同じ G1 ロボット、同じく果物を皿にピック&プレースしますが、実機で記録されています。

課題 4

実機データにはシミュレーションに必要な情報が不足

関節角と動画のみで**物体の位置がありません**。実画像はレンダリング画像と一致せず、実機とシミュレーションでは action の意味が異なります。

6 提案手法

real2sim で解決します：実データの関節軌道から対応するシミュレーションシーンを再構築し、Isaac Lab で再生してシミュレーションデータセットとして記録します – テレオペ操作者の実際の動きを保ちつつ、画像と物理は評価環境と一致させます。

ステップ 1
順運動学（FK）
URDF G1 29-DoF + Dex3 → 胴体姿勢、手の軌道、把持/解放のタイミング

→

ステップ 2
物体位置の推定
果物 = 把持時の手のひら、皿 = 解放時の位置

→

ステップ 3
安全な妨害物
手の掃引範囲、果物の運搬経路、皿の外に配置

→

ステップ 4
再生 & フィルタ
Isaac Lab。成功し、妨害物に触れないエピソードのみ保持

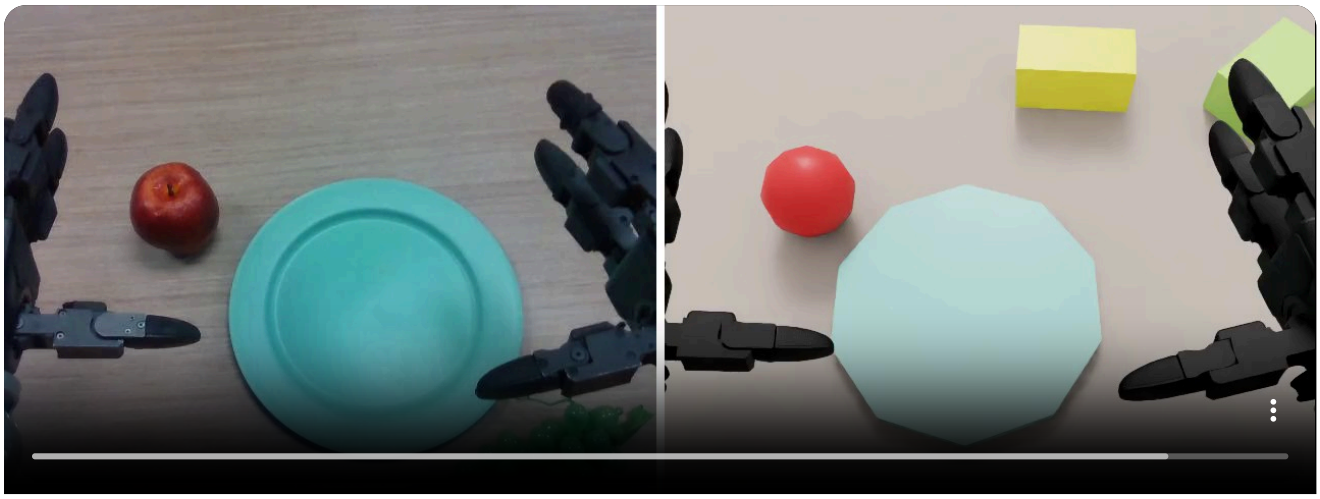
→

ステップ 5
LeRobot データセット
果物 4 種類のマルチタスク。元エピソード単位で train/test を分割

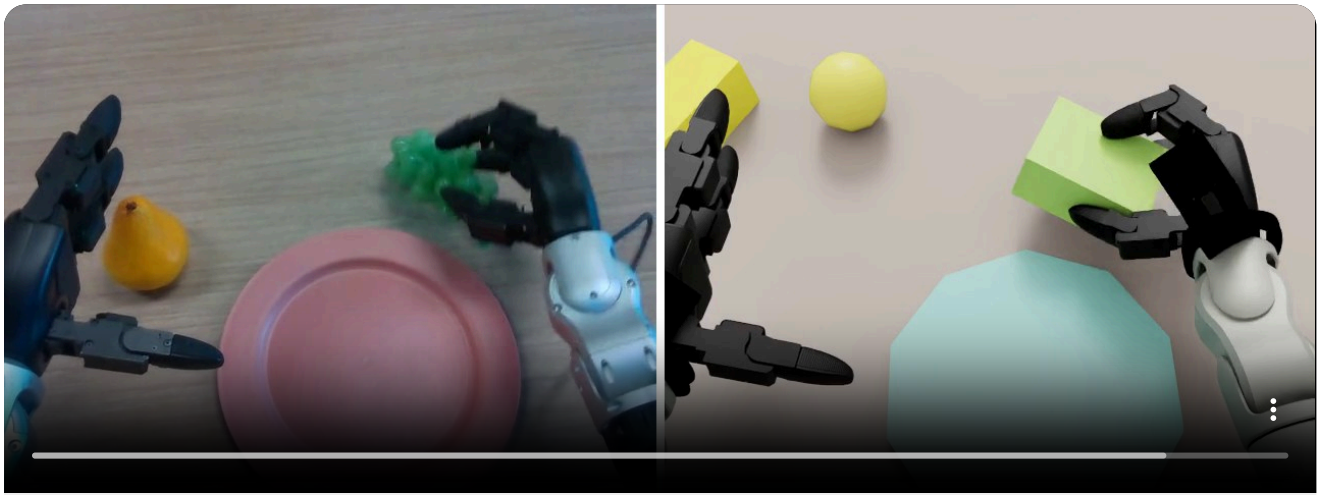
- **視覚を使わない再構築：**両足が床に平らに接していると仮定して胴体の傾きを推定。指の閉じ具合から把持/解放を検出。FK は numpy のみで独自実装。
- **シミュレーション：**G1 + Dex3 で腰を固定、RealSense の仕様に合わせた D435 カメラ、物理 200 Hz / 制御 20 Hz、腕は剛な PD、果物の摩擦 1.5、皿は固定。
- **採点：**果物を 3 cm 超持ち上げる、最終的に皿の上にある、どの妨害物も 1 cm 超動かない。

- **データリーク防止：**テストシーンは train に一切含めない。再生に失敗したシーンも含めて評価シーン 100 件を保持し、容易なシーンへの偏りを避ける。

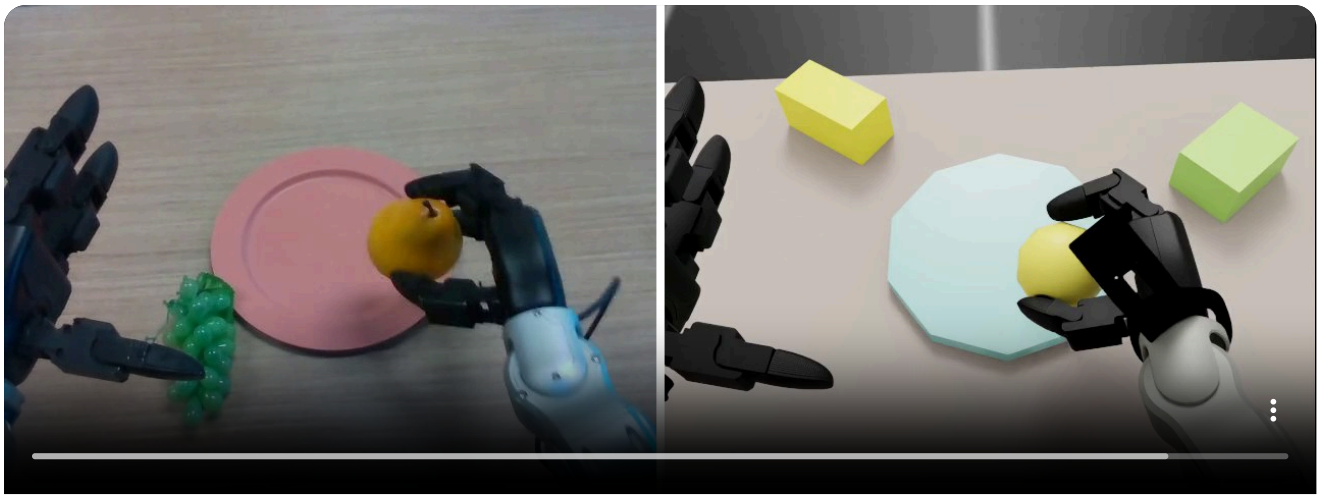
実機 ↔ シミュレーションの動画 3 組。 ロボット頭部カメラ、20 Hz：左は元のテレオペ動画、右は同じ関節軌道から再生した real2sim エピソード。2 つのエピソードは長さが等しいためフレーム単位で同期して再生されます。各動画のポスター画像は 1 つのフェーズ（開始、把持、配置）の代表フレームです。



組 1 · リンゴ · 実エピソード 0 ↔ sim 0 · 111 フレーム（5.6 s）。視覚を使っていないにもかかわらず、最初のフレームからシミュレーション内のリンゴと皿の位置は実画像とほぼ一致する：リンゴは把持時の手のひらから、皿は解放時の位置から推定。実画像の隅にあるブドウの房は関節データに含まれないため再構築されず、シミュレーションでは新たな妨害物 2 個（スターフルーツ、ブドウ）を手の掃引範囲から離れた位置に置く。左：実機 · 右：シミュレーション。



組 2 · ブドウ · 実エピソード 1 ↔ sim 0 · 102 フレーム（5.1 s）。関節角が同じであるため、手を伸ばす、指を閉じる、持ち上げる際の腕と Dex3 ハンドの姿勢はいずれも実動画と一致する。柔らかいブドウの房は剛体の緑のブロックに置き換え。左側の実物の洋ナシは再構築されず、シミュレーションでは洋ナシとスターフルーツを別の位置に妨害物として置く。左：実機 · 右：シミュレーション。



組 3 · 洋ナシ · 実エピソード 0 ↔ sim 0 · 131 フレーム（6.6 s）。果物は実動画と同じタイミングで皿の領域に運ばれる。実物の皿はピンク、シミュレーションの皿は常にターコイズの多角形。木製の天板は滑らかな平面になり、シミュレーションの視点ではテーブルの縁と背後の背景まで見える。左：実機 · 右：シミュレーション。

共通点

動作とタスク配置

腕の軌道、指の閉じ具合、把持/解放のタイミングはテレオペのまま。目標果物と皿の位置は実画像と一致。左手は同じ姿勢で静止。指示文は原文のまま。

相違点

画像と妨害物

果物はシンプルな幾何ブロック。皿、天板、照明はシミュレーションのもの。実画像の妨害物は除外し、安全に配置したランダムな妨害物 2 個に置き換え。カメラ角度はわずかにずれる。このためモデルはレンダリング画像のみで学習し、実画像は使わない。

セット	エピソード	フレーム	備考
Train	**790**	**86,822**	**果物 4 種類、20 Hz**
Test	89	–	train に含まれないシーン
評価シーン	100	–	リンゴ 30、洋ナシ 24、スターフルーツ 26、ブドウ 20

7 実施プロセス

元データセットは実機 G1 のテレオペであり、シミュレーションでそのまま実行できません：**real2sim 変換**で対応するシーンとシミュレーションデータを得て、そのデータで GR00T を**ファインチューニング**し、その後に Isaac Sim

で閉ループを実行します。

ステップ 1
実データ
G1 テレオペ軌道 1,095 件、関節角 + 動画のみ

→

ステップ 2
real2sim 変換
FK で果物と皿の位置を推定。Isaac Lab で再生 & フィルタ

→

ステップ 3
シミュレーションデータセット
マルチタスク LeRobot · train 790 / test 89

→

ステップ 4
ファインチューニング
GR00T N1.7 · 20,000 ステップ · 1× H100

→

ステップ 5
実行 & 評価
ランダムシーン、50 エピソード

ステップ	作業内容	NOTEBOOK / ツール	結果
1	果物 4 種類（リンゴ、洋ナシ、スターフルーツ、ブドウ）のテレオペデータセットを取得。シナリオ 1 のリンゴ用チェックポイントを試験	PhysicalAI-Robotics-GR00T-Teleop-G1	流用したリンゴ用チェックポイント：0 %
2	URDF 順運動学 → 手の軌道、把持/解放のタイミング → 果物と皿の位置。妨害物を手の掃引範囲外に配置。Isaac Lab で action を再生し、成功エピソードを保持	`g1_real2sim`（urdf_fk、scene、distractors、isaac_scene）	実軌道ごとのシミュレーションシーン
3	果物 4 種類を 1 つのマルチタスクデータセットに統合。元エピソード単位で train/test を分割。評価シーン 100 件を保持	make_groot_dataset、lerobot_io	train 790 / 86,822 フレーム · test 89
4	GR00T N1.7-3B をファインチューニング、`NEW_EMBODIMENT`、state/action 31 次元、horizon 16 ステップ @ 20 Hz、batch 32	H100 サーバー上の launch_finetune.py	`g1_fruits_n17` · 2 時間 34 分 · loss 1.50 → 0.019
5	データセットと同様のシーンを構築し、指示文を選択、モデルが自律的に把持。個別 seed で N エピソードを実行	nb09_multi_object_task_pipeline、nb10_eval_multi_object	66 %、誤った果物の把持はゼロ

8 ファインチューニング

NVIDIA 公式ガイド（fine-tuning on sim data）に従いました：GR00T N1.7-3B、`NEW_EMBODIMENT`、カメラ `rs_view`、state/action 5 グループ（31 次元）、horizon 16 ステップ @ 20 Hz（0.8 s）。H100 で batch 32。LLM は凍結、視覚 + projector + diffusion を調整。color jitter（明るさ 0.3 · コントラスト 0.4 · 彩度 0.5 · 色相 0.08）。

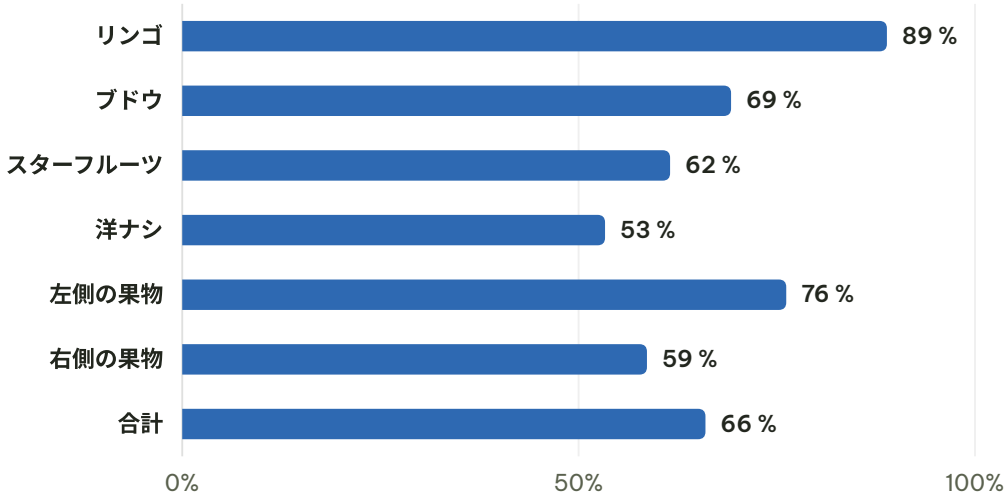
9 得られた結果

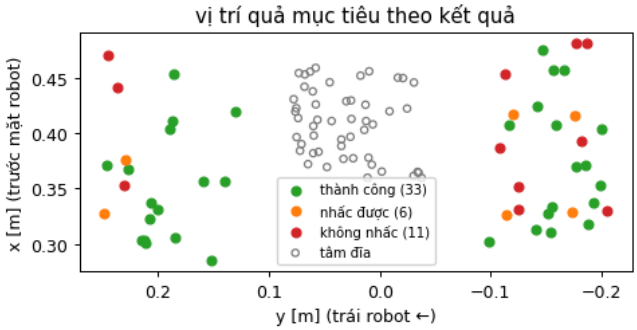
各エピソードで目標果物、果物の位置/向き、皿の位置、妨害果物をランダム化し、エピソードごとに個別の seed を使用します。指示文はデータセットの原文です（例："Pick up the green grapes and place it on the plate"）。16 ステップを受け取り、8 ステップ実行して再度問い合わせ、最大 10 s です。50 エピソード、技術的エラーは 0 件でした。

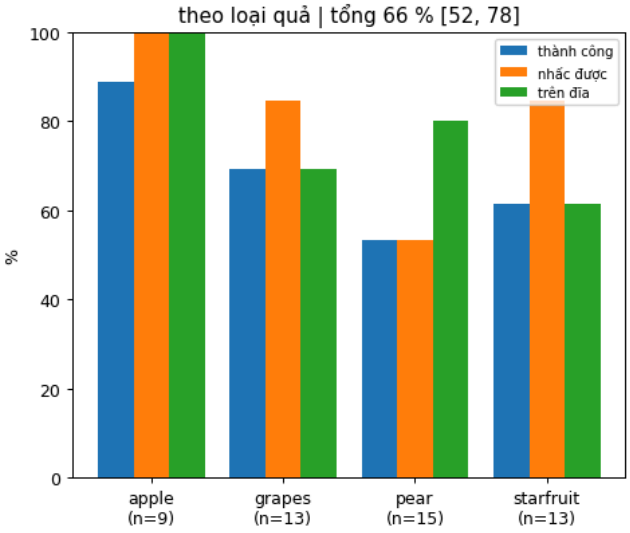
モデル	成功	WILSON 95 %	持ち上げ	皿の上	誤把持
real2sim ファインチューニング	**33/50 (66 %)**	**52–78 %**	**78 %**	**76 %**	**0 %**
Arena のリンゴ用チェックポイント（対照）	0/50 (0 %)	0–7 %	0 %	0 %	0 %

グループ別成功率 · real2sim モデル

バーにマウスを重ねると試行回数を表示。n が小さい（9–29）⇒ 信頼区間が広い。







結果別の目標果物の位置（緑 = 成功、オレンジ = 持ち上げたが不成功、赤 = 持ち上げ失敗、灰色の円 = 皿の中心）と果物別の成功率。

所見

モデルは指示文を理解している

テーブル上に常に他の果物があるにもかかわらず、誤った果物をつかんだ回数はゼロでした。

所見

失敗は主に持ち上げフェーズ

11/50 が持ち上げ失敗でした。洋ナシの持ち上げ率が最も低くなっています（53 %）。

注意

対照は別タスクのモデル

ファインチューニング前の N1.7 そのものではありません。0 % は別タスクのモデルを流用できないことを示すにとどまります。

技術的教訓：最初の試験実行では、制御ステップごとにカメラをレンダリングしない構成（Isaac Lab 3.0 固有）を使っていました – モデルは静止した 1 枚の画像を見ていました。これを発見して修正し、その試験の結果は使用していません。

10 結果の分析

ファインチューニング後のモデルの 50 エピソードをフェーズごとに追跡し、どこで損失が生じるかを確認します。

フェーズ（50 エピソード）	エピソード数	割合
目標果物を持ち上げ	39	78 %
持ち上げ、果物が皿の上	34	68 %
成功（妨害物を動かさない）	**33**	**66 %**

失敗の種類（17 件）	件数	症状
持ち上げ失敗	11	把持が滑る、または手が十分に閉じない。洋ナシの持ち上げ率が最も低い（53 %）
持ち上げたが皿に載らない	5	皿から外れて解放。スターフルーツは持ち上げ 85 % にもかかわらず皿に載る率が最も低い（62 %）
皿に載ったが妨害物が移動	1	手が妨害果物に触れ 1 cm 超移動

考察 1

モデルは指示文を理解している

常に他の果物がある 50 エピソードで誤把持 0 件：言語部分は real2sim データからよく学習されています。

考察 2

ボトルネックは把持フェーズ

失敗 17 件中 11 件は果物を持ち上げられなかったものです。今後の改善は把持データ（果物ごとの形状、摩擦）に注力すべきです。

考察 3

果物の種類とテーブル側による差

リンゴ 89 %、ブドウ 69 %、スターフルーツ 62 %、洋ナシ 53 %。左側 76 %、右側 59 %。n が小さく信頼区間が重なるため、結論には不十分です。

考察 4

結論の範囲

学習と評価はいずれもシミュレーション内：結果は相対比較のためのものであり、実機での性能を予測するものではありません。

11 シナリオの結論

結論

関節角と動画しかない実機 G1 のテレオペデータから、本プロジェクトは**real2sim によりシミュレーションデータセットを構築**し（790 エピソード）、GR00T をファインチューニングしました：流用したリンゴ用チェックポイントの **0 %** から、妨害果物を含むランダムシーンで **66 %** に向上し、**誤った果物をつかんだ回数はゼロ**でした。モデルは指示文を理解しており、残る損失は主に把持フェーズにあります。

- **関節データのみからの real2sim** パイプラインを構築：順運動学で果物と皿の位置を推定し、妨害物を安全に配置、Isaac Lab で再生とフィルタを自動実行。
- H100 1 枚で GR00T N1.7 を 2 時間 34 分でファインチューニング。失敗 17 件中 11 件は果物を持ち上げられなかったもので、洋ナシの持ち上げ率が最も低い。
- 結果はシミュレーション内での**相対比較**にのみ用いる：シミュレーションは現実より容易（剛な PD、高い摩擦、固定された皿）であり、実機での性能は予測しない。
- 今後の方向性：妨害物の配置、照明、材質を変えて再生することで安価にデータを増やし、果物ごとの形状と摩擦に応じた把持データを追加する。

III.03 シナリオ Hammer & Bin

G1 が棚の上のハンマーをつかみ、箱に入れる。

本プロジェクトで新たに設計したタスクです。左手でハンマーの柄をつかみ、箱の縁を越えて持ち上げ、箱の中央に置いて手を離します。ハンマーは**長く薄く、棚の面に近い**ためリンゴより難しく、箱の縁に引っかかりやすいのが特徴です。環境を構築し、Meta Quest ヘッドセットで **100 件のテレオペデモ**を収集、デモを追加生成して GR00T をファインチューニングしました（H100 1 枚で約 1 時間）。全工程は約 6 日間です。結果：未調整 **0 %** → 200 エピソード **66 %** → 400 エピソード **63 %** → 静止しきい値を緩和 **73 %**。

DATASET

200 · 400 エピソード

データ源	**テレオペ + 生成**
フレーム	**28,022 · 54,541**
周波数	**50 Hz**

FINETUNE

10k · 20k ステップ

学習時間	**61 分 · 2 時間 15 分**
Batch · GPU	**32 · 1× H100**
最終 loss	**0.020 · 0.019**

MODEL

GR00T N1.7 3B

mix200	`hammer_mix200_n17`
mix400	`hammer_mix400_n17`
アクション列	**40 ステップ · 50 Hz**

RESULT

73 % 成功

0 %	66 %	63 %	73 %
ベース	200 …	400 …	しき…

静止しきい値を 0.01 → 0.02 に緩和

1 Video Showcase



ロボットがハンマーの柄をつかみ、箱の縁を越えて持ち上げ、箱の中央に置いて手を離す

> モデルへの指示文
> `"put the hammer in the bin"`

3.0 s
成功までの時間

12.8 cm
ハンマーの最大持ち上げ高さ

0.7 cm
ハンマー中心と箱中心の距離

0 N
手がハンマーに加える力：解放済み

i 最良構成（73/100）での 100 エピソード評価から抜粋。

2 シナリオの状況

G1 ロボットは Galileo ラボの棚の前に立っています。ハンマーは左側にあり、ヘッドが小さなブロックの上に載っています。赤いプラスチック箱は右側にあります。下の画像は Video Showcase のエピソードの**最初のフレーム**です。



フレーム 0 · 左：ロボット頭部カメラ（モデルへの入力画像） · 右：外部カメラ（動画撮影専用）。

シーンと物体	詳細
背景	Galileo ラボ。手が衝突しないよう shield_controller_box_02、jetson_orin_05、jetson_orin_07 の箱を非表示（隙間 ≥ 2 cm）
ハンマー	`hammer_2_handal_robolab` 33 × 13 × 3 cm、位置 (0.63, 0.25) m、yaw 0。柄をつかみやすいようヘッドを 5 cm のブロックに載せる（約 10° 傾斜）
箱	`red_container` 縮尺 0.65 → 40 × 27 × 5.9 cm、位置 (0.68, 0.00) m
距離	ハンマーと箱の縁の距離 5.1 cm

ロボット・カメラ・制御	詳細
ロボット	Unitree G1 + Dex3、位置 (0.25, 0.08) m。左手で柄をつかみ、高さ 5.9 cm の箱の縁を越えて持ち上げ、右へ約 25 cm 移動して離す
カメラ	ロボット頭部カメラ（モデル入力）+ 撮影用の外部カメラ 1 台
周波数	制御 50 Hz。モデル呼び出し 1 回につき 40 ステップのアクション列を返す
エピソード	最大 300 ステップ = 6 s
指示文	"put the hammer in the bin"
成功条件	ハンマー中心が箱中心から 10 cm 以内かつ縁より低い。ハンマーが静止（< 0.01 m/s、< 0.1 rad/s – のちに 0.02 m/s へ緩和）。手を離している（手の 22 リンクすべて ≤ 0.01 N）。10 ステップ連続で維持

3 シナリオの目標

このシナリオでは、プロジェクトが自ら構築したツールとデータで**ロボットに新しいタスクをゼロから教える**能力を検証します。

目標 1

まったく新しいタスク

既存のデータもモデルもないタスクを自ら設定する：棚のハンマーをつかみ、箱に入れる。

目標 2

全工程を通す

環境構築、テレオペによるデモ収集、データ拡張、ファインチューニング、評価のすべての工程を実施する。

目標 3

コストと効果の測定

必要なデータ量と学習時間、そして各改善が成功率にどれだけ寄与するかを把握する。

4 シナリオのデータ

新しいシナリオのため既存のデータセットはありません。Isaac Lab 上で **Meta Quest ヘッドセットによるテレオペ**（50 Hz）でデモを自ら収集し、生成デモ（カード 6）を追加しました。すべてのデモは学習に使う前に厳格な成功条件で再生・確認しています。

データセット HAMMER_TO_BIN	値
データ源	Isaac Lab 上の Meta Quest テレオペ、50 Hz。最初の実デモ 100 件を 10 セッションで収集
実デモ	約 13,700 ステップ。ハンマーの平均持ち上げ高さ 10.6 cm。最終時のハンマー中心と箱中心の距離 3.7 cm（最大 8.4 cm）
mix200	200 エピソード（実 100 + 拡張 100）、28,022 フレーム（約 9.3 分）、LeRobot v2.1、131 MB
mix400	400 エピソード（mix200 + 新規テレオペデモ 200）、54,541 フレーム（約 18 分）
指示文	"put the hammer in the bin"
確認	全デモが成功。mix200：持ち上げ高さ最小 6.4 cm、最終距離の平均 3.8 cm

5 直面した課題

ハンマー → 箱のタスクはまったく新しく、データもモデルも既成のシーンもありません。最大の難しさは、ファイン

チューニングに十分な質の良いデータを用意することです。

> **出発点が何もない。**データはすべてテレオペで自ら収集する必要があり、これが最も手間のかかる工程です。実デモ 100 件に 10 セッションを要しました。

課題 1

既存データがない

G1 でハンマーをつかんで箱に入れるタスクのデータセットもチェックポイントも存在しません。未調整のベースモデルではタスクを実行できません。

課題 2

テレオペは手間がかかり遅い

操作者が Meta Quest を装着してデモを 1 件ずつ実演します。1 セッションで約 10 件しか取れず、十分なデータには多くのセッションが必要です。

課題 3

テレオペが不安定

コントローラーの信号が途切れるとロボットの手が止まり、2 回目以降はカクつきが発生します。多くの試行を破棄してやり直しました。

課題 4

デモの品質にばらつき

ハンマーが箱の縁に引っかかったまま、または手を離していないのに成功と判定されたデモがありました。不良デモが混ざるとモデルが誤った動作を学習します。

6 提案手法

データの追加生成で解決します。実際のテレオペデモから MimicGen 方式で新しいデモを自動生成し、シミュレーションで再生して、厳格な成功条件を満たすものだけを残します。テレオペ時間を増やさずにデータを増やせ、学習に使うデモはすべて検証済みになります。

生成方法

テンプレートデモの変形

実デモをテンプレートとして使用（60 種類）。時間を ×0.85–1.2 で伸縮し、開始点（σ 1 cm）、把持点（σ 4 mm）、解放点（±1–1.2 cm）をずらして min-jerk 軌道でつなぎ、0.2–0.6 mm の手の揺れを加えます。

フィルタリング

133 回中 100 回を採用

生成した各デモをシミュレーションで厳格な成功条件のもと再生し、133 回中 100 回が合格して mix200 に加えられました。

指標	拡張	実
エピソード長（ステップ）	145	137
ハンマーの持ち上げ高さ	10.5 cm	10.6 cm
箱中心までの距離（平均）	4.0 cm	3.7 cm
箱中心までの距離（最大）	8.0 cm	8.4 cm



拡張デモの一例：最初・中間・最後のフレーム。主要なポイントでの姿勢は実デモとの差が 1 cm 未満。60 種類のテンプレートデモを使用。

7 実施プロセス

まったく新しいシナリオで、既存のデータもモデルもありません。環境を構築し、**テレオペでデータを自ら作成、データ拡張**で量を増やし、GR00T を**ファインチューニング**してから、閉ループで実行・評価しました。

ステップ 1 · 10/07
環境設計
物体、配置、支えブロック、成功条件

→

ステップ 2 · 10/08-10
テレオペでデモ収集
Meta Quest · 実デモ 100 件

→

ステップ 3 · 10/08-10
データ拡張
MimicGen 方式で生成 · 133 中 100 採用

→

ステップ 4 · 10/12
ファインチューニング
mix200 10k ステップ · mix400 20k ステップ

→

ステップ 5 · 10/12-19
実行と評価
Arena とローカル Isaac Sim · 50–100 エピソード

ステップ	作業内容	NOTEBOOK / ツール	結果
1	ハンマーと箱を選定、ハンマーを 5 cm のブロックに載せる、手の邪魔になる背景の箱を非表示、厳格な成功条件を記述	Isaac Lab Arena, hammer_to_bin/scripts	最終配置、隙間 ≥ 2 cm
2	プリセットとテレオペ診断ツールを使い Meta Quest でデモを収集、各デモを厳格な条件で再生	teleop_kit, record_demos.sh	実デモ 100 件（10 セッション）、のちに 200 件追加

ステップ	作業内容	NOTEBOOK / ツール	結果
3	テンプレートデモを変形（時間、把持／解放点、手の揺れ）、再生してフィルタリング	gen_scripted_demos.py	拡張デモ 100 件 → mix200
4	GR00T N1.7-3B をファインチューニング。LLM は凍結、視覚 + projector + diffusion を調整	H100 サーバー	mix200：61 分 · mix400：2 時間 15 分
5	閉ループで N エピソード実行、失敗を分類、動画を撮影	Arena eval, nb11_hammer_to_bin, nb12_eval_hammer_to_bin	ベース 0 % → 66 % → 63 % → 73 %

8 Fine-tune

学習時間

61 分

1× H100 · 0.36 s/ステップ · 約 53 GB VRAM

ステップ数 · batch

10k · 32

約 11 epoch · 2k ステップごとにチェックポイント

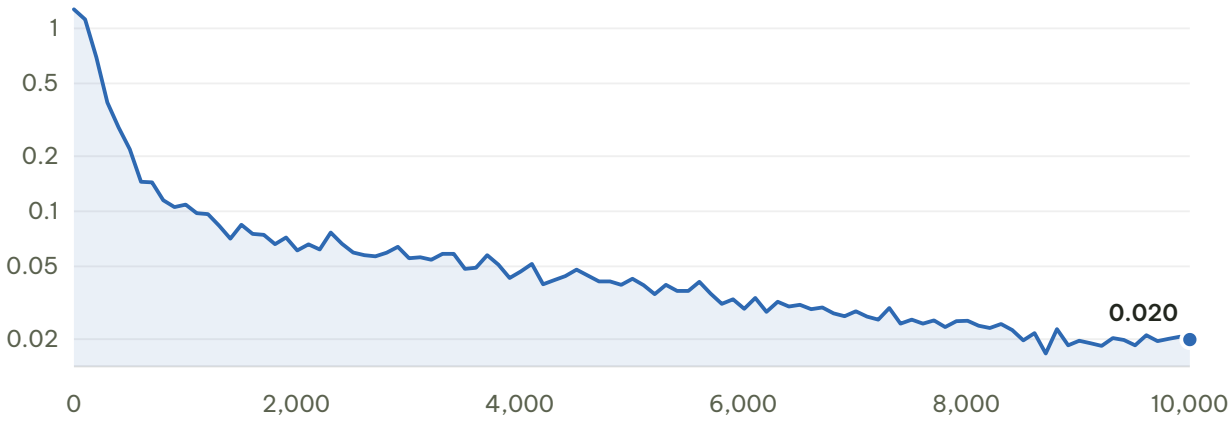
最終 loss

0.020

ステップ 10 の 1.27 から

学習 loss · 対数スケール

mix200 での GR00T N1.7-3B。LLM 凍結、視覚 + projector + diffusion を調整。マウスを重ねると各ステップを表示。



9 達成した結果

テレオペ → 拡張 → ファインチューニング → 評価の各工程で測定した数値です。評価はローカルの Isaac Sim で実施

（`inside` 条件：ハンマー中心が箱の内側、静止、手を離している）。

最良 · ローカル
73 %
mix400 · しきい値 0.02 · 73/100

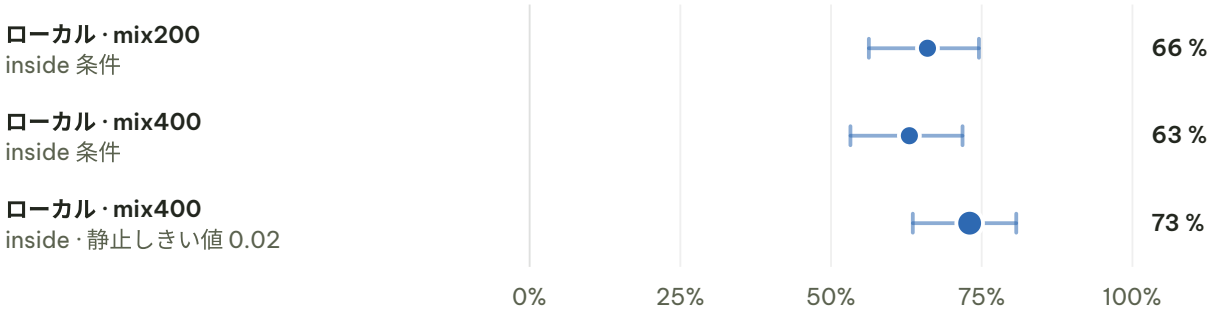
ローカル · mix200
66 %
66/100 · Wilson 56–75 %

データ
400 デモ
54,541 フレーム · テレオペ 300 + 生成 100

Fine-tune loss
0.019
mix400 · 20k ステップ · 2 時間 15 分

成功率と Wilson 95 % 信頼区間

ローカル Isaac Sim · inside 条件 · 各実行 100 エピソード。

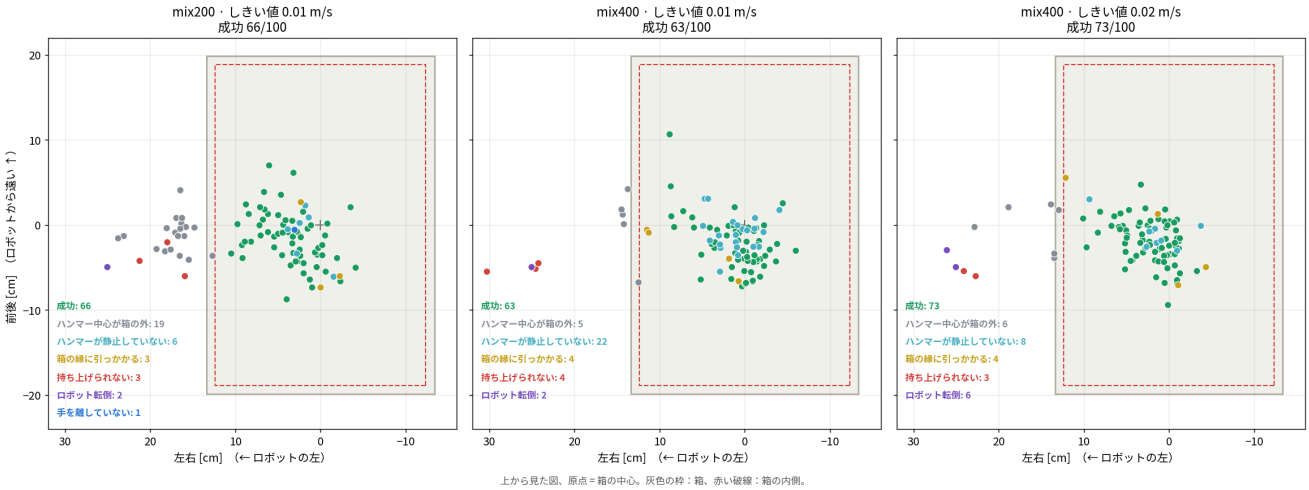


工程	測定値
テレオペ	実デモ 100 件（10 セッション、約 13,700 ステップ）、のちに 200 件追加
データ拡張	生成 133 回 → 厳格な条件を満たすデモ 100 件
データセット	mix200：200 エピソード、28,022 フレーム · mix400：400 エピソード、54,541 フレーム
Fine-tune	mix200：10k ステップ、61 分、loss 0.020 · mix400：20k ステップ、2 時間 15 分、loss 0.019
ローカル · mix200	66/100。3 cm 以上持ち上げ 95/100。最終距離の中央値 6.1 cm
ローカル · mix400	63/100。3 cm 以上持ち上げ 94/100。最終距離の中央値 4.4 cm
ローカル · mix400、しきい値 0.02	**73/100。3 cm 以上持ち上げ 91/100。成功までの平均時間 2.5 s**

10 結果の分析

ローカル Isaac Sim での 100 エピソード × 3 回の結果を分類し、データ追加としきい値の緩和で何が変わったかを確認します。

結果（100 エピソード）	MIX200	MIX400	MIX400 · 目視カウント	MIX400 · しきい値 0.02
成功	**66**	**63**	**87**	**73**
ハンマー中心が箱の外	19	5	5	6
時間切れ時にハンマーが静止していない	6	22	0	8
ハンマーが箱の縁に引っかかる	3	4	2	4
持ち上げられない（< 1 cm）	3	4	4	3
ロボット転倒	2	2	2	6
手を離していない	1	0	0	0



エピソード終了時のハンマー中心位置（上から見た図、原点は箱の中心）· 左：mix200 · 中央：mix400 · 右：静止しきい値 0.02 m/s の mix400。mix200 では、ハンマーが左手側にずれて箱の壁のすぐ外に置かれることが多くありました（灰色の集まり）。mix400 では点が箱の中心付近に集まっています。しきい値 0.01 の mix400 では、失敗の大半は箱の中央にあるのに時間切れ時点で静止していないハンマーです（水色の点 22 個）。しきい値 0.02 の実行では、このグループは 8 個に減りました。赤（持ち上げられない）と紫（ロボット転倒）の点は、ハンマーが箱まで運ばれなかったケースです。

所見 1

把持と持ち上げは安定

3 回の実行すべてで 91–95/100 回、ハンマーを 3 cm 以上持ち上げました。持ち上げられなかったのは 3–4 回だけです。損失は主に置く段階で発生しています。

所見 2

データ追加で置く位置の精度が向上

mix400 では、ハンマーが箱の外に落ちた回数が 19 から 5 に、最終距離の中央値が 6.1 cm から 4.4 cm に減りました。

所見 3

静止しきい値が残るボトルネック

中央の図から、mix400（しきい値 0.01 m/s）の「静止していない」失敗 22 件はすべて箱の内側にあり、手も離していることがわかります。これらの点は成功例よりも箱の中心に近いほどです（中央値 2.9 cm 対 4.3 cm）。ハンマーは 6 s の時間切れまでゆっくり動き続けるだけで（最終速度 0.005–0.023 m/s、回転 0.04–0.32 rad/s）、一度も静止条件を満たしませんでした。置き方の失敗というより、採点方法による失敗です。しきい値 0.02 m/s の実行（右図）では、このグループは 8 件に減り、成功率は 73 % に上がりました。

所見 4

静止しているハンマーが「静止していない」と判定される理由

静止条件（Arena のまま変更なし）は、各シミュレーションステップにおけるハンマーの**瞬間速度**だけを見ています。0.01 m/s 未満かつ 0.1 rad/s 未満を同時に満たす必要があります。22 個のハンマーはすべて箱の内側にあり手も離れていて、確認した動画ではハンマーが滑っている様子は見られません。もし測定された速度（0.5–2.3 cm/s）で箱の中心に着いてから時間切れまでの約 3.5 s 間動き続けていたら、数センチは移動するはずです。実際には、最終時の箱中心までの距離は最接近時との差が中央値で 1.2 cm にすぎません。つまりハンマーは主に**その場で微振動**しています。ハンマーが箱のメッシュ状の底に置かれると、物理シミュレーションは速度を正確に 0 に戻しません（22 件中 12 件は底に平らに置かれており、成功例では 63 件中 8 件）。さらに箱自体も棚の上に置かれた動的な物体です。この小さな振動が 0.01 m/s を超えるため、瞬間値のしきい値はシミュレーションのノイズに対して厳しすぎます。一定時間（例：0.5 s）内にハンマーが移動したかどうかで判定する方が妥当です。

所見 5

転倒の増加はモデルやしきい値のせいではない

73 % の実行ではロボットが 6 回転倒し、他の 2 回の実行ではそれぞれ 2 回でした。6 回とも、手がハンマーに届く前のエピソード開始直後、0.02–0.78 s 以内（最大 300 ステップ中 39 ステップ）に発生しています。ハンマーは棚の上にあり、箱の中心から 25.6 cm 離れたままでした。この時点ではモデルはほとんど制御を始めておらず、静止しきい値はエピソード終了時の採点にしか使われないため、転倒の原因にはなりません。これは nb12 で記録済みのリセットの問題と一致します。前のエピソードが腕と胴体をデフォルト姿勢から大きく離れた状態で終わると、リセットで以前の状態が完全には消えず（腰が曲がったまま、関節に速度が残る）、ロボットは最初のティックで足を踏み出すか転倒します。リセットを 2 回行っても、デモ 20 件の再生で 2/20 回は開始時に失敗しました。6 対 2 の差は統計的に有意ではなく（Fisher $p = 0.28$）、3 回の実行を合わせると 10/300 エピソード（約 3 %）です。これらの転倒を除くと、73 % の実行は 73/94 ≈ 78 % になります。このノイズを評価から除くには、リセット工程の修正（例：エピソード開始前に姿勢と関節速度を確認する）が必要です。

> **注意：**しきい値 0.02 の実行は新たな 100 エピソードの実行であり、以前の実行を採点し直したものではないため、ランダムなばらつきの影響を受けます。

11 シナリオの結論

結論

本プロジェクトは **G1 にまったく新しいタスクを自力で教える**ことに成功しました。未調整の 0 % から、わずか 200 エピソード（テレオペ 100 + 生成 100）と H100 1 枚での約 1 時間のファインチューニングで **66 %** に到達し、全工程は約 6 日間でした。把持と持ち上げは安定しており、残る損失はハンマーを箱に置く段階と採点方法にあります。

- 自社開発のツールで全工程を実施：環境構築、テレオペによるデモ収集、**データ生成**、ファインチューニング、評価。
- デモの生成により、テレオペ時間を増やさずにデータを増やせる（生成 133 回 → 厳格な条件を満たすデモ 100 件）。
- データを 2 倍（mix400）にするとハンマーをより正確に置けるようになった（箱の外 19 → 5 回）が、全体の成功率は上がらなかった（63 %）。静止しきい値 0.02 では **73 %** に達した。
- 未対応：「静止」の判定を瞬間速度ではなく一定時間内のハンマーの移動量で行うよう変更すること。エピソード開始時にロボットが転倒する原因となるリセット工程の修正（エピソードの約 3 %）。

IV.01 成果物

顧客が受け取るもの、その場所、使い方。

本フェーズの成果物はすべて**6 部構成の 1 つのフォルダ**に収め、推奨する読む順に番号を付けています：レポート → ソースコード → データ → モデル → 結果 → シミュレーションアセット。元データとチェックポイントは**別途提供**します。

フォルダ	メインパッケージ	別途提供
shimizu-robot/	6 部	元データ + チェックポイント

1 成果物パッケージの構成

各タイルは 1 つのサブフォルダです。タイルをクリックすると中のファイル一覧を表示します（カード 3）。

01 report/

レポート

本インタラクティブレポート（ブラウザで開く、インストール不要）と編集用の markdown 版。

内容を見る →

02 codes/

ソースコード

独立した制御パイプライン、研究用 notebook 12 本、real2sim ツール、Arena パッチ、テレオペ用スクリプト一式。

内容を見る →

03 datasets/

学習データ

ハンマー → 箱のデータセット（200 および 400 エピソード）と real2sim 果物 4 種のデータセット（学習 790 + テスト 89）、LeRobot 形式。

内容を見る →

04 models/

学習済みモデル

ファインチューニング済みチェックポイント 3 つ（ハンマー → 箱 mix200・mix400、real2sim 果物 4 種）。GR00T サーバーですぐに実行できます。

内容を見る →

05 evals/

評価結果

3 シナリオのエピソードごとのログ、集計表、グラフ、成功 / 失敗動画。

内容を見る →

06 assets/

シミュレーションアセット

シーン構築に使う 3D ファイル：NVIDIA ファイルのオフラインコピー（Galileo 背景、ハンマー、ビン、G1 ロボット）とリンゴ・皿。

内容を見る →

2 役割別の想定利用者

全員がパッケージ全体を必要とするわけではありません。下表は役割ごとに先に読むべき部分の目安です。

部分	プロジェクトマネージャー	ソフトウェアエンジニア	AI / ロボットエンジニア	データ収集担当（テレオペ）
01 レポート	✓	·	·	·
02 ソースコード		✓	✓	·
03 データ			✓	·
04 モデル		·	✓	
05 結果	✓		✓	
06 アセット		·	✓	

■ 主に使用　□ 必要に応じて参照

3 詳細な内容一覧

フォルダをクリックすると中の主なファイルを表示します。

01 **report/ – レポート** –

開く: index.html　インストール不要

- `index.html` – **インタラクティブレポート**：目次、動画、比較表、PDF 出力
- `assets/`, `images/` – UI、デモ動画、画像
- `src/`, `build.py`, `README.md` – 各節の内容とレポートの編集方法
- `markdown/` – 引用・編集用のテキスト版レポート（10 ファイル）

02 codes/ – ソースコード

Python　unit test 84 件　インストール済み環境は含まない

- `standalone_g1_gr00t/` – GR00T で G1 を制御するパイプライン、Arena に依存しない
  - `g1gr00t/` – パッケージ：サーバー接続、関節マッピング、観測、アクション、安全レイヤー、AGILE、Isaac Lab バックエンド、runner
  - `notebooks/` – 研究用 notebook 12 本（nb01 → nb12）
  - `configs/` – リンゴ・ハンマーのシナリオとバランス制御器の設定
  - `scripts/`, `tools/` – 閉ループ実行、open-loop 評価、アセットのダウンロード、notebook の自動実行
  - `tests/` – unit test とシミュレーション内テスト
  - `README.md`, `Makefile` – インストールと実行コマンド
- `g1_real2sim/` – 実機 G1 データからのシーン再構成：URDF からの FK、妨害物体、Isaac Lab シーン、LeRobot の読み書き、GR00T 設定
- `arena_bua_thung/` – ハンマーシナリオ用の Isaac Lab Arena 拡張
  - `patches/` – 環境パッチ 4 件：ハンマー–箱の配置、支持ブロック、厳格な成功条件
  - `teleop/` – テレオペの起動、記録、再生、デモの統合、診断用スクリプト
  - `scripts/` – 拡張デモの生成、データセットのチェック、シーンの検証

03 datasets/ – 学習データ

LeRobot v2.1 形式　H.264 動画 640×480

フォルダ	内容
`hammer_to_bin/static_hammer_mix200/`	200 エピソード（実テレオペ 100 + 生成 100）、28,022 フレーム @ 50 Hz
`hammer_to_bin/static_hammer_mix400/`	400 エピソード（mix200 + 新規テレオペデモ 200）、54,541 フレーム @ 50 Hz
`real2sim_multi_fruit/g1-fruits-sim-train/`	実機 G1 データから作成したシミュレーション学習セット：790 エピソード、86,822 フレーム @ 20 Hz（リンゴ 246、洋ナシ 211、スターフルーツ 174、ブドウ 159）
`real2sim_multi_fruit/g1-fruits-sim-test/`	テストセット：89 エピソード、9,906 フレーム、学習に含まれないシーン。`test_scenes.json` は評価シーンの一覧

各データセットは `meta/`（説明、指示文、統計）、`data/`（エピソードごとの関節角とアクション）、`videos/`（ロボット頭部カメラ）で構成されます。

04 models/ – 学習済みモデル

各フォルダは GR00T サーバーでそのまま使えるチェックポイントです：重み（`safetensors` ファイル 3 つ）、設定、processor、正規化統計、学習ログと学習スクリプト。optimizer の状態は含まれません。

CHECKPOINT	学習データ
ハンマー → 箱、200 エピソード `hammer_to_bin_mix200/` · ステップ 10,000	mix200 · 10,000 ステップ · 1× H100 · 61 分
ハンマー → 箱、400 エピソード `hammer_to_bin_mix400/` · ステップ 20,000	mix400 · 20,000 ステップ · 1× H100 · 2 時間 15 分
Real2sim 果物 4 種 `real2sim_multi_fruit/` · ステップ 20,000	シミュレーション 790 エピソード · 20,000 ステップ · バッチ 32
リンゴ → 皿（NVIDIA 提供） パッケージには含まない	再学習なし – Hugging Face から `models/isaaclab_arena/static_apple_tutorial/gn1x_tuned_static_apple/` （設定の既定パス）にダウンロード

05 evals/ – 評価結果

- `apple_plate/` – リンゴ → 皿、NVIDIA チェックポイント、各実行 50 エピソードと頭部カメラ動画 50 本
  - `self_built_scene_50ep/` – 自作シーン：0/50
  - `galileo_scene_50ep/` – 再構築した Galileo シーン：28/50
- `multi_fruit/` – 果物 4 種、各実行 50 エピソード。動画は最初の 10 エピソードのみ
  - `finetune_real2sim_50ep/` – real2sim でファインチューニングしたモデル：33/50
  - `baseline_apple_ckpt_50ep/` – 比較対照、リンゴのチェックポイントを再利用：0/50
- `hammer_to_bin/` – ハンマー → 箱、各実行 100 エピソードと動画 100 本
  - `arena_mix200_100ep/` – Arena での mix200（外部カメラ + 頭部カメラ）：56/100
  - `local_mix200_100ep/` – mix200、ローカル Isaac Sim：66/100
  - `local_mix400_100ep/` – mix400、ローカル Isaac Sim：63/100
  - `local_mix400_at_rest_0.02_100ep/` – mix400、静止しきい値を 0.02 m/s に緩和：73/100

最上位の `README.md` に各実行の結果と出所をまとめています。各実行の共通ファイル：`episodes.jsonl`（試行ごとに 1 行）、`summary.json` / `summary.md`（成功率、信頼区間、失敗分類）、動画 `…_success.mp4` / `…_fail.mp4`。

06 assets/ – シミュレーションアセット

配置場所：~/workspace/shimizu-robot/assets　ソースコードはこのパスを参照します

- `nucleus_cache/` – Isaac Lab がシーン構築時にダウンロードする NVIDIA ファイルのオフラインコピー：Galileo 背景、ハンマー `hammer_2`、赤いビン、G1 ロボット、下半身制御モデル（WBC）。これがあればネットワークなしで実行でき、なければ Isaac Lab が再ダウンロードします。
- `objects/objaverse/apple_01.usd`、`objects/hot3d/clay_plates.usd` – スタンドアロンパイプラインのリンゴ → 皿シナリオで使うリンゴと皿。

4 種類の果物の real2sim シナリオはこのフォルダを使いません（ロボットの URDF は `codes/g1_real2sim/assets/` にあります）。

4 別途提供する元データ

ハンマータスクの元 HDF5 データです。**学習データを再変換・追加生成する**場合にのみ必要で、モデルの実行、レポートの閲覧、結果の確認には必要ありません。

コンポーネント	必要な場面
ハンマータスクの元テレオペデータ（HDF5）	再変換、デモの追加生成
mix200 学習セット（HDF5 形式）	別の設定で LeRobot に変換
mix400 学習セット（HDF5 形式）	同上

含まないもの: インストール済みの Python 環境（ソースコード内のコマンドで再作成）、NVIDIA の元データとチェックポイント（Hugging Face からダウンロード）。

5 ライセンスと利用条件

コンポーネント	条件
ベースモデル GR00T N1.7-3B とそこからファインチューニングしたチェックポイント	NVIDIA Open Model Agreement に準拠
NVIDIA のリンゴ → 皿チェックポイント（Arena）	Model card に "ready for non-commercial use only" と記載
NVIDIA データセット（リンゴ → 皿、実機 G1 データ）	CC-BY-4.0：利用可、出典表記が必要

コンポーネント	条件
ハンマータスクのテレオペデータ、生成データセット、real2sim データセット	本プロジェクトで作成
VLA モデルのデモ動画（II.01 節）	第三者のもの
`assets/` 内の 3D アセット（NVIDIA Isaac / Arena のファイル、Objaverse のリンゴ、HOT3D の皿）	NVIDIA および第三者のもの。再配布前に条件を確認

IV.02 環境構築

シナリオを再実行するために 2 台のマシンを準備する。

IV.02〜IV.04 節では、報告書の数値を検証するためにシナリオを自分で再実行する手順を説明します。必要なマシンは 2 台です：シミュレーションとノートブックを実行する**ワークステーション**（クライアント）と、GR00T を実行する顧客の **GPU サーバー**（サーバー）。本節では両側の環境を構築します。サーバーの起動と評価の実行は IV.03 節と IV.04 節で説明します。

CLIENT

Isaac Sim 6.0.1 · Isaac Lab 3.0.0-beta2 · standalone_g1_gr00t

SERVER

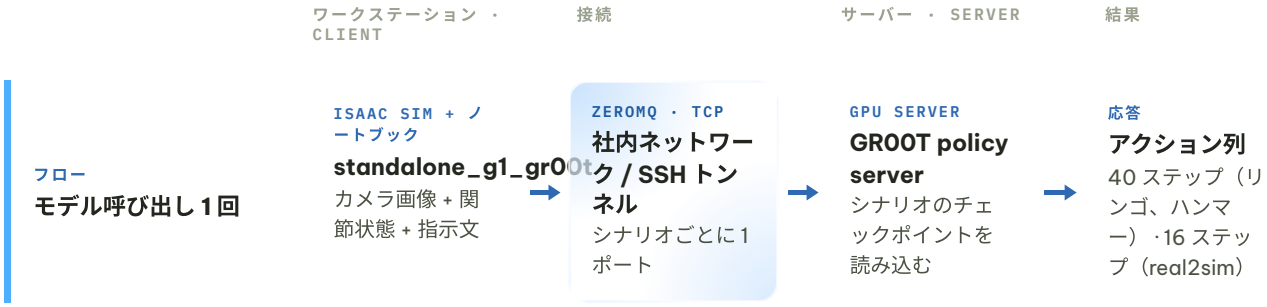
顧客の GPU サーバー · Isaac-GR00T 4b1dca9

接続

社内ネットワークまたは SSH トンネル経由の ZeroMQ

1 接続構成

ワークステーション上のノートブックがカメラ画像、関節状態、指示文を GR00T サーバーに送り、サーバーはアクション列を返します。2 台のマシンは社内ネットワークで直接接続するか、サーバーが `127.0.0.1` のみで待ち受ける場合は SSH トンネル経由で接続します。



2 ハードウェアとソフトウェアの要件

ワークステーション · CLIENT

シミュレーションとノートブック

- Ubuntu 24.04、RTX 対応の NVIDIA GPU（本プロジェクトでは RTX 5090）、ドライバー 580
- Isaac Sim **6.0.1** を `~/isaac-sim-6` にローカルインストール（Python 3.12）
- Isaac Lab **release/3.0.0-beta2** を `~/IsaacLab` に配置し、Isaac Sim と同じ Python にインストール
- Isaac Lab Arena **release/0.2.1** を `~/IsaacLab-Arena` に配置（関節 YAML ファイルと modality config の取得元）
- `uv`、`make`、`git`。初回インストール時はインターネット接続が必要

サーバー · SERVER

顧客の GPU サーバー

- Linux、CUDA 対応の NVIDIA GPU（本プロジェクトでは 1× H100 80 GB）
- Isaac-GR00T コミット **4b1dca9**、`uv` 環境（Python 3.10）、`ffmpeg`
- ベースモデル `nvidia/GR00T-N1.7-3B` への Hugging Face アクセス権（NVIDIA のライセンスに同意済み）
- `models/` 内のチェックポイントと各シナリオの modality config（カード 5）
- ワークステーションに向けてポートを開放できること、または SSH を許可できること

バージョンの組み合わせを守ること：報告書の結果はすべて Isaac Sim 6.0.1 + Isaac Lab 3.0.0-beta2 と Isaac-GR00T 4b1dca9 で測定しています。Isaac Sim 6.1 はドライバー ≥ 595 が必要で、このソースコードではまだテストしていません。

3 ワークステーションへの成果物パッケージの配置

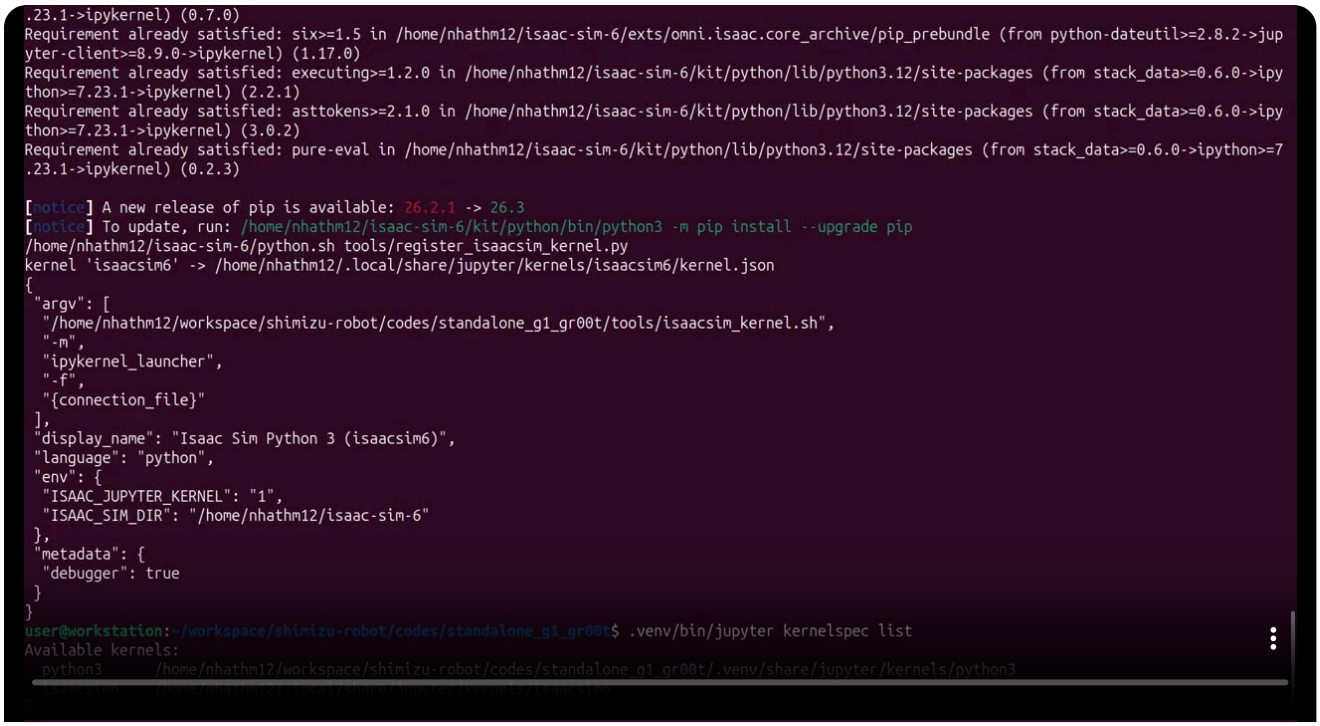
ソースコードは `assets/`、`datasets/`、`models/` をパス `~/workspace/shimizu-robot/` から読み込むため、パッケージはこの場所に展開してください。Isaac Sim、Isaac Lab、Arena はカード 2 のとおりホームディレクトリに配置します。

```
~/workspace/shimizu-robot/      # 成果物パッケージ（IV.01）
├── codes/standalone_g1_gr00t/  # クライアント：g1gr00t パッケージ、ノートブック、Makefile
├── assets/                     # nucleus_cache + リンゴ、皿
├── datasets/                   # LeRobot データセット
└── models/                     # チェックポイント（サーバーへコピー、カード 6）
~/isaac-sim-6/   ~/IsaacLab/   ~/IsaacLab-Arena/
```

別の場所にインストールする場合：Makefile では `ISAAC_PY`（Isaac Sim の Python）、`ARENA_DIR`、`ISAAC_GR00T_DIR` を上書きできます。例：`make sim-test ISAAC_PY=/opt/isaac-sim/python.sh`。データセットとモデルのパスは `configs/*.yaml` にあります。

4 ワークステーションへのクライアントのインストール

`standalone_g1_gr00t` パッケージは 2 つの環境を使います：**.venv**（Python 3.10、テストとシミュレーションなしのノートブック用）と **Isaac Sim の Python**（シミュレーションありのノートブック用）。



サンプル動画：Isaac Sim をインストール済みのワークステーションでクライアントの確認コマンドを実行します（`make sim-test-scene` の部分は早送り）。表示されるホスト名は `user@workstation` と `user@gpu-server` です。

```
cd ~/workspace/shimizu-robot/codes/standalone_g1_gr00t

# .venv 環境
make venv             # uv venv .venv --python 3.10、パッケージ + dev + notebook をインストール
make sync             # ~/IsaacLab-Arena から関節/WBC の YAML ファイル 4 つをコピーし、取得元のハッシュを記録
make assets           # AGILE の脚 policy（ONNX）を assets/wbc/ にダウンロード
make test             # シミュレーション不要の unit test

# Isaac Sim 環境
export PYTHONNOUSERSITE=1   # ~/.local のパッケージが Isaac Sim の Python に混入しないようにする
make sim-install      # パッケージを ~/isaac-sim-6/python.sh にインストール
make sim-test         # Isaac Sim の Python で unit test
make sim-test-scene   # シーンありのテスト：ロボットが安定して立つ、テーブル / リンゴ / 皿
make sim-kernel       # シミュレーション用ノートブックの Jupyter カーネル "isaacsim6" を登録
```

ワークステーション上のデータ	パス	用途
AGILE の脚 policy	`codes/standalone_g1_gr00t/assets/wbc/`（`make assets` でダウンロード）	すべてのシミュレーション用ノートブック
Galileo シーン、ハンマー、箱、G1 ロボットのアセット	`assets/nucleus_cache/`（パッケージに同梱）	リンゴ（Galileo シーン）とハンマーのシナリオ。ネットワーク不要で実行可能
リンゴ、皿	`assets/objects/`	リンゴのシナリオ（自作シーン）
ハンマー → 箱 mix200 データセット	`datasets/hammer_to_bin/static_hammer_mix200/`	nb12：デモ再生、画像の色合わせ
NVIDIA のリンゴ → 皿データセット	`datasets/isaaclab_arena/static_apple_tutorial/`（Hugging Face からダウンロード）	デモ再生、リンゴのシナリオの open-loop

確認完了の目安： `make test`、`make sim-test`、`make sim-test-scene` のいずれも失敗したテストがなく、Jupyter でカーネル **Isaac Sim Python 3 (isaacsim6)** が見えること。

5 GPU サーバーへのサーバーのインストール

学習時に使ったものと同じコミットの Isaac-GR00T をインストールします。下表は各シナリオに必要なチェックポイントと modality config を示します。それらをサーバーに配置する方法はカード 6 にあります。



サンプル動画：Isaac-GR00T を指定のコミットで clone し、`uv` 環境を作成してサーバーの GPU を確認します（ダウンロードの部分は早送り）。表示されるホスト名は `user@workstation` と `user@gpu-server` です。

```
# 事前に必要：git、uv、ffmpeg（sudo apt-get install -y ffmpeg）
git clone https://github.com/NVIDIA/Isaac-GR00T.git ~/Isaac-GR00T
cd ~/Isaac-GR00T
git checkout 4b1dca9d88d2a0b9ea5a65aa61c82ff89f5c4f0e
git submodule update --init --recursive
uv sync --python 3.10      # 環境を作成し、GR00T と GPU ライブラリをインストール（数分）
uv run python -c "import gr00t, torch; print('GR00T ok | CUDA', torch.cuda.is_available())"
# ベースモデルが Hugging Face のキャッシュにない場合は、
# nvidia/GR00T-N1.7-3B のライセンスに同意済みのアカウントでログイン：
hf auth login
```

シナリオ ・ クライアントのポート	`MODELS/` 内のチェックポイント	MODALITY CONFIG
リンゴ → 皿 5556 · `g1_static_apple.yaml`	Hugging Face からダウンロードする NVIDIA のチェックポイント	Arena 版

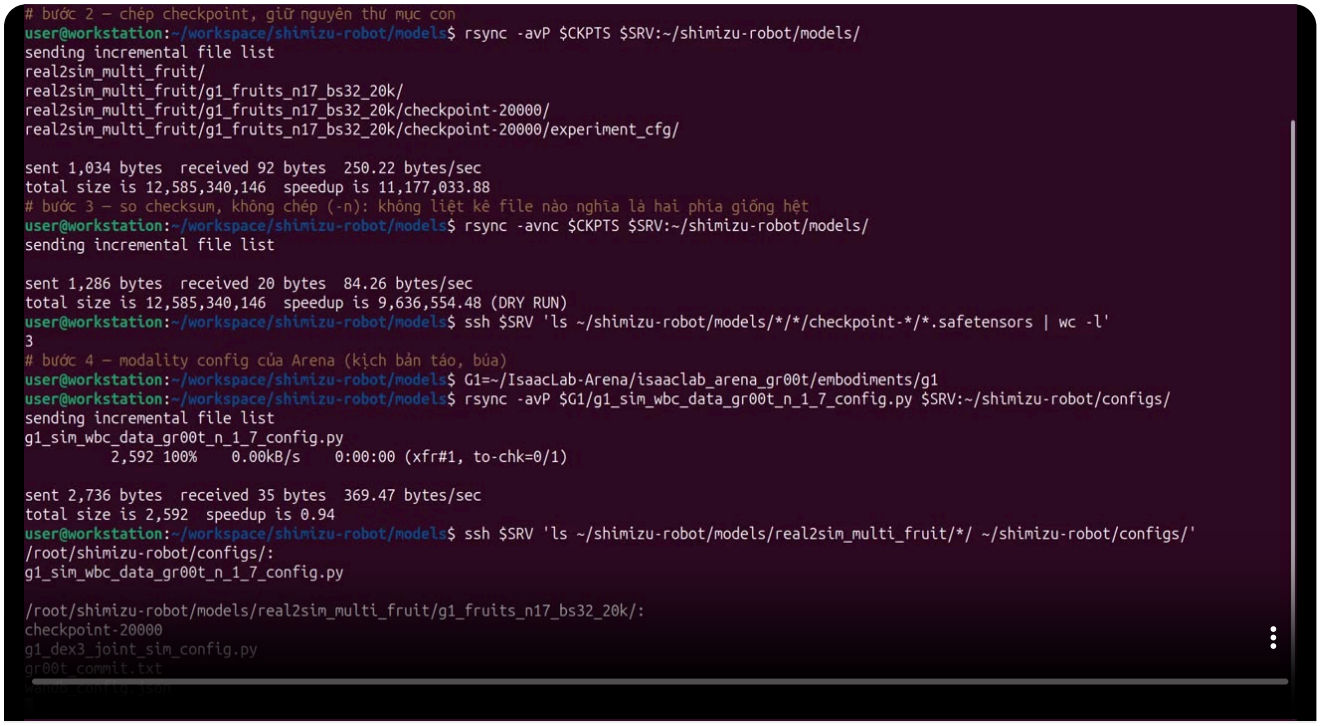
ハンマー → 箱
5557 · `NB12_PORT`
	`hammer_to_bin_mix200/`
	`hammer_mix200_n17/checkpoint-10000`
	`hammer_to_bin_mix400/`
	`hammer_mix400_n17/checkpoint-20000`
		Arena 版

Real2sim 果物 4 種 5556 · `NB10_PORT`	`real2sim_multi_fruit/` `g1_fruits_n17_bs32_20k/checkpoint-20000`	real2sim 版

Modality config は学習時と一致させること：誤ったファイルを使ってもサーバーは動作しますが、モデルは誤った関節順序を受け取ります。Arena 版は Arena **release/0.2.1** 内の `isaaclab_arena_gr00t/embodiments/g1/g1_sim_wbc_data_gr00t_n_1_7_config.py` です（このファイルだけをサーバーにコピーするか、Arena を clone します）。real2sim 版は `g1_dex3_joint_sim_config.py` で、`real2sim_multi_fruit/g1_fruits_n17_bs32_20k/` フォルダ（`checkpoint-20000` の隣）と `codes/g1_real2sim/groot/` にあります。実行前に両側で `md5sum` を比較してください。

6 納品チェックポイントのサーバーへの配置

ファインチューニング済みチェックポイント 3 つは成果物パッケージの `models/` フォルダにあり、各フォルダは GR00T サーバーですぐに使えます：重み（`safetensors` ファイル 3 つ）、設定、processor、正規化統計。検証したいシナリオのチェックポイントだけをコピーすれば十分です。



real2sim チェックポイントでのサンプル動画。動画のサーバーには同一のコピーがすでにあるため、rsync は照合のみ行い、12 GB を再コピーしていません。初回のコピーはネットワーク速度に応じてより長くかかります。表示されるホスト名は `user@workstation` と `user@gpu-server` です。

ステップ 1
サーバーにフォルダを作成
例：`~/shimizu-robot/models`

→

ステップ 2
チェックポイントをコピー
ワークステーションから `rsync`、中断しても再開可能

→

ステップ 3
完全性の確認
両側のチェックサムを比較し、重みファイル 3 つが揃っていることを確認

→

ステップ 4
modality config をコピー
リンゴとハンマーのシナリオ用の Arena 版

```
# ワークステーションで実行
SRV=user@gpu-server
CKPTS="hammer_to_bin_mix200 hammer_to_bin_mix400 real2sim_multi_fruit"
cd ~/workspace/shimizu-robot/models
```

```
# ステップ 1 — サーバーにフォルダを作成
ssh $SRV 'mkdir -p ~/shimizu-robot/models ~/shimizu-robot/configs'

# ステップ 2 — 必要なチェックポイントをサブフォルダ構成のままコピー
rsync -avP $CKPTS $SRV:~/shimizu-robot/models/

# ステップ 3 — チェックサムを比較、コピーはしない（-n）：
# ファイルが 1 つも表示されなければ両側は同一
rsync -avnc $CKPTS $SRV:~/shimizu-robot/models/
# 重みファイルの数：期待値は 9（チェックポイント 3 つ × ファイル 3 つ）
ssh $SRV 'ls ~/shimizu-robot/models/*/*/checkpoint-*/*.safetensors | wc -l'

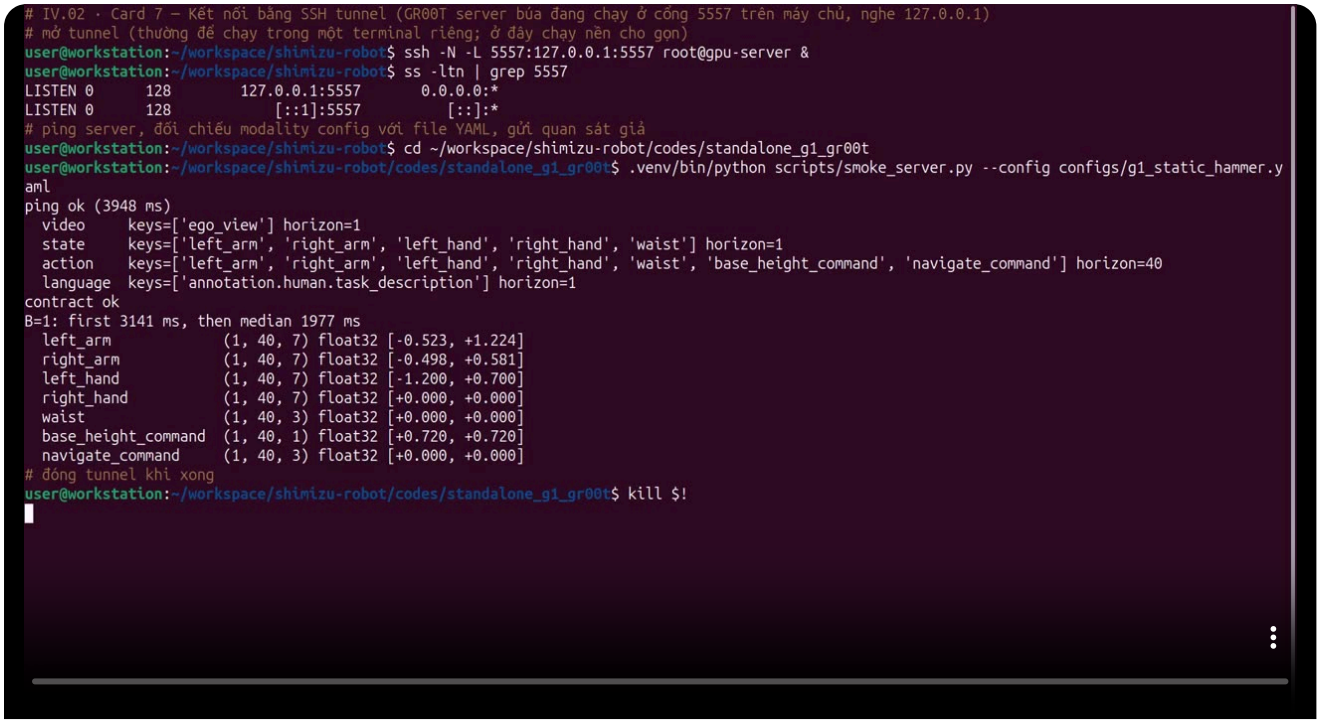
# ステップ 4 — Arena の modality config（リンゴ、ハンマーのシナリオ）
G1=~/IsaacLab-Arena/isaaclab_arena_gr00t/embodiments/g1
rsync -avP $G1/g1_sim_wbc_data_gr00t_n_1_7_config.py \
      $SRV:~/shimizu-robot/configs/
```

`MODELS/` 内のフォルダ	サーバー上の `--MODEL-PATH` のパス
`hammer_to_bin_mix200/`	`~/shimizu-robot/models/hammer_to_bin_mix200/hammer_mix200_n17/checkpoint-10000`
`hammer_to_bin_mix400/`	`~/shimizu-robot/models/hammer_to_bin_mix400/hammer_mix400_n17/checkpoint-20000`
`real2sim_multi_fruit/`	`~/shimizu-robot/models/real2sim_multi_fruit/g1_fruits_n17_bs32_20k/checkpoint-20000`

> **ネットワーク経由で転送できない場合：** `models/` フォルダを外付けディスクでコピーし、同じ構成でサーバーに配置します。その場合もステップ 3 は両側の `safetensors` ファイルに対して `sha256sum` で実施してください。NVIDIA のリンゴ → 皿チェックポイントはパッケージに含まれないため、Hugging Face からサーバーに直接ダウンロードします。

7 接続の設定

次の 2 つの方法のいずれかを選びます。サーバー側のポートはクライアントが想定するポート（カード 5）と一致させる必要があります。クライアントのポートは環境変数 `NB10_PORT`、`NB12_PORT` または YAML ファイルの `server.port` フィールドで変更できます。



サンプル動画：ハンマーのサーバー（ポート 5557）への SSH トンネルを開き、ポートを確認して `smoke_server.py` でサーバーに ping します。表示されるホスト名は `user@workstation` と `user@gpu-server` です。

方法 1

社内ネットワーク

- サーバーは `--host 0.0.0.0` で待ち受け
- ファイアウォールでシナリオのポートをワークステーションに対してのみ開放
- クライアントのホストをサーバーの IP に設定。例：`NB12_HOST=10.0.0.5`

方法 2

SSH トンネル

- サーバーは `--host 127.0.0.1` で待ち受け、ポートを外部に開放しない
- ワークステーションでトンネルを開き、評価中は開いたままにする
- クライアントのホストは既定の `127.0.0.1` のまま

```
# 方法 2、ワークステーションで実行（例：ハンマーのシナリオ、ポート 5557）
ssh -N -L 5557:127.0.0.1:5557 user@gpu-server
```

IV.03 GR00T サーバーの起動

シナリオごとに1つのチェックポイント、1つのポート。

GR00T サーバーは顧客の GPU サーバー上で動作し、検証するシナリオのチェックポイントを読み込んで1つのポートで待ち受けます。ワークステーションはそのポートに接続し（社内ネットワークまたは SSH トンネル経由、IV.02 節のカード 7）、その後 IV.04 節のノートブックを実行します。

コマンド
gr00t/eval/run_gr00t_server.py

ポート
5556（リンゴ、real2sim）・5557（ハンマー）

確認
ポートの待ち受け・ワークステーションから ping

1 シナリオ別のサーバー設定

以下のパスは IV.02 節のカード 6 の配置（サーバー上のルート `~/shimizu-robot/`）に従います。ポートはクライアントが既定で想定する値です。

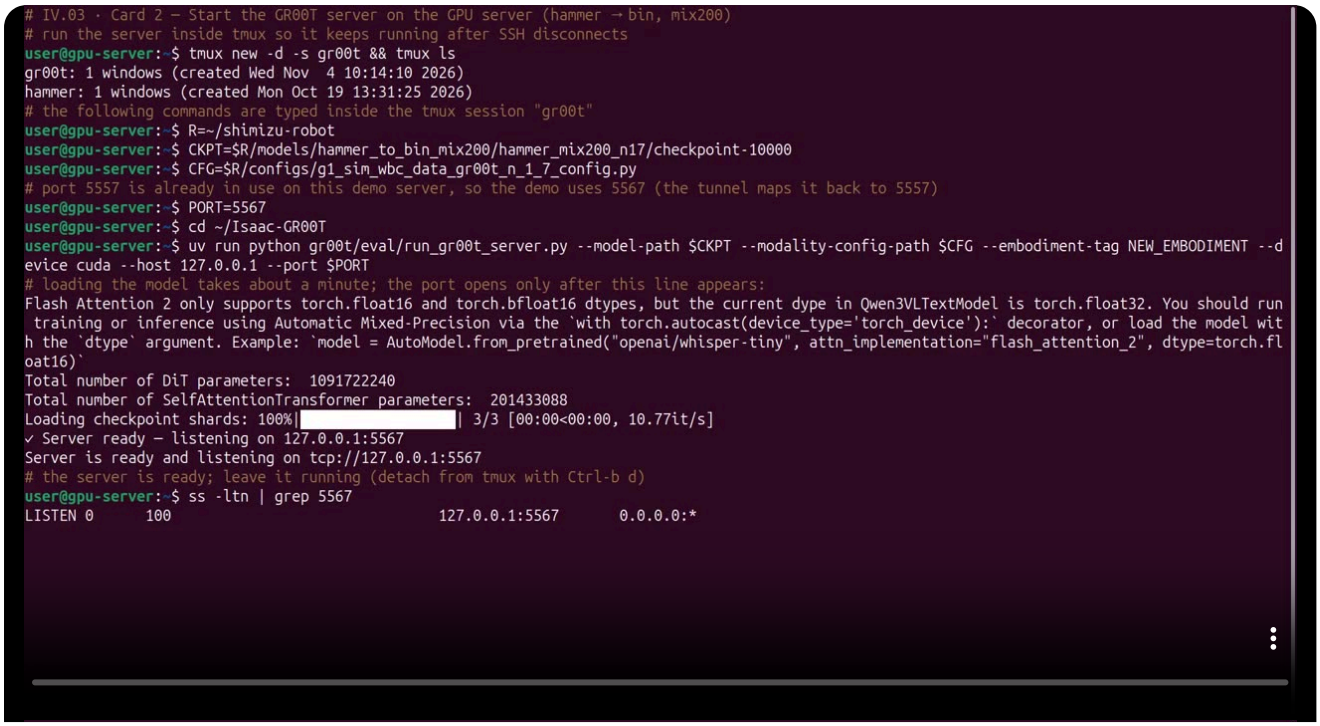
シナリオ ・ ポート	チェックポイント（`--MODEL-PATH`）	MODALITY CONFIG
リンゴ → 皿 5556	`models/gn1x_tuned_static_apple` （Hugging Face からダウンロード）	`configs/g1_sim_wbc_data_gr00t_n_1_7_config.py`
ハンマー → 箱、mix200 5557	`models/hammer_to_bin_mix200/hammer_mix200_n17/checkpoint-10000`	同上
ハンマー → 箱、mix400 5557	`models/hammer_to_bin_mix400/hammer_mix400_n17/checkpoint-20000`	同上
Real2sim 果物 4 種 5556	`models/real2sim_multi_fruit/g1_fruits_n17_bs32_20k/checkpoint-20000`	`models/real2sim_multi_fruit/g1_fruits_n17_bs32_20k/g1_dex3_joint_sim_config.py`

リンゴと real2sim は同じポート 5556 を使うため、同時に実行できるのはどちらか一方です。ハンマーはポート 5557 を使うため、GPU のメモリが十分であれば並行して実行できます。

```
# NVIDIA のリンゴ → 皿チェックポイントをサーバーにダウンロード（1 回のみ）
hf download nvidia/GN1x-Tuned-Arena-G1-Static-PickNPlace \
   --local-dir ~/shimizu-robot/models/gn1x_tuned_static_apple
```

2 サーバーの起動

SSH を切断してもサーバーが停止しないよう、`tmux` 内で実行します。以下はハンマー mix200 シナリオの例です。他のシナリオでは、カード 1 に従って最初の 3 つの変数を変更するだけです。



サンプル動画：サーバー上の tmux でハンマー mix200 のサーバーを実行し、`✓ Server ready` の行を待ちます（モデル読み込みの部分は早送り）。動画のサーバーではポート 5557 が別の用途で使われていたため、サンプルのサーバーはポート 5567 で待ち受けています。カード 3 のトンネルがワークステーション側のポート 5557 に転送します。

```
tmux new -s gr00t                  # 一時的に抜ける：Ctrl-b d · 再接続：tmux a -t gr00t
R=~/shimizu-robot
CKPT=$R/models/hammer_to_bin_mix200/hammer_mix200_n17/checkpoint-10000
CFG=$R/configs/g1_sim_wbc_data_gr00t_n_1_7_config.py
PORT=5557

cd ~/Isaac-GR00T
uv run python gr00t/eval/run_gr00t_server.py \
  --model-path $CKPT --modality-config-path $CFG \
  --embodiment-tag NEW_EMBODIMENT --device cuda \
  --host 127.0.0.1 --port $PORT    # ワークステーションが社内ネットワークで直接接続する場合は 0.0.0.0
```

モデル読み込み
約 1 分
ポートはまだ開いておらず、この間のクライアント呼び出しはタイムアウトします

→

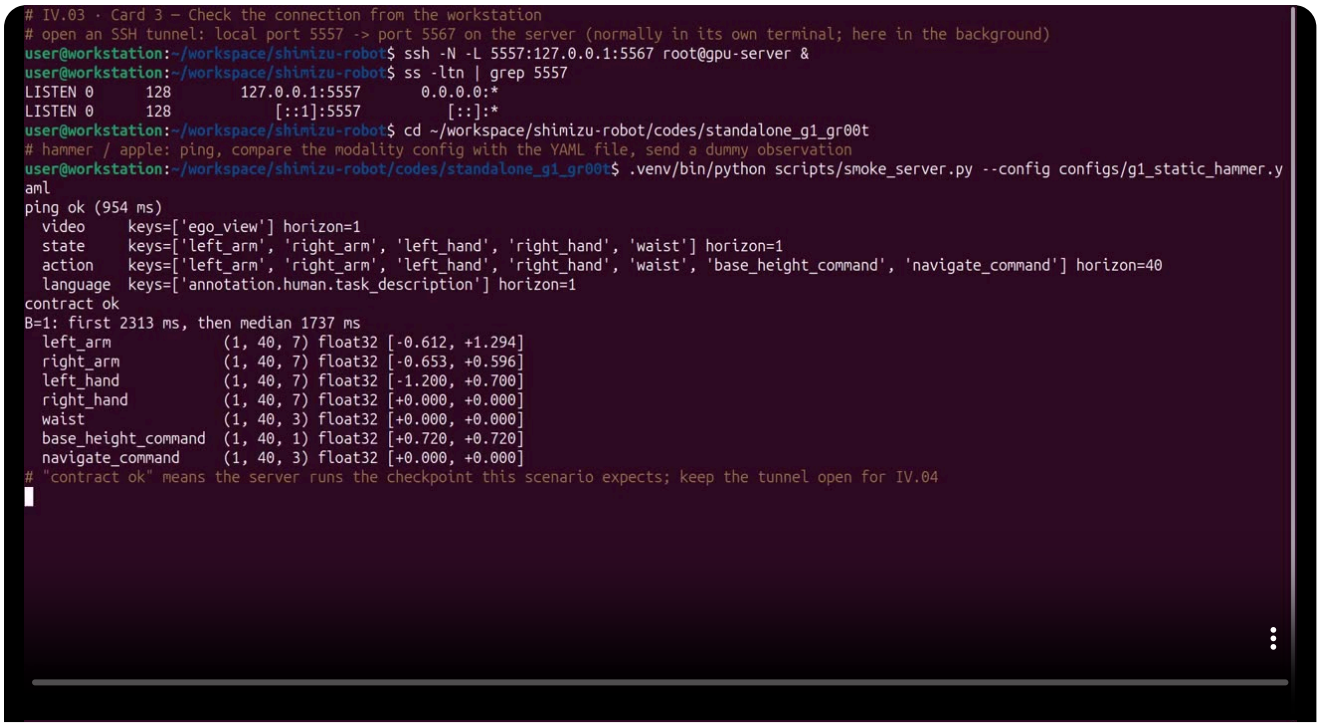
準備完了
準備完了の行を表示
`✓ Server ready — listening on …`

→

シナリオの切り替え
Ctrl-C して再実行
新しいチェックポイント、config、ポートで

3 ワークステーションからの接続確認

SSH トンネルを使う場合は、別のターミナルでトンネルを開き、評価中は開いたままにします。その後、`.venv` 環境の Python でサーバーに ping します。



サンプル動画：SSH トンネルを開き、ワークステーションでポート 5557 を確認して、ハンマーの設定で `smoke_server.py` を実行します（結果 `contract ok`）。

```
# （SSH トンネルを使う場合）別のターミナルで、開いたままにする
ssh -N -L 5557:127.0.0.1:5557 user@gpu-server

# サーバーに ping し、読み込み中の modality config を問い合わせる
cd ~/workspace/shimizu-robot/codes/standalone_g1_gr00t
# リンゴ、ハンマー：ping、modality config を YAML ファイルと照合、ダミーの観測を送信
.venv/bin/python scripts/smoke_server.py --config configs/g1_static_hammer.yaml
#    （リンゴ：configs/g1_static_apple.yaml）
# real2sim：ping し、サーバーが読み込んでいる modality config を表示
.venv/bin/python -c "from g1gr00t.client import Gr00tZmqClient, modality_summary as m
c = Gr00tZmqClient('127.0.0.1', 5556); print(c.ping(), m(c.get_modality_config()))"
```

症状	原因	対処
最初の呼び出しですぐにタイムアウト	トンネルが開いていない、ポートが違う、またはサーバーがモデルを読み込み中	両側で `ss -ltn | grep 555` を実行し、準備完了の行を待つ
途中で接続が切れる	トンネルまたはネットワークの切断	トンネルを開き直してノートブックを再実行。nb10 と nb12 は途中のエピソードから再開
ノートブックが video key や関節数の不一致を報告	サーバーが別のシナリオのチェックポイントを読み込んでいる	カード 1 の正しいチェックポイントと modality config でサーバーを再起動
1 回の呼び出しが > 3 s	GPU が混雑しているか、ネットワークが遅い	結果には影響しない（シミュレーションはモデルを待って停止する）。実行時間が長くなるだけ

IV.04 検証の実行

ノートブックを再実行し、報告書の数値と比較する。

すべてのシナリオは `codes/standalone_g1_gr00t` 内のノートブックで評価し、ローカルの Isaac Sim 上で直接実行するため、**Docker Arena は不要**です。各ノートブックには GUI なし（headless）で実行して結果を `data/` に書き出す `make` コマンドがあります。

フォルダ
codes/standalone_g1_gr00t

実行
make sim-nbXX（headless）・Jupyter（対話）

出力
data/：エピソードごとのログ、集計、動画

1 共通の手順

01 · サーバー
正しいチェックポイントを読み込む
IV.03 節、カード 1–2

→

02 · 接続
必要ならトンネルを開く
評価中は開いたままにする

→

03 · 確認
サーバーに ping
IV.03 節、カード 3

→

04 · 実行
シナリオの make コマンド
下のカード 3

→

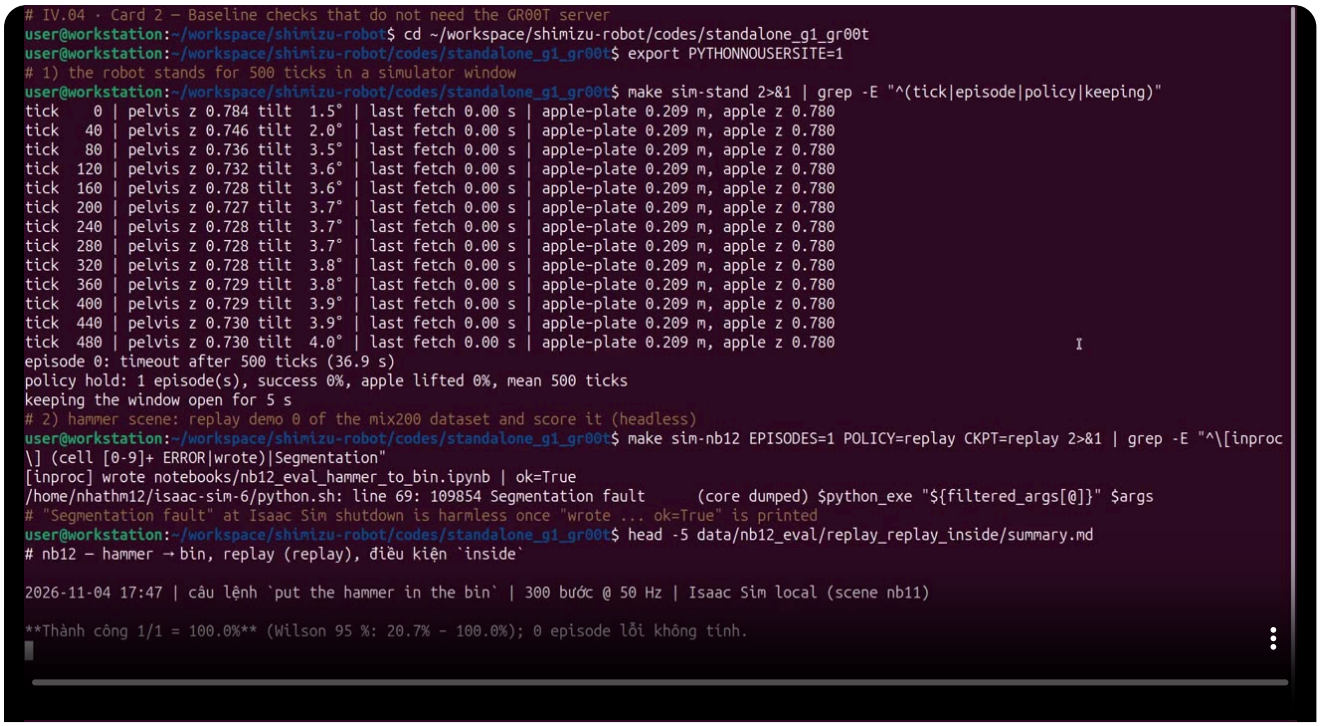
05 · 比較
集計を読む
信頼区間と比較（カード 4）

パッケージ内の既存の結果は上書きされます： `make sim-nbXX` コマンドは新しい出力をその `.ipynb` ファイル自体に書き込みます。さらに nb07 は `data/runs_galileo.jsonl` に追記し、動画 `data/videos/gr00t_galileo/epN.mp4` を上書きします。実行前に `notebooks/` と `data/` をバックアップし、集計が新しい実行だけを含むよう `runs_galileo.jsonl` の名前を変更してください。結果の原本は `evals/` にもあります。

Isaac Sim 終了時のエラー：各実行の最後に `Segmentation fault` と `make: *** Error 1` が表示されることがあります。その前に `[inproc] wrote notebooks/… | ok=True` の行が出ていれば、ノートブックは最後まで実行され、結果もすべて書き込まれています。

2 サーバー不要の基本確認

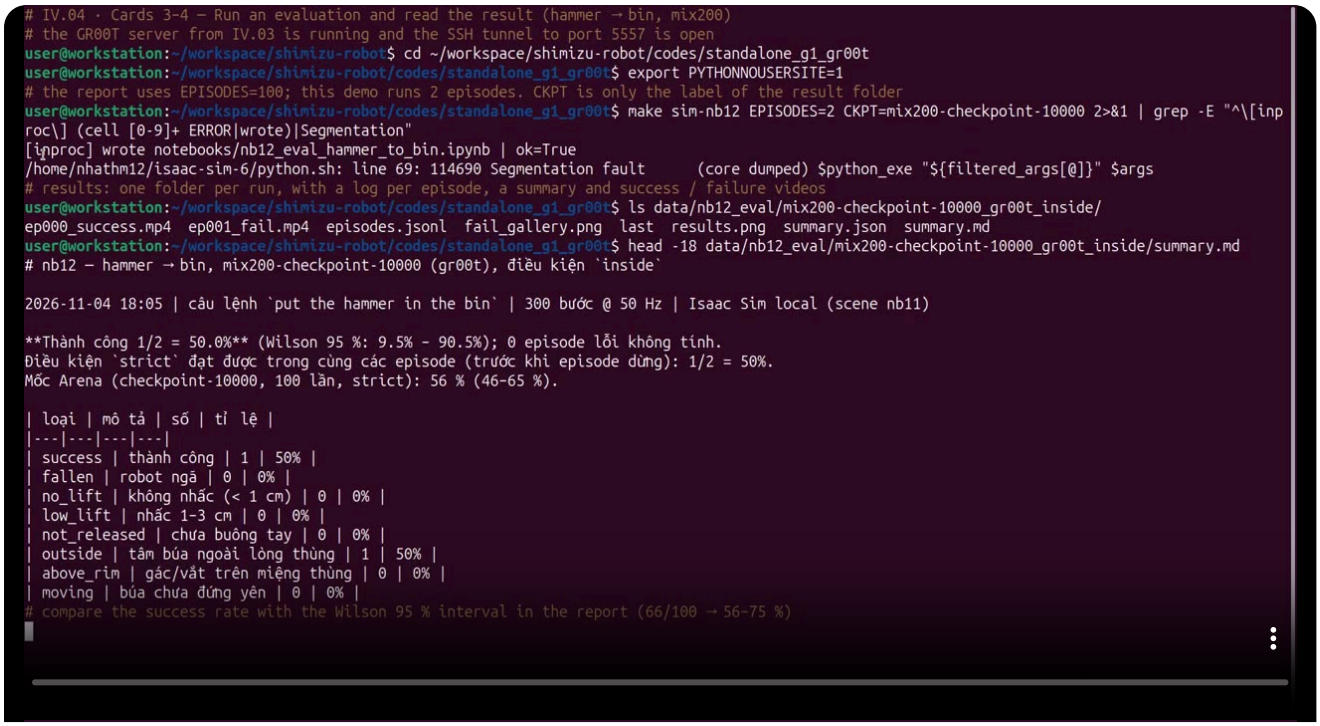
モデルの実行に時間をかける前に、シミュレーション、ロボット、シーン、採点方法が正しく動作することを確認します。



サンプル動画：`make sim-stand` でシミュレーションウィンドウを開き、ロボットが 500 tick 立ち続けます。続いてハンマーのデータセットのデモ 0 を再生し（headless）、集計 1/1 を確認します。シミュレーションの実行部分は早送りしています。

コマンド	内容	合格条件
`make sim-stand`	ロボットが 500 tick 立つ、ウィンドウあり	ロボットが安定して立ち、転倒しない
`make sim-replay`	リンゴのデータセットのエピソード 0 のデモを再生、ウィンドウあり	ロボットがリンゴをつかんで皿に置く
`make sim-nb12 EPISODES=1 POLICY=replay CKPT=replay`	ハンマー → 箱のシーンで mix200 データセットのデモ 0 を再生	集計が 1/1 成功を報告

3 シナリオごとの検証コマンド



サンプル動画：IV.03 節のサーバーを使って nb12（ハンマー → 箱、mix200）を実行し、カード 4 のとおり結果フォルダと `summary.md` を確認します。動画は短くするため 2 エピソードのみ実行しています（結果 1/2）。報告書の数値は 100 エピソードを使用しています。シミュレーションの部分は早送りしています。

```
cd ~/workspace/shimizu-robot/codes/standalone_g1_gr00t

# リンゴ → 皿、Galileo シーン（リンゴのサーバー、ポート 5556）
make sim-nb07 EPISODES=50

# Real2sim 果物 4 種（real2sim のサーバー、ポート 5556）
make sim-nb10 EPISODES=50

# ハンマー → 箱（ハンマーのサーバー、ポート 5557）。CKPT は結果フォルダ名のラベルにすぎない
make sim-nb12 EPISODES=100 CKPT=mix200-checkpoint-10000
make sim-nb12 EPISODES=100 CKPT=mix400-checkpoint-20000
```

シナリオ	ノートブック · 所要時間	報告書の結果
リンゴ → 皿、Galileo	`nb07_full_pipeline_loop` 約 45 s / エピソード	**56 %** · 28/50 · Wilson 42–69 %
Real2sim 果物 4 種	`nb10_eval_multi_object` 約 1–2 分 / エピソード	**66 %** · 33/50 · Wilson 52–78 %
ハンマー → 箱、mix200	`nb12_eval_hammer_to_bin` 約 15–30 s / エピソード	**66 %** · 66/100 · Wilson 56–75 %
ハンマー → 箱、mix400	同上	**73 %** · 73/100 · Wilson 64–81 % Arena の静止しきい値では 63 % · 63/100

ハンマーの静止しきい値：現在の nb12 は、速度 < 0.02 m/s かつ角速度 < 0.12 rad/s のときにハンマーが静止したとみなします（§3 の変数 `LIN_VEL_TH` 、 `ANG_VEL_TH` ）。73 % の結果はこのしきい値で測定したものです。63 %（Arena 本来のしきい値、0.01 m/s と 0.1 rad/s）を再現するには、ノートブック内のこの 2 つの変数を変更し、別の `CKPT` ラベルで再実行してください。

上記コマンドの対象外：リンゴのシナリオの 92 % と Arena でのハンマーの 56/100 は、Docker 内の Arena 本来のクライアントで測定したもので、スタンドアロンノートブックは使っていません。

4 結果の読み方と比較

ノートブック	結果フォルダ	成功率の記載場所
nb07	`data/runs_galileo.jsonl` 、動画は `data/videos/gr00t_galileo/`	ノートブック末尾の集計表
nb10	`data/nb10_eval/` （ `NB10_OUT` で変更）	`seed0_….summary.json`
nb12	`data/nb12_eval/<CKPT>_gr00t_inside/`	`summary.json` 、 `summary.md`

報告書と一致

信頼区間の範囲内

各エピソードはランダム化されています。50 エピソードでは約 ±13 ポイント、100 エピソードでは約 ±9 ポイントの差は正常です。

技術的なエラー

成功率には含めない

エラーになったエピソード（サーバーのタイムアウト、接続切れ）は理由を記録して成功率から除外します。サーバーが応答しなくなるとノートブックは停止します。

再開

最初からやり直す必要はない

nb10 と nb12 は結果フォルダに既にあるエピソードをスキップします。サーバーを再起動し、同じコマンドを再実行してください。

5 Jupyter での対話実行

各ステップやシミュレーションウィンドウを確認したい場合は、headless で実行する代わりに Jupyter でノートブックを開きます。

```
make sim-kernel && make nb     # Jupyter Lab、カーネル "Isaac Sim Python 3 (isaacsim6)" を選択
```

症状	対処
`No module named 'isaaclab'`	`.venv` カーネルになっている：`isaacsim6` カーネルに切り替える
`print` がセルに表示されない	Isaac Lab が `sys.stdout` を置き換えている。Jupyter を起動したターミナルで出力を確認
シミュレーション終了時にハングする	`app.close()` を呼ばず、Kernel ▸ Restart を使う

IV.05 VSS による動画評価

視覚言語モデルに動画を見せ、結果を自動で判定させる。

NVIDIA VSS（Video Search & Summarization）は、視覚言語モデル（VLM）で動画を分析するツール群です。本節では GPU サーバーへの VSS 3.2.1 のインストール手順を示し、**Isaac Sim → VSS ブリッジ**を使ってハンマー → ビンのエピソード動画を VSS に見せ、成功か失敗かを判定させて Arena の判定と比較します。各ステップにはコマンド、期待される結果、実際の操作を録画した動画があります。

バージョン
VSS 3.2.1 · LVS プロファイル · H100 ×1

3 つの判定方法
動画要約 · Q&A · リアルタイムアラート

結論
リアルタイムアラート、`top_k: 1`：94.2 %（要確認）

1 本プロジェクトにおける VSS の役割

Arena はシミュレーションの状態（ハンマーとビンの位置）からエピソードを判定します。実機ロボットにはそのような状態がなく、あるのは動画だけです。VSS は次の問いに答えるために試しました：**動画だけを見て、VLM は Arena と同じ判定を下せるか。**

01 · エピソード
動画 + Arena の判定
ハンマー → ビンのシナリオ（III.03 節）

→

02 · 書き出し
15 fps、480p
最終状態が VLM に見えるよう最後のフレームを 1.5 秒保持

→

03 · VSS
VLM が動画を見る
Cosmos3 nano reasoner を GPU で実行、Nemotron LLM は build.nvidia.com 経由

→

04 · 判定
成功 / 失敗
イベント、YES/NO の回答、またはアラートから

→

05 · 比較
Arena の判定と比較
カード 5

コンテナ
26
常時稼働 17 · 起動時のみ 9

VLM の GPU
~44 GB
Cosmos3 モデル 16.3 GB · vLLM は GPU メモリの 0.5 まで

ディスク
~80 GB
Docker イメージ + モデル、最大 42.9 GB

インストール
~12 分
インストールと初回のモデル読み込み

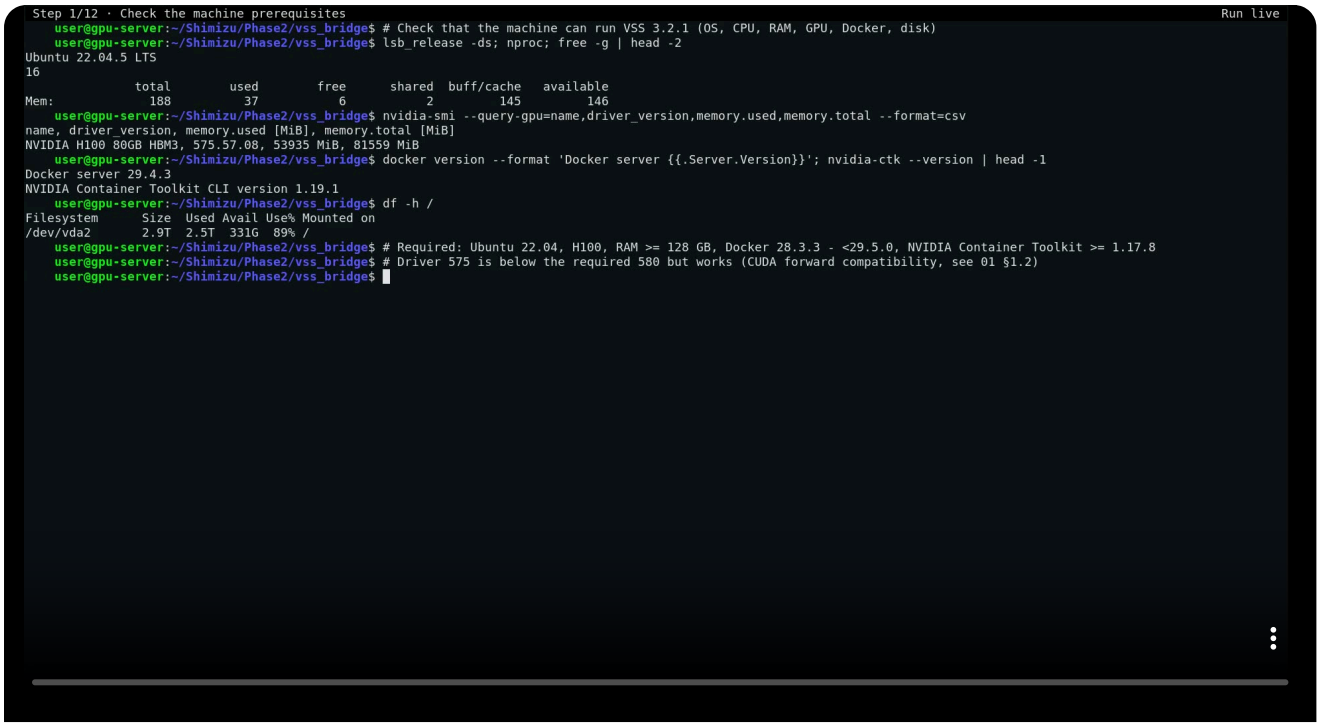
手順書と動画：フォルダ `vss_guide/` には手順書 `07_step_by_step_guide.md` と 12 本の動画（1 ステップ 1 本、冒頭に 5 秒のタイトルカード）があります。動画のモードは 2 つです：**実行**（録画中にコマンドを実際に実行）と **確認のみ**（インストール時に実施済みで、再実行すると害があるため結果の確認だけを行う）。本報告書の動画では、サーバー名と IP アドレスを `user@gpu-server` と `<server-ip>` に置き換えています。

2 サーバーの要件

インストール前に、サーバーを VSS 3.2.1 の要件と比較します。中央の列は録画に使ったマシンです。

要件	録画時のマシン	備考
Ubuntu 22.04 または 24.04	Ubuntu 22.04.5 LTS	OK
CPU 18 コア以上	16 コア	やや不足だが動作する
RAM 128 GB 以上	188 GB	OK
H100 GPU	H100 80 GB HBM3	GPU を共有：VSS 約 44 GB、残りは GR00T などに使用
ドライバー 580.65.06	575.57.08	アップグレードせず、CUDA forward compatibility で動作
Docker 28.3.3 以上 29.5.0 未満	29.4.3	OK
NVIDIA Container Toolkit 1.17.8 以上	1.19.1	OK
ディスク空き容量	約 331 GB	インストールで約 80 GB 使用、Elasticsearch は 15 GB 超の空きが必要

```
lsb_release -ds; nproc; free -g | head -2
nvidia-smi --query-gpu=name,driver_version,memory.used,memory.total --format=csv
docker version --format 'Docker server {{.Server.Version}}'; nvidia-ctk --version | head -1
df -h /
```



ステップ 1 · 実行：OS、CPU、RAM、GPU、ドライバー、Docker、Container Toolkit、ディスク空き容量を確認。

3 VSS 3.2.1 のインストール

5 つのステップをこの順に行います。ステップ 3 と 6 は root 権限が必要で共有サーバーを中断させるため、管理者が事前に告知した時間に実施します。

02 · キー
NGC Personal Key
nvcr.io にログイン
→
03 · DOCKER
29.4.3 にダウングレード
管理者 · バージョンを固定
→
04 · ソース
v3.2.1 を clone
パッチ 4 件を適用
→
05 · 試行
Dry-run
一時的な clone でのみ
→
06 · インストール
イメージ取得、起動
26 コンテナ、モデル 16.3 GB

```
# ステップ 2：キーは専用ファイルに権限 600 で保存。チャットに貼らず、commit しない
mkdir -p ~/.config/vss && umask 077
cat > ~/.config/vss/keys.env <<'EOF'
export NGC_CLI_API_KEY='nvapi-...'
export NVIDIA_API_KEY='nvapi-...'
EOF
bash -c 'source ~/.config/vss/keys.env; echo "$NGC_CLI_API_KEY" | docker login nvcr.io -u "\$oauthtoken'

# ステップ 3（管理者）：NVIDIA が検証した範囲の Docker
VER=5:29.4.3-1~ubuntu.22.04~jammy
sudo apt-get install -y --allow-downgrades docker-ce=$VER docker-ce-cli=$VER
sudo apt-mark hold docker-ce docker-ce-cli

# ステップ 4：リリース 3.2.1 をそのまま取得し、パッチを適用
```

```
git clone --branch v3.2.1 https://github.com/NVIDIA-AI-Blueprints/video-search-and-summarization.git vss
cd vss && git apply /path/to/local-patches.diff

# ステップ 6：インストール（LLM はリモート、VLM はサーバーの GPU で実行）
set -a && source ~/.config/vss/keys.env && set +a
export LLM_ENDPOINT_URL=https://integrate.api.nvidia.com RTVI_VLLM_GPU_MEMORY_UTILIZATION=0.5
bash deploy/docker/scripts/dev-profile.sh up -p lvs -H H100 -i <server-ip> \
  --use-remote-llm --llm nvidia/nemotron-3.5-lightning-30b-a3b
```

#	ファイル	パッチ（`LOCAL-PATCHES.DIFF`）
1	`dev-profile.sh`、`state_down`	削除するのは接頭辞 `mdx_` の未使用ボリュームのみ。他のユーザーのボリュームは削除しない
2	`dev-profile.sh`、`state_up`	カーネル設定の変更（ホスト全体の IPv6 無効化）は `VSS_APPLY_KERNEL_SETTINGS=true` のときだけ
3	`dev-profile.sh`、`state_up`	事前に設定した `RTVI_VLLM_GPU_MEMORY_UTILIZATION`（0.7 ではなく 0.5）を維持し、GR00T 用の GPU メモリを残す
4	`vss-agent/configs/config.yml`	エージェントが `openai` クライアント経由で LLM を呼ぶようにし、400 `verify_ssl` エラーを修正

`dev-profile.sh up --dry-run` **は無害ではありません：** `generated.env`（キーと LLM 設定を含むファイル）を上書きし、`data-dir` で `chmod -R 777` を実行します。Dry-run は一時的な clone でのみ行ってください（ステップ 5 の動画）。また `dev-profile.sh up` は必ず先に `down -v` を実行し、**17.8 GB のモデルボリュームを削除**します。インストール済みのマシンでは再実行しないでください。

4 インストール後の設定とヘルスチェック

最初のリクエストで初めて表面化する問題が 2 つあるため、インストール直後に修正します：Elasticsearch のディスク閾値（既定では約 150 GB の空きが必要）と、エージェントの LLM 呼び出し方法（パッチ 4、その後 `docker restart vss-agent`）です。

```
# Elasticsearch のディスク閾値：何度実行しても安全
curl -s -X PUT 'http://127.0.0.1:9200/_cluster/settings' -H 'Content-Type: application/json' -d '{"persi
  "cluster.routing.allocation.disk.watermark.low":"15gb",
  "cluster.routing.allocation.disk.watermark.high":"10gb",
  "cluster.routing.allocation.disk.watermark.flood_stage":"5gb",
  "cluster.routing.allocation.disk.watermark.flood_stage.frozen":"5gb"}}'

# generated.env にはキーが含まれる：必要な変数名だけを grep し、cat はしない
E=../vss/deploy/docker/developer-profiles/dev-profile-lvs/generated.env
grep -E '^(LLM_MODE|VLM_MODE|LLM_NAME|VLM_NAME|RTVI_VLLM_GPU_MEMORY_UTILIZATION)=' $E
```

確認項目	合格の結果
プロジェクト `mdx` のコンテナ	`Up` 17（ヘルスチェックがあるものは `healthy`）、`Exited (0)` 9
LVS サービス、ポート 38111	`200`、モデル `nim_nvidia_cosmos3-nano-reasoner_bf16-final`
VLM、ポート 8018、「Say OK」	`OK` と回答（`EngineDeadError` ではない）
エージェント、ポート 8000、「hi」	約 17 秒後にエージェントの挨拶
Elasticsearch、ポート 9200	`green`、閾値 15 / 10 / 5 GB
Kafka、トピック `mdx-vlm-incidents`	8 パーティション、アラート件数を読み取れる
GPU	`VLLM::EngineCore` プロセス 約 43.7 GB
Web UI、ポート 7777	`200`
ブリッジの `tests/smoke_test.sh`	`SMOKE TEST PASSED`（モック使用、VSS は呼ばない）

> **よくある障害：**ヘルスチェックが `healthy` のまま VLM エンジンが停止 → `docker restart vss-rtvi-vlm`。Kafka が停止 → `docker start kafka`。Elasticsearch が `green` でない → 多くはディスク不足。

5 3 つの判定方法と 99 エピソードでの結果

ブリッジには VSS の 3 つの仕組みに対応する 3 つのプログラムがあります。それぞれ YAML の設定ファイルを読み、エピソードごとの結果（`epNNN.json`）と集計表 `report.md` を書き出します。

判定方法	仕組み	時間	デコード
動画要約（LVS） `vss_bridge.run_pipeline`	動画を `/v1/summarize`（ポート 38111）に送信。VLM が要約とイベントのタイムラインを書き、ブリッジが厳密なルールでイベントから判定を読み取る	約 22 秒 / エピソード	既定（ランダムサンプリングあり）
Q&A `vss_bridge.qa_agent`	エージェント（ポート 8000）が VLM に質問：「動画の終了時、ハンマーは完全にビンの中にあるか？ YES/NO」	約 82 秒 / エピソード	既定（ランダムサンプリングあり）
リアルタイムアラート（RT alert） `vss_bridge.rt_alert`	各エピソードを固定の 8 秒枠に収める。VLM（ポート 8018）が「失敗はあるか？ Yes / No」に回答。Yes = アラート = 失敗	ライブストリームの 8 秒区間ごとに約 0.7 秒	既定、修正版：`top_k: 1`

評価には Arena の判定が付いた**ハンマー → ビンの 99 エピソード**を使いました。13 エピソードは質問と結果の読み取り方の調整用、**86 エピソードは判定専用に取り分けたもの**です。比較の基準は常に「成功」と答える方法で、ほとんどのエピソードが成功するため 63/86（73.3 %）正解します。この基準を上回って初めて判定方法として役に立ちます。

固定の 8 秒枠が重要な理由：成功したエピソードは成功条件を満たした時点で止まるため、クリップが短くなります。動画要約と Q&A では、クリップの長さだけで 86 エピソードすべての結果を当てられてしまいます。リアルタイムアラートはすべてのエピソードを同じ長さにそろえるためこの漏れがなく、これがその結果を最も公平な試験とする理由の一つです。

判定方法 · 取り分けた 86 エピソード	ARENA との一致	成功の認識	失敗の認識	基準を上回るか
動画要約	52/86（60.5 %）	56 %	74 %	いいえ
Q&A	51/86（59.3 %）	51 %	83 %	いいえ
リアルタイムアラート、同じ設定で 5 回実行	74–80 %	83–89 %	35–65 %	いいえ、実行ごとに異なる
リアルタイムアラート、`top_k: 1`	**81/86（94.2 %）**	**100 %**	**78 %（18/23）**	**はい（p < 0.001）、結果を知った後に測定**
基準：常に「成功」と答える	63/86（73.3 %）	100 %	0 %	–

結果がばらつく原因（確認済み）：同じ設定・同じフレームでも、リアルタイムアラートの実行同士で 86 エピソード中 28 件の判定が食い違いました。1 つのクリップを 8 回送ると「Yes」が 5 回、「No」が 3 回でした。リアルタイム VLM サービスは `temperature` が 0 でないときだけモデルに渡すため、`temperature 0` は捨てられ、モデルは既定の 1.0 でサンプリングします。`top_k: 1` を加えると決定的なデコードになり、2 回の実行が 99 エピソードすべてで一致し、Arena との一致率は 94.2 % に上がりました。成功はすべて、失敗は 23 件中 18 件を認識しています。

94.2 % は有力な手がかりであり、まだ確定した結果ではありません。質問は変えていませんが、この実行は取り分けたエピソードの結果を知った後に行いました。調整用の 13 エピソードでは 8/13 しか正解せず、ロボットの転倒 2 件をどちらも見逃しています。取り分けたエピソードには転倒が 1 件もありません。新しいデータでの実行で確認する必要があります。動画要約と Q&A も同じデコードの欠陥の影響を受けており、まだ `top_k: 1` で再実行していないため、3 つの方法の比較はまだ公平ではありません。

- **Q&A** は成功 63 件中 31 件を「ハンマーは一部だけビンの中」などの回答で失敗と判定しました。2 つのグループの最終フレームと Arena の測定値はほぼ同じです。原因として考えられるのは、ビンの底が浅いことと、質問中の「完全に」（*fully*）という語です。
- **VSS のエージェントは質問を書き換えて**から VLM に渡しており、それが 99 件中 46 件ありました。実際に尋ねた質問は一定ではありませんでした。

6 2 エピソードでの 3 つの方法の再実行（ステップ 9–11 の動画）

手順書では 2 つのエピソード、ep003（Arena：成功）と ep025（Arena：失敗、ハンマーがビンの外）で 3 つのコマンドを録画しました。各エピソードは同じ（ビット単位で同一の）動画と同じ設定で複数回実行しています：以前に保存した実行、ベトナム語版の録画、英語版の録画です。Q&A は UI デモ動画でもう 1 回実行しています。

```
cd <vss_bridge フォルダ>
python3 -m vss_bridge.run_pipeline --config configs/guide_en_lvs.yaml \
  --inputs ../data/W8/vss_inputs/ep003.mp4 ../data/W8/vss_inputs/ep025.mp4      # 動画要約
python3 -m vss_bridge.qa_agent --config configs/guide_en_qa.yaml --episodes ep003,ep025   # Q&A
python3 -m vss_bridge.rt_alert --config configs/guide_en_rt.yaml --episodes ep003,ep025   # アラート、top
cat out/guide_en/{lvs,qa,rt}/report.md
```

エピソード · ARENA	動画要約	Q&A	リアルタイムアラート
ep003 成功	成功、成功、**失敗** 3 回中 2 回正解	NO、YES、NO、YES 4 回中 2 回正解	No、No、No **3 回中 3 回正解**
ep025 失敗（ハンマーがビンの外）	空 → 失敗、「ハンマーは完全にビンの中」→ **成功**、「ハンマーは持ち上げられていない」→ 失敗 3 回中 2 回正解、正しい説明は 0 回	毎回 NO 毎回正解	Yes、Yes、Yes **3 回中 3 回正解**

録画の結果はカード 5 の全体評価と一致します：動画要約と Q&A は実行ごとに答えが変わり、`top_k: 1` のリアルタイムアラートは毎回同じ答えを返します。動画内の時間（要約 19–21 秒、Q&A 52–54 秒、アラート 0.3–0.45 秒 / エピソード）は録画時のもので、全体評価の数値ではありません。



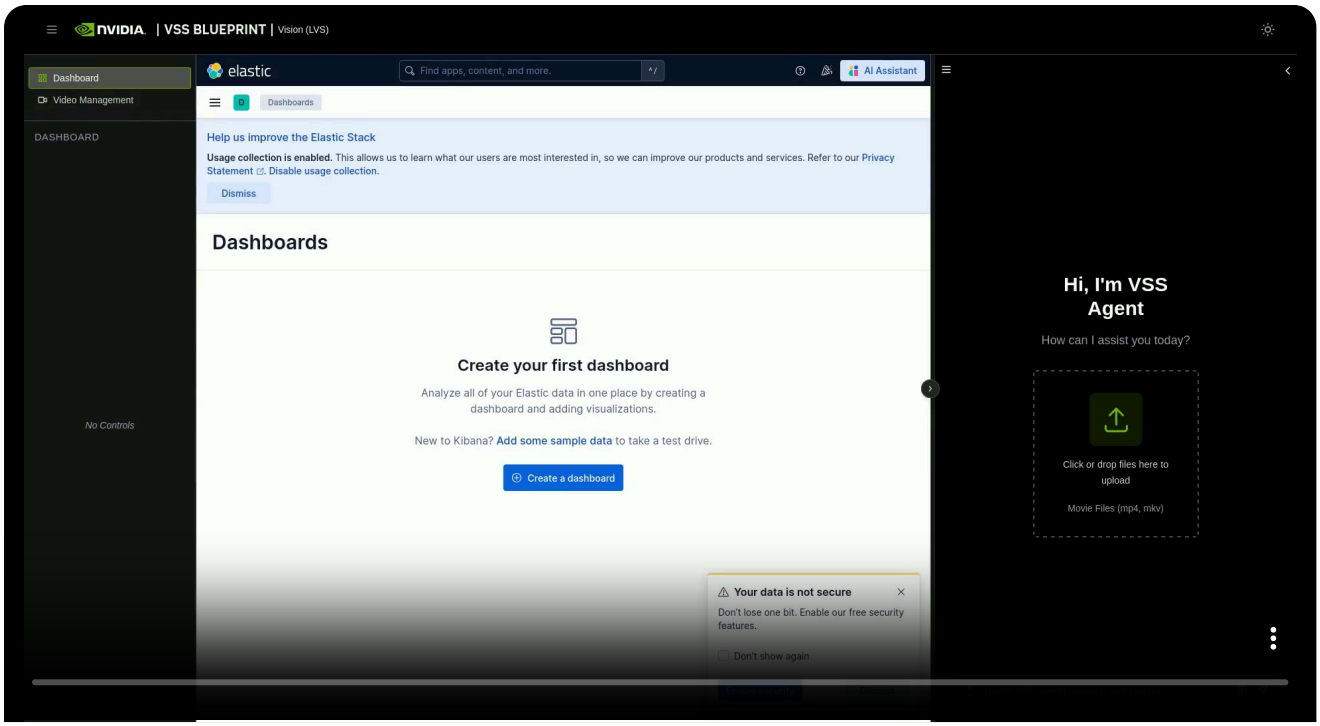
ステップ 9 · 実行：ep003 と ep025 を要約し、`report.md` と VLM が実際に受け取ったプロンプトを確認。43 秒の待ち時間は 6 倍速。

7 Web UI

ポート 7777 の UI では、アップロード済みの動画を見たり、エージェントに手動で質問したりできます。UI はサーバーの内部アドレスか `localhost` でのアクセスしか受け付けないため、ワークステーションからは SSH トンネル経由で開きます：

```
ssh -N -L 7777:127.0.0.1:7777 user@gpu-server     # その後 http://localhost:7777 を開く
```

動画内の時刻	画面
0:05	**Dashboard**：ページに埋め込まれた Kibana、ダッシュボードは未作成
0:11	**Video Management**：アップロード済みの動画 99 本。動画のアップロード、RTSP ストリームの追加、検索、各動画の「+ Chat」ボタン
0:17–0:21	`ep003` を検索して再生：2 つの半画面（ロボットの頭部カメラ、三人称視点）
0:34	**Chat** パネルを開く：「Hi, I'm VSS Agent」



ステップ 12 · 実行：Web UI の一巡。この動画では質問は送っていません。

8 発見事項と対応

#	発見事項	対応	担当
1	`dev-profile.sh --dry-run` でも `generated.env` を上書きし、`data-dir` の権限を変更する	Dry-run は一時的な clone でのみ（ステップ 5）	スクリプトを実行する人
2	`docker-ce`、`docker-ce-cli` が現在固定されていない。`apt upgrade` で Docker が検証済みの範囲外になりうる	`sudo apt-mark hold docker-ce docker-ce-cli`	管理者
3	VLM サービスが `temperature 0` を捨てて 1.0 でサンプリングする：同じ動画でも答えが変わる	3 つの方法すべてで `top_k: 1` を送る。動画要約と Q&A を `top_k: 1` で再実行	技術リーダー
4	94.2 % は取り分けたエピソードの結果を知った後に測定。そのエピソードにはロボットの転倒がない	転倒を含む新しいエピソードでの実行で確認	技術リーダー
5	エージェントが 99 件中 46 件で質問を書き換えた。Q&A は成功 63 件中 31 件を失敗と判定	VLM に送る質問を固定し、「完全に」の代わりに「ビンの中」を明確に定義する	技術リーダー
6	ヘルスチェックが `healthy` のまま VLM エンジンが 10 月 19 日から停止していた。ディスク満杯で Kafka が止まり、Elasticsearch が読み取り専用になった	ヘルスチェックだけでなく、実際の VLM 呼び出しとディスク使用量で監視する	管理者

結論：現在の設定では、動画要約と Q&A は常に「成功」と答える基準（73.3 %）を上回りません。`top_k: 1` のリアルタイムアラートは取り分けた 86 エピソードで 94.2 % に達し、高速（8 秒区間ごとに 1 秒未満）で安定しており、今後追うべき方向です。VSS で Arena の判定を置き換えたり補ったりする前に、新しいエピソードでの確認の実行と、残る 2 つの方法の `top_k: 1` での再実行が必要です。

9 日常の運用

以下は `vss/deploy/docker` で、`E=developer-profiles/dev-profile-lvs/generated.env` として実行します。

作業	コマンド
ログを見る	`docker logs --tail 200 vss-lvs`（または `vss-rtvi-vlm`、`vss-agent`）
データを残したまま全体を停止 / 起動	`docker compose --env-file $E stop` / `start`
GPU を空ける	`docker stop vss-rtvi-vlm`、再開は `docker start vss-rtvi-vlm`
Docker デーモンの再起動後	`docker start kafka phoenix`（この 2 つは自動で再起動しない）
LLM モデルの変更	`$E` の `LLM_NAME` を編集し、`docker compose --env-file $E up -d --no-deps --force-recreate lvs-server vss-agent`
定期確認	VLM を実際に呼び出し（カード 4 の表の VLM の行）、`df -h /` を確認する。ヘルスチェックは VLM の停止を検出できない。ディスク満杯で Kafka が止まり、Elasticsearch が読み取り専用になる（承認を得て古いデータ 79 GB を削除した）

インストール済みのマシンで `dev-profile.sh up` や `down` を**実行しないでください**：17.8 GB のモデルボリュームが削除され、sudo も必要です。`generated.env` や `keys.env` を `cat` したり、`grep` なしで `docker compose config` や `docker inspect` を実行したりも**しないでください**：キーが画面に表示されます。

V.01 総合評価

3 つのシナリオ、1 つの全体像。

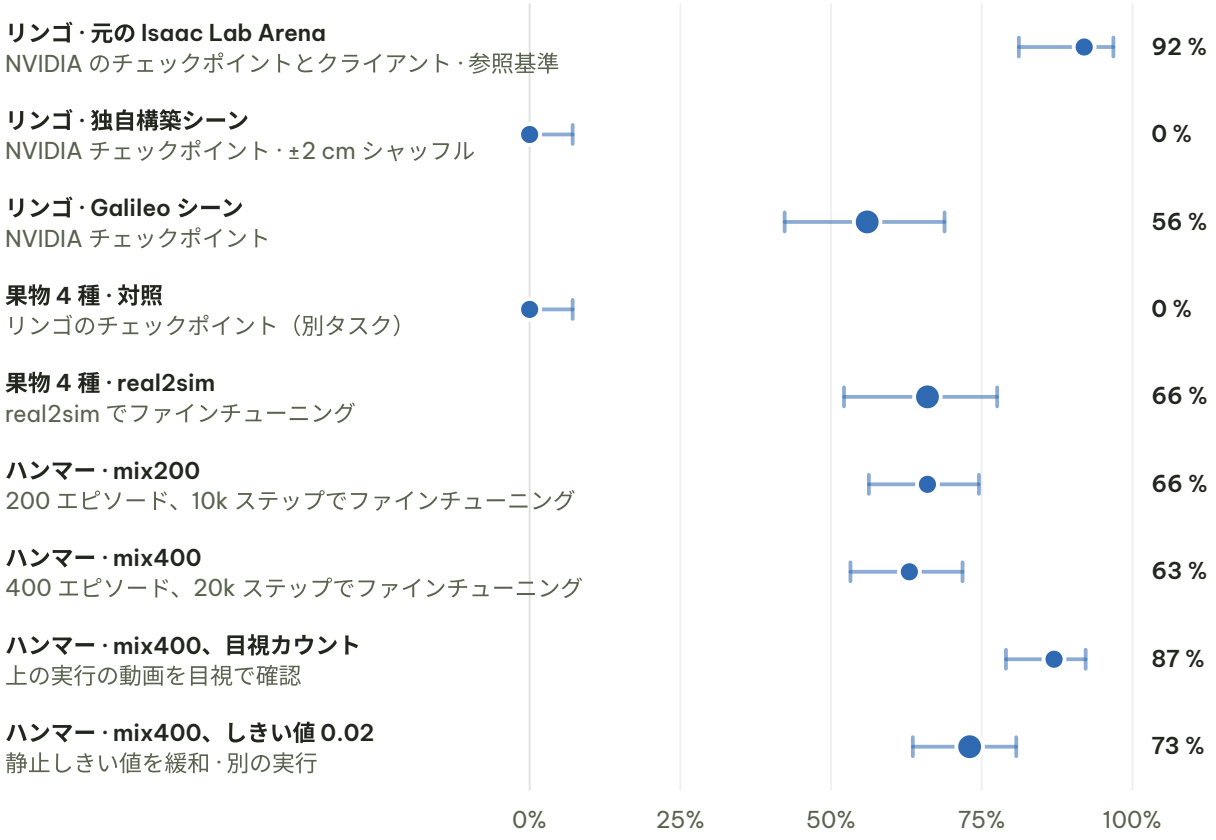
GR00T N1.7 はシミュレーション内で G1 によるピック&プレースを制御できました：リンゴのタスクで **56 %**、果物 4 種で **66 %**、ハンマーのタスクで **66–73 %**。結果を左右するのはまず**シーンの見た目**と**ファインチューニングのデータ**であり、採点方法はその次です。

1 結果のまとめ

シナリオ	対策前	最良の結果	決定要因	残るボトルネック
III.01 · リンゴ → 皿	0 %（自作シーン）	**92 %** · 46/50 · 元の Isaac Lab Arena、NVIDIA クライアント **56 %** · 28/50 · 再構築した Galileo	データ記録時と同じシーン	認識と接近のフェーズ：見慣れないシーンではリンゴを持ち上げられたのは 22 % のみ
III.02 · 果物 4 種	0 %（リンゴのチェックポイント）	**66 %** · 33/50 · real2sim	real2sim 790 エピソードでのファインチューニング	把持のフェーズ：失敗 17 件中 11 件は果物を持ち上げられなかった
III.03 · ハンマー → 箱	**0 %（ファインチューニング前）**	**66 %** · 66/100 · mix200 **73 %** · 73/100 · mix400、しきい値 0.02 **87 %** · 87/100 · mix400、動画からの目視カウント	**自ら収集したデータ：テレオペ + 追加生成**	**解放のフェーズと「静止」の採点方法**

成功率と Wilson 95 % 信頼区間

点 = 測定した成功率、横棒 = 信頼区間。マウスを重ねるか Tab キーで詳細を表示。



注記：元の Isaac Lab Arena 環境での NVIDIA クライアントはリンゴのタスクで 92 %（46/50）に達しました。この実行は参照基準です（グラフの先頭行）。その他の評価はすべて、自作パイプラインとローカルの Isaac Sim による閉ループで実行しました。

動画による判定（IV.05 節）：ハンマー → ビンのエピソード動画 99 本を NVIDIA VSS に見せ、成否を自動で判定させました。取り分けた 86 エピソードでは、動画要約（60.5 %）と Q&A（59.3 %）は常に「成功」と答える基準（73.3 %）を上回りませんでした。デコードの修正（ `top_k: 1` ）後、リアルタイムアラートは Arena と 94.2 % 一致しました。この実行は結果を知った後に行ったため、新しいデータでの確認が必要です。

2 全体的な考察

01 · 画像

モデルよりもシーンの影響が大きい

同じチェックポイントと指示文で、リンゴのタスクは 92 %（NVIDIA クライアント、元の Arena）から 56 %（照明のみの違い）、さらに 0 %（シーン全体の違い）へ低下しました。1 つのシーンでファインチューニングしたモデルは別のシーンへ汎化しません。

02 · FINE-TUNE

新しいタスクにはチューニングが必須、そして短時間で済む

果物のタスクはリンゴのチェックポイントを流用すると 0 %、ハンマーのタスクはチューニング前で 0 % でしたが、どちらもファインチューニング後に 66 % に達しました。ハンマーのタスクは 200 エピソードと H100 1 枚で約 1 時間しか要さず、主なコストはデータにあります。

03 · 言語

モデルは指示文どおりに動作する

テーブル上に常に別の果物や妨害物体があったにもかかわらず、果物のタスク 50 エピソードで誤った物体をつかんだのは 0 回でした。言語の部分は real2sim データからよく学習されています。

04 · ボトルネック

シナリオごとに失敗するフェーズが異なる

リンゴは認識と接近、果物は把持、ハンマーは解放のフェーズで失敗しています。そのため改善の方向性はタスクごとに考える必要があり、共通の解決策はありません。

05 · データ

同じ種類のデータを増やす効果には限界がある

ハンマーのタスクで 200 から 400 エピソードに増やすと配置の精度は上がりました（箱の内側から外れた回数 19 → 5）が、全体の成功率は上がりませんでした（66 % → 63 %、誤差の範囲内）。

06 · 採点

採点方法が数値に影響する

ハンマーのタスク：100 回中 22 回は、ハンマーが箱の中にあり手も離していたのに、条件が瞬間速度のみを見るため「静止していない」と判定されました。しきい値を 0.02 m/s に緩和した実行では 73 % に達しました。

3 得られた教訓

01

モデルよりもデータ & 環境の影響が大きい

同じチェックポイントでも、シーンの違いだけで 0 % か 56 % になる。

02

指示文は「契約」の一部

指示文は管理された設定パラメータとして扱う。

03

段階的な検証で時間を節約できる

Open-loop → 再生 → 閉ループ。

04

成功条件はテストが必要

緩い条件はデータを汚し、厳しすぎる条件は指標を下げる。比較の前にしきい値を統一する。

05

評価には多くの試行が必要

構成ごとに 50–100 回の試行を行い、常に信頼区間を併記する。

06

データの追加生成は低コスト

133 回の生成 → 厳格な条件を満たすデモ 100 件、テレオペの追加時間は不要。

07

ファインチューニングは速く低コスト

H100 1 枚で約 1 時間（200 エピソード）〜 2 時間 15 分（400 エピソード）。

4 今後の方向性

本フェーズでは、GR00T N1.7 がシミュレーションで G1 を制御してピック＆プレースできることを示し、新しいタスクのためのデータ → ファインチューニング → 評価のプロセスを構築しました。次のステップは 3 つのグループに分かれます：**モデルとセンシングの調査**、**シミュレーションと現実の見た目の差の縮小**、そして**実機ロボットでモデルを動かす準備**（フェーズ 2）です。

01 · 調査 · モデル

より優れた他のモデルの調査

新しい GR00T のバージョンや他のオープンソース VLA モデル（例：π0、OpenVLA）を、同じ 3 つのシナリオと同じ判定方法で評価し、GR00T N1.7 の位置づけを把握します。スタンドアロンパイプラインは ZMQ サーバー経由でモデルと通信するため、新しいモデルにはサーバーを用意するだけで済み、シーン、ランナー、評価ノートブックはそのまま再利用できます。

02 · シミュレーション · 画像

シーンのドメインランダム化

データ生成時に照明、材質、背景、カメラ、物体の位置をランダム化し（手法は III.01 節）、モデルが 1 つのシーンに依存しないようにします。リンゴのタスクは照明の違いだけで 92 % から 56 % に下がっています。これは sim2real の前提となるステップです。

03 · シミュレーション · 再構築

3D Gaussian Splatting による実環境の再構築

実際の作業環境を写真や動画で撮影して 3D Gaussian Splatting で再構築し、シミュレーションの背景として描画することで、照明や材質を現実に合わせます。リンゴのタスクでは照明の違いだけで成功率が 92 % から 56 % に下がっています。real2sim と組み合わせれば、学習データは実際の動きと実際の見た目の両方を持つことになります。Gaussian Splatting が提供するのは画像だけで、物理には机、棚、物体の衝突モデルが別途必要です。

04 · 評価 · VSS

動画による判定の確認

転倒を含む新しいエピソードで `top_k: 1` のリアルタイムアラートを再実行して 94.2 % を確認し、動画要約と Q&A も同じ修正で再実行します。確認できれば、シミュレーションの状態がない実機ロボットの自動判定に使えます（IV.05 節）。

05 · 実機 · データ

組み合わせたデータプロセス

実際の作業環境で少数のテレオペデモ → real2sim でシーンを再構築 → デモを追加生成してシミュレーションで選別 → ドメインランダム化 → ファインチューニング、シミュレーションで評価してから実機で評価（II.04 節）。3 つの手法のうち 2 つは本フェーズで検証済みです。

06 · 調査 · センシング

距離をより正確に把握するための深度マップ

モデルは頭部カメラの RGB 画像しか受け取っていませんが、失敗の多くは距離の把握に起因します：リンゴを皿から 9–20 cm 離れて置く、ハンマーがビンの外に落ちる、縁に掛かるなど。G1 頭部の RealSense D435 の深度マップ（または画像からの深度推定）を、モデルへの追加の観測として、あるいはリリース前の位置確認に使う方法を調査します。深度付きのデータ収集とファインチューニングが必要です。

優先順位：04 は最も手間が少なく、以降のすべての数値の信頼性を高めます。02 と 03 はどちらも見た目の差を対象とし、実機に移る前の必須条件です。01 と 06 はモデルの能力を高めるための調査の方向性です。05 はフェーズ 2 の中心です。今後の評価でも本フェーズのやり方を続けるべきです：構成ごとに 50–100 回の試行、常に信頼区間を添えること。

V.02 付録

用語 & 参考文献。

成果物の一覧は IV.01 節を参照してください。

A 用語

VLA	画像と言語を入力とし、ロボットの制御指令を直接出力するモデル
Foundation model	大規模データで事前学習したモデル。少量のデータで特定タスク向けにファインチューニングする
Embodiment	特定のロボットの身体の種類（関節、カメラ、手）
Action chunk	1 回の推論でモデルが返すアクション列（40 ステップ = 50 Hz で 0.8 s）
Flow matching / DiT	ノイズを段階的に除去して連続データを生成する手法。アクション列の生成に使う
Teleop	人がロボットを遠隔操作して（VR ヘッドセット）見本データを作成すること
Open-loop / closed-loop	記録済みデータでの評価 / モデルが自ら連続して制御する評価
Real2sim	実世界のデータをシミュレーションに移すこと
Domain gap	学習環境と実行環境の違い
Visual gap	実行時のシーンの見た目がデータ記録時と異なると、モデルの成功率が低下する現象（リンゴのタスク 92 → 56 → 0 %）
Domain randomization	シミュレーション内で照明、材質、背景、カメラ角度をランダムに変え、モデルが 1 つのシーンを丸暗記しないようにする手法
Demo generation	既存のテレオペデモ（時間、開始点、把持、解放）を変形してシミュレーションで再実行し、成功したデモだけを残す手法（MimicGen の方式）
FK / IK	順運動学：関節角度から手の位置を計算する。逆運動学：目標位置から関節角度を計算する
PD 制御器	比例 – 微分：トルク = Kp × (目標角度 – 現在角度) – Kd × 関節速度。減衰付きのばねのように関節を目標へ引き戻す
Rate limiter	制御ステップ（20 ms）ごとに、関節指令が前回の指令から変化できる量を一定のしきい値までに制限する（腕 0.2 rad、腰 0.1 rad、手 1.5 rad）

WBC / AGILE	全身制御器。AGILE は G1 の脚のバランスを保つ
Wilson 95 %	成功率の信頼区間。試行回数が少ない場合に適する
MAE	平均絶対誤差

B 参考文献

1. NVIDIA. *GR00T N1: An Open Foundation Model for Generalist Humanoid Robots.* arXiv:2503.14734, 2025.
2. NVIDIA. GR00T N1.7-3B および GN1x-Tuned-Arena-G1-Static-PickNPlace の Model card（Hugging Face）。
3. Isaac Lab Arena — github.com/isaac-sim/IsaacLab-Arena
4. Isaac-GR00T — github.com/NVIDIA/Isaac-GR00T
5. NVIDIA. GR00T end-to-end workflow: fine-tuning on sim data, sim evaluation (docs.nvidia.com).
6. データセット nvidia/Arena-G1-Static-PickNPlace-Task および nvidia/PhysicalAI-Robotics-GR00T-Teleop-G1（CC-BY-4.0）。
7. Black, K. et al. *π0: A Vision-Language-Action Flow Model for General Robot Control.* arXiv:2410.24164, 2024.
8. Liu, X., Gong, C. *Flow Straight and Fast: Rectified Flow.* ICLR 2023.